Daml & Canton

A Practical Guide

Vladimir Trifonov

March 25, 2026

A hands-on guide to building distributed applications
with Daml smart contracts on the Canton Network

Contents

I Quick Start **1**

1 Introduction **2**
1.1 What This Book Is About . . . 2
1.2 What Is Daml? . . . 2
1.3 What Is Canton? . . . 3
1.4 How Daml and Canton Relate . . . 3
1.5 What You Will Build . . . 4
1.6 Prerequisites . . . 4

2 Environment Setup **5**
2.1 Installing DPM . . . 5
2.1.1 What DPM Provides . . . 5
2.2 The cn-quickstart Environment . . . 6
2.2.1 Cloning and Setup . . . 6
2.2.2 What Is Running . . . 6
2.2.3 The Three Participants . . . 7
2.2.4 Local URLs . . . 7
2.2.5 Key Make Targets . . . 7
2.3 Verifying Your Setup . . . 8
2.4 Your First Daml Project (Outside cn-quickstart) . . . 8
2.4.1 Creating a Project with dpm new . . . 8
2.4.2 Tutorial Templates . . . 9

3 Your First Contract **10**
3.1 A Naive First Attempt . . . 10
3.2 Adding Dual Signatories . . . 11
3.3 The Propose-Accept Pattern . . . 12
3.4 Choices: Transforming Contracts . . . 13
3.5 Role Contracts and Delegation . . . 14
3.6 Non-Transitivity of Authority . . . 15
3.7 Building and Testing . . . 16
3.8 Summary . . . 16

4 Deploying to Canton **17**
4.1 Understanding the Deployment Target . . . 17
4.2 Method 1: Canton Console . . . 17
4.2.1 Listing Participants . . . 17
4.2.2 Uploading a DAR . . . 18
4.3 Method 2: JSON Ledger API . . . 18

4.3.1 Getting an Auth Token . . . 18
4.3.2 Uploading the DAR . . . 19
4.3.3 Verifying the Upload . . . 19
4.4 What Happens After Upload . . . 19
4.5 Next Steps . . . 19

II The Daml Language **21**

5 Templates and Signatories **22**
5.1 Anatomy of a Template . . . 22
5.2 Signatories . . . 22
5.3 Observers . . . 23
5.4 Visibility Rules . . . 23
5.5 The ensure Clause . . . 24
5.6 Contract IDs . . . 24
5.7 Finding Contracts: The Business Identifier Pattern . . . 25
5.7.1 Add a Business Identifier Field . . . 25
5.7.2 Discovering Contracts . . . 26
5.7.3 The Find-Then-Exercise Pattern . . . 26
5.7.4 What You Lose Without Contract Keys . . . 27
5.8 Putting It Together: An Asset Template . . . 27
5.9 Summary . . . 28

6 Data Types **29**
6.1 Records . . . 29
6.1.1 Nested Records . . . 30
6.2 Variants (Sum Types) . . . 30
6.2.1 Variants with Data . . . 30
6.2.2 Pattern Matching . . . 31
6.3 Enums . . . 31
6.4 Type Aliases . . . 31
6.5 Tuples . . . 31
6.6 Lists . . . 32
6.7 Optional . . . 32
6.8 The deriving Clause . . . 33
6.9 Putting It Together: Token Registry . . . 34
6.10 Summary . . . 36

7 Choices and Controllers **38**
7.1 Anatomy of a Choice . . . 38
7.2 The Expense Approval Domain . . . 38
7.3 Data Types . . . 39
7.4 The ExpenseReport Template . . . 39
7.5 Consuming Choices . . . 40
7.6 Nonconsuming Choices . . . 41
7.7 Flexible Controllers . . . 41
7.8 Return Types from Choices . . . 42
7.9 Chaining Choices in a Transaction . . . 43
7.10 Complete Test Script . . . 43
7.11 Summary . . . 45

8 Authorization Model **47**
8.1 The Authorization Rules . . . 47
8.2 Setup: The Lab Templates . . . 47
8.3 Test 1: Cannot Create Without All Signatories . . . 48
8.4 Test 2: Cannot Exercise Without Controller Authority . . . 49
8.5 Test 3: Cannot Archive Without Signatory Authority . . . 50
8.6 Test 4: Authority Flows Through Exercise Chains . . . 51
8.7 Test 5: No Authority Laundering . . . 52
8.8 Test 6: Common Pitfalls and Fixes . . . 53
8.8.1 Pitfall: Creating Contracts for Other Parties . . . 53
8.8.2 Pitfall: Modifying Contracts You Observe . . . 54
8.8.3 Pitfall: Forgetting Observer Visibility . . . 55
8.9 Summary . . . 56

9 The Propose-Accept Pattern **57**
9.1 Why Propose-Accept? . . . 57
9.2 The Job Marketplace Domain . . . 57
9.3 Data Types . . . 58
9.4 Phase 1: Job Posting . . . 58
9.5 Phase 2: Application (Propose) . . . 59
9.6 Phase 3: Offer and Acceptance . . . 60
9.7 The Final Contract . . . 62
9.8 Variation: Propose-Accept-Reject . . . 63
9.9 Variation: Propose-with-Expiry . . . 63
9.10 The Pattern as Architecture . . . 64
9.11 Complete Test Script . . . 64
9.12 Summary . . . 67

10 Composing Choices **68**
10.1 The Asset Template . . . 68
10.1.1 The Split Choice . . . 68
10.1.2 The Merge Choice . . . 69
10.1.3 The Transfer Choice . . . 69
10.2 Exercise Chains . . . 70
10.3 Authority Accumulation . . . 70
10.4 Delegation with Role Contracts . . . 71
10.5 Building an Atomic Swap . . . 72
10.5.1 The Trade Template . . . 72
10.5.2 The Accept Choice . . . 73
10.5.3 Authority in the Swap . . . 73
10.6 Testing the Swap . . . 74
10.7 Partial Swaps with Split . . . 75
10.8 Multi-Step Composition Patterns . . . 75
10.8.1 Batch Operations . . . 75
10.8.2 Conditional Composition . . . 76
10.9 Summary . . . 76

11 Error Handling and Constraints **77**
11.1 The Error Handling Spectrum . . . 77
11.2 Template Invariants with ensure . . . 77
11.3 Choice-Level Validation with assertMsg . . . 78
11.4 Unconditional Failure with abort . . . 79

11.5 Structured Error Handling with failWithStatus . . . 79
11.5.1 The FailureStatus Record . . . 79
11.5.2 Failure Categories . . . 79
11.5.3 Using failWithStatus in Choices . . . 80
11.5.4 Error ID Conventions . . . 80
11.5.5 Metadata with meta . . . 81
11.5.6 failWithStatusPure for Pure Contexts . . . 81
11.5.7 How Callers Receive the Error . . . 81
11.6 Common Error Patterns . . . 82
11.6.1 Validate-Then-Act . . . 82
11.6.2 Reusable Validation Helpers . . . 82
11.7 Choosing the Right Approach . . . 83
11.8 Testing Error Cases . . . 84
11.9 Summary . . . 85

12 Daml Script and Testing **86**
12.1 What Is Daml Script? . . . 86
12.2 Core Script Functions . . . 86
12.2.1 Party Allocation . . . 86
12.2.2 Submitting Commands . . . 87
12.2.3 Testing Failure . . . 87
12.3 Command Types . . . 88
12.4 Querying the Ledger . . . 88
12.4.1 queryFilter . . . 88
12.4.2 queryContractId . . . 89
12.4.3 queryFilter for Lookups . . . 89
12.5 Time Manipulation . . . 90
12.6 Test Organization . . . 90
12.6.1 Setup Helpers . . . 90
12.6.2 Naming Conventions . . . 91
12.6.3 Module Organization . . . 92
12.7 A Complete Test Suite . . . 92
12.7.1 Creation Tests . . . 92
12.7.2 Split Tests . . . 93
12.7.3 Merge Tests . . . 93
12.7.4 Transfer Tests . . . 94
12.7.5 Key Lookup Tests . . . 94
12.8 Running Tests . . . 95
12.9 Debugging Tips . . . 95
12.9.1 Using debug . . . 95
12.9.2 Using trace . . . 95
12.9.3 Inspecting Contract State . . . 96
12.10 Summary . . . 96

13 Standard Library and Dependencies **97**
13.1 The Standard Library . . . 97
13.2 DA.List . . . 97
13.3 DA.Optional . . . 98
13.4 DA.Map . . . 98
13.5 DA.Text . . . 99
13.6 DA.Time and DA.Date . . . 100
13.7 DA.Assert . . . 100

13.8 Multi-Package Projects . . . 101
13.8.1 Project Structure . . . 101
13.8.2 The Library Package . . . 101
13.8.3 The Test Package . . . 102
13.8.4 The multi-package.yaml File . . . 104
13.9 Package Versioning . . . 104
13.9.1 Versioning Best Practices . . . 105
13.10 Depending on Third-Party DARs . . . 105
13.11 Summary . . . 105

III Building Real Applications **107**

14 Project Structure **108**
14.1 The Full Project Layout . . . 108
14.2 Configuring multi-package.yaml . . . 109
14.3 The DAR Dependency Graph . . . 109
14.4 Package Versioning Strategy . . . 110
14.5 CI/CD Pipeline Basics . . . 110
14.6 Daml Interfaces . . . 111
14.6.1 Defining an Interface . . . 111
14.6.2 Implementing an Interface . . . 112
14.6.3 Why Interfaces Matter . . . 112
14.7 Contract Upgrades . . . 112
14.7.1 Upgrade Safety Rules . . . 112
14.7.2 Checking Upgrade Compatibility . . . 113
14.7.3 Adding an Optional Field . . . 113
14.8 Summary . . . 113

15 Ledger API and Backend Services **115**
15.1 Two Ledger APIs . . . 115
15.2 JSON Ledger API Endpoints . . . 115
15.2.1 Packages: /v2/packages . . . 115
15.2.2 Parties: /v2/parties . . . 116
15.2.3 Commands: /v2/commands/submit-and-wait . . . 116
15.2.4 Active Contracts: /v2/state/active-contracts . . . 117
15.2.5 Transaction Streams: /v2/updates . . . 118
15.3 Code Generation . . . 118
15.3.1 JavaScript/TypeScript Code Generation . . . 118
15.3.2 Java Code Generation . . . 119
15.4 Building a Backend Service . . . 119
15.4.1 Project Setup . . . 119
15.4.2 Configuration . . . 120
15.4.3 Ledger Client . . . 120
15.4.4 Command Submission . . . 121
15.4.5 Reading Contract State . . . 122
15.4.6 REST API Layer . . . 123
15.5 OAuth2 Authentication with Keycloak . . . 130
15.5.1 Obtaining a Token . . . 130
15.5.2 Token Contents . . . 130
15.6 Error Handling . . . 131
15.7 Summary . . . 132

16 PQS and Efficient Queries **133**
16.1 What PQS Is and Why It Exists . . . 133
16.2 PQS Architecture . . . 133
16.3 Setting Up PQS . . . 134
16.3.1 Verifying PQS Is Running . . . 134
16.3.2 PQS Configuration . . . 134
16.4 Querying Contracts via SQL . . . 134
16.4.1 Connecting to the Database . . . 135
16.4.2 The PQS Schema . . . 135
16.4.3 Basic Queries . . . 135
16.4.4 JSONB Queries . . . 136
16.5 When to Use PQS vs Ledger API . . . 136
16.6 Performance and Indexing . . . 136
16.6.1 Adding Indexes . . . 136
16.6.2 Query Performance Tips . . . 137
16.7 Daml Shell for Interactive Queries . . . 137
16.7.1 Starting Daml Shell . . . 137
16.7.2 Basic Commands . . . 137
16.7.3 Exploring Data . . . 138
16.8 Summary . . . 138

17 Frontend and Wallet Integration **139**
17.1 Frontend Architecture . . . 139
17.2 Generated TypeScript Types . . . 139
17.3 Connecting to the JSON Ledger API . . . 140
17.4 OAuth2 Authentication for SPAs . . . 141
17.4.1 The PKCE Flow . . . 141
17.4.2 Keycloak Configuration . . . 142
17.4.3 Integration with React . . . 142
17.5 Building the UI . . . 143
17.5.1 Listing Contracts . . . 143
17.5.2 Submitting Commands . . . 144
17.5.3 Real-Time Updates with SSE . . . 145
17.6 Wallet Integration . . . 146
17.6.1 Canton Coin Overview . . . 146
17.6.2 Wallet Operations . . . 146
17.6.3 Token Transfers . . . 147
17.7 The cn-quickstart Frontend . . . 147
17.8 Frontend Development Tips . . . 148
17.9 Summary . . . 148

18 End-to-End Example **149**
18.1 The Application: Software Licensing . . . 149
18.2 Project Structure . . . 149
18.3 The Daml Model . . . 150
18.3.1 Types Module . . . 150
18.3.2 License Template . . . 150
18.3.3 Renewal Request Template . . . 151
18.3.4 Package Configuration . . . 152
18.4 Testing the Daml Model . . . 152
18.4.1 Test Suite . . . 152
18.5 The Backend . . . 154

18.5.1 Express.js Backend . . . 154
18.5.2 REST Endpoints . . . 154
18.5.3 License Service . . . 155
18.5.4 Express Routes . . . 156
18.5.5 Spring Boot Backend (Java Alternative) . . . 157
18.6 The Frontend . . . 160
18.6.1 Application Shell . . . 161
18.7 Deployment . . . 161
18.7.1 Step 1: Build DARs . . . 161
18.7.2 Step 2: Upload to Canton . . . 161
18.7.3 Step 3: Start the Backend . . . 162
18.7.4 Step 4: Start the Frontend . . . 162
18.8 End-to-End Testing with Playwright . . . 162
18.8.1 Test Setup . . . 162
18.8.2 Writing an E2E Test . . . 163
18.8.3 Running E2E Tests . . . 163
18.9 Putting It All Together . . . 163
18.10 Next Steps . . . 164
18.11 Summary . . . 164

IV Canton Network & Production 165

19 Canton Architecture Deep Dive 166
19.1 Canton Protocol Overview . . . 166
19.2 Participant Nodes . . . 166
19.2.1 Ledger API . . . 167
19.2.2 Sync Service . . . 167
19.2.3 Contract Store . . . 167
19.3 Synchronizers . . . 168
19.4 The Sequencer . . . 168
19.4.1 Encryption Envelopes . . . 168
19.4.2 Sequencer Implementations . . . 169
19.5 The Mediator . . . 169
19.6 Sub-Transaction Privacy . . . 170
19.6.1 Transaction Views . . . 170
19.6.2 Merkle Trees and Blinding . . . 170
19.6.3 View Encryption . . . 171
19.7 Transaction Flow: Step by Step . . . 171
19.7.1 Step 1: Submission . . . 171
19.7.2 Step 2: Sequencing . . . 171
19.7.3 Step 3: Validation . . . 171
19.7.4 Step 4: Confirmation . . . 172
19.7.5 Step 5: Commit . . . 172
19.8 Comparison with Other Architectures . . . 172
19.8.1 Canton vs Ethereum . . . 172
19.8.2 Canton vs Hyperledger Fabric . . . 173
19.9 Canton's Consistency Model . . . 173
19.10 Summary . . . 174

20 The Global Synchronizer **175**
20.1 A Network of Networks . . . 175
20.2 The Global Synchronizer (sync.global) . . . 175
20.3 Super Validators . . . 176
20.3.1 Role and Responsibilities . . . 176
20.3.2 CometBFT Consensus . . . 176
20.4 Validators . . . 176
20.4.1 Onboarding Process . . . 177
20.5 Canton Coin (Amulet) . . . 177
20.5.1 Tokenomics . . . 177
20.5.2 Rewards and Issuance . . . 178
20.6 The Splice Stack . . . 178
20.6.1 Splice Components . . . 178
20.6.2 Relationship to Your Application . . . 178
20.7 Token Standard APIs . . . 179
20.7.1 Core Concepts . . . 179
20.7.2 Metadata and Holdings . . . 179
20.7.3 Transfers and Allocations . . . 179
20.8 Network Environments . . . 180
20.8.1 LocalNet (cn-quickstart) . . . 180
20.8.2 DevNet and TestNet . . . 180
20.8.3 MainNet . . . 181
20.9 Connecting Your Application . . . 181
20.10 The Scan API . . . 181
20.10.1 Querying the Scan API . . . 182
20.11 Summary . . . 182

21 Identity, Topology and Keys **183**
21.1 Party Identity at the Protocol Level . . . 183
21.2 Namespaces and Identifiers . . . 183
21.2.1 Identifier Structure . . . 184
21.3 Party Allocation . . . 184
21.3.1 Canton Console Allocation . . . 184
21.3.2 Admin API Allocation . . . 184
21.3.3 Ledger API Allocation . . . 185
21.4 Topology Transactions . . . 185
21.4.1 Types of Topology Transactions . . . 185
21.4.2 How Topology Transactions Propagate . . . 185
21.5 Key Management . . . 186
21.5.1 Signing Keys . . . 186
21.5.2 Encryption Keys . . . 186
21.5.3 Key Listing and Inspection . . . 186
21.5.4 Key Rotation . . . 186
21.6 Party-to-Participant Mapping . . . 187
21.6.1 Permission Levels . . . 187
21.6.2 Viewing Mappings . . . 187
21.7 Multi-Hosting . . . 187
21.7.1 High Availability . . . 188
21.7.2 Geographic Distribution . . . 188
21.7.3 Migration . . . 188
21.8 Namespace Delegation . . . 188

21.8.1 Use Cases . . . 188
21.8.2 Delegation Hierarchy . . . 189
21.9 Decentralized Identity . . . 189
21.9.1 Self-Sovereign Namespaces . . . 189
21.9.2 Cross-Synchronizer Identity . . . 189
21.10 Summary . . . 190

22 Operations and Deployment **191**
22.1 From Docker Compose to Kubernetes . . . 191
22.1.1 Why Kubernetes . . . 191
22.1.2 Deployment Architecture . . . 191
22.1.3 Helm Charts . . . 192
22.2 Canton Console Operations . . . 192
22.2.1 Health Checks . . . 192
22.2.2 Diagnostics . . . 192
22.2.3 Ping Test . . . 193
22.3 DAR Lifecycle . . . 193
22.3.1 Upload . . . 193
22.3.2 Vetting . . . 193
22.3.3 Unvetting . . . 194
22.3.4 Package Dependencies . . . 194
22.4 Monitoring . . . 194
22.4.1 Prometheus Metrics . . . 194
22.4.2 Grafana Dashboards . . . 195
22.4.3 Structured Logging . . . 195
22.5 Backups and Disaster Recovery . . . 196
22.5.1 What to Back Up . . . 196
22.5.2 Recovery Procedure . . . 196
22.6 Pruning . . . 196
22.6.1 What Gets Pruned . . . 197
22.6.2 Configuring Pruning . . . 197
22.7 Upgrades . . . 197
22.7.1 Participant Upgrades . . . 197
22.7.2 Synchronizer Upgrades . . . 198
22.8 Scaling . . . 198
22.8.1 Party Limits per Validator . . . 198
22.8.2 Topology Batching . . . 198
22.8.3 Horizontal Scaling . . . 199
22.9 Performance Tuning . . . 199
22.9.1 Throughput . . . 199
22.9.2 Latency Optimization . . . 199
22.9.3 Traffic Management . . . 199
22.10 Summary . . . 200

V Security & Best Practices **201**

23 Daml Security Model **202**
23.1 Secure by Design . . . 202
23.2 No Reentrancy . . . 202
23.3 No Silent Integer Overflow . . . 203
23.4 No Unauthorized Access . . . 204

23.5 Front-Door Security: The Daml Model . . . 204
23.6 Back-Door Security: The Canton Protocol . . . 205
23.6.1 Sub-Transaction Privacy . . . 205
23.6.2 Integrity Without Global Visibility . . . 205
23.6.3 Non-Repudiation . . . 206
23.7 The Authorization Model as a Security Primitive . . . 206
23.8 Sub-Transaction Privacy as Security . . . 206
23.9 Comparison: Daml vs Solidity vs Hyperledger Fabric . . . 207
23.10 What Daml Does NOT Protect Against . . . 207
23.10.1 Off-Ledger Attacks . . . 207
23.10.2 Key Compromise . . . 207
23.10.3 Social Engineering . . . 208
23.10.4 Denial of Service . . . 208
23.10.5 Side-Channel Attacks . . . 208
23.10.6 Bugs in Business Logic . . . 208
23.11 Summary . . . 208

24 Common Pitfalls and Anti-Patterns **210**
24.1 Pitfall 1: Missing Signatories . . . 210
24.2 Pitfall 2: Visibility Leaks . . . 211
24.3 Pitfall 3: Stale Contract IDs . . . 211
24.4 Pitfall 4: Non-Atomic Operations . . . 212
24.5 Pitfall 5: Time-of-Check/Time-of-Use . . . 213
24.6 Pitfall 6: Unbounded Lists . . . 214
24.7 Pitfall 7: Missing ensure Clauses . . . 216
24.8 Pitfall 8: Authority Confusion . . . 216
24.9 Pitfall 9: Upgrade-Unsafe Changes . . . 218
24.10 Pitfall 10: Over-Trusting Observers . . . 219
24.11 Quick Reference . . . 220
24.12 Summary . . . 220

25 Auditing Daml Contracts **221**
25.1 What a Daml Audit Covers . . . 221
25.2 Checklist 1: Authorization Review . . . 221
25.2.1 Signatory Completeness . . . 221
25.2.2 Unilateral Action Analysis . . . 222
25.2.3 Controller Appropriateness . . . 222
25.3 Checklist 2: Privacy Review . . . 222
25.3.1 Observer Minimality . . . 222
25.3.2 Transitive Visibility . . . 222
25.3.3 Divulgence Awareness . . . 223
25.4 Checklist 3: Choice Analysis . . . 223
25.4.1 Input Validation . . . 223
25.4.2 Consuming vs Nonconsuming . . . 224
25.5 Checklist 4: Contract Identity and Discovery . . . 224
25.5.1 Contract ID Lifecycle . . . 224
25.5.2 Stable Identifiers . . . 224
25.6 Checklist 5: State Machine Review . . . 225
25.6.1 Enumerate States and Transitions . . . 225
25.6.2 Terminal State Handling . . . 225
25.7 Checklist 6: Ensure Clause Review . . . 225
25.8 Checklist 7: Upgrade Safety . . . 226

25.9 Automated Tools . . . 226
25.9.1 Test Coverage . . . 226
25.9.2 Upgrade Verification . . . 226
25.10 Manual Review Techniques . . . 226
25.10.1 Trace Authority Flow . . . 226
25.10.2 Enumerate All Possible Transitions . . . 227
25.10.3 Adversarial Thinking . . . 227
25.11 Writing an Audit Report . . . 227
25.11.1 Report Structure . . . 227
25.11.2 Severity Levels . . . 227
25.12 Example Audit: The IOU Contract . . . 227
25.12.1 Finding 1: Missing Signatory (Critical) . . . 228
25.12.2 Finding 2: Missing ensure Clause (High) . . . 228
25.12.3 Finding 3: Transfer Input Validation (Medium) . . . 228
25.12.4 Finding 4: Settle Has No Side Effects (Medium) . . . 229
25.12.5 Finding 5: No Stable Business Identifier (Low) . . . 229
25.12.6 Audit Summary . . . 229
25.13 Summary . . . 229

26 Production Security **231**
26.1 Canton Participant Hardening . . . 231
26.1.1 Minimal Attack Surface . . . 231
26.1.2 Configuration Hardening . . . 231
26.2 TLS Configuration . . . 232
26.2.1 Participant-to-Synchronizer TLS . . . 232
26.2.2 Ledger API TLS . . . 232
26.2.3 TLS Certificate Management . . . 233
26.3 Key Management . . . 233
26.3.1 Key Types . . . 233
26.3.2 Key Protection with KMS . . . 233
26.3.3 Key Rotation . . . 234
26.3.4 Key Backup . . . 234
26.4 Authentication with Keycloak . . . 234
26.4.1 Realm Configuration . . . 234
26.4.2 Client Configuration . . . 235
26.4.3 Scope and Claim Mapping . . . 235
26.4.4 Canton Authentication Configuration . . . 235
26.5 Network Segmentation . . . 236
26.5.1 Network Zones . . . 236
26.5.2 What Should Not Be Exposed . . . 236
26.6 API Security . . . 236
26.6.1 Rate Limiting . . . 236
26.6.2 Input Validation . . . 236
26.6.3 Authentication Enforcement . . . 237
26.7 Security Monitoring . . . 237
26.7.1 Transaction Monitoring . . . 237
26.7.2 Infrastructure Monitoring . . . 237
26.7.3 Log Management . . . 237
26.8 Incident Response . . . 238
26.8.1 Key Compromise Response . . . 238
26.8.2 Synchronizer Compromise . . . 238

26.9 Compliance Considerations 238
26.9.1 GDPR and Data Privacy 238
26.9.2 Audit Trails 239
26.9.3 Regulatory Reporting 239
26.10 Resilience Under Attack 239
26.10.1 DoS Protection 239
26.10.2 Traffic Limits 240
26.11 Production Security Checklist 240
26.12 Summary 242

Part I

Quick Start

Chapter 1

Introduction

1.1 What This Book Is About

This book teaches you how to build distributed applications using Daml, a smart contract language, running on Canton, a privacy-first blockchain interoperability protocol. By the end, you will be able to model multi-party workflows in Daml, deploy them to a Canton network, and build full-stack applications on top.

We assume you already understand blockchain and distributed ledger concepts—consensus, immutability, smart contracts, finality. What you may not know is how Daml and Canton approach these problems differently from platforms like Ethereum or Hyperledger Fabric. This chapter gives you that context.

1.2 What Is Daml?

Daml is a purpose-built language for writing smart contracts that model multi-party business processes. Unlike Solidity, which operates on a global shared state, Daml models the world as a set of *contracts*—structured data with explicit rules about who can see them, who must agree to them, and what actions can be taken on them.

A Daml contract has three essential properties:

1. **Signatories** — the parties who must authorize the contract's creation. A contract cannot exist without their consent.
2. **Observers** — parties who can see the contract but did not sign it.
3. **Choices** — actions that authorized parties can take on the contract, potentially creating new contracts or archiving existing ones.

This model enforces authorization at the language level. You cannot accidentally write a contract that lets one party steal from another—the type system and runtime prevent it. This is a fundamental difference from most smart contract platforms, where authorization bugs are the leading cause of exploits.

Contracts, Not State

In Daml, there is no mutable global state. The ledger is a sequence of contract create and archive events. A contract either exists (is *active*) or has been consumed (is *archived*). This is closer to the UTXO model than the account model, but with richer structure.

Daml compiles to a portable intermediate representation called **Daml-LF**, packaged into

DAR (Daml Archive) files. These DARs can be deployed to any Daml-compatible ledger—Canton being the primary one.

1.3 What Is Canton?

Canton is the distributed ledger protocol that runs Daml contracts. It consists of several components:

Participant Nodes run Daml smart contracts, store contract data, and expose the Ledger API to applications.

Synchronizers coordinate transaction processing across participants. A synchronizer contains:

- **Sequencer** — orders and delivers messages between participants
- **Mediator** — validates transaction confirmations without seeing contract data

Topology Manager manages party-to-participant mappings, keys, and permissions.

Canton's defining feature is **sub-transaction privacy**: a participant only sees the parts of a transaction that involve its hosted parties. The sequencer and mediator never see contract payloads. This is fundamentally different from Ethereum (where all data is public) or Hyperledger Fabric (where channels provide coarse-grained privacy).

Canton's Privacy Model

When Alice transfers an IOU to Bob via Charlie, only the participants hosting Alice, Bob, and Charlie see the relevant sub-transactions. The sequencer sees encrypted envelopes. The mediator sees confirmation hashes, not data. No party learns more than they need to.

1.4 How Daml and Canton Relate

Think of Daml as the *language* and Canton as the *runtime*:

- Daml defines *what* can happen—contract templates, authorization rules, choices.
- Canton ensures *how* it happens—ordering, privacy, consistency, finality.

You write your business logic in Daml, compile it to a DAR, and deploy that DAR to Canton participant nodes. Applications interact with the ledger through the **Ledger API**—a gRPC and JSON API exposed by each participant node.

The development workflow looks like this:

1. Write Daml templates (smart contracts)
2. Test locally with `dpm test` (runs Daml Scripts in a simulated ledger)
3. Build a DAR with `dpm build`
4. Deploy the DAR to a Canton participant
5. Build applications against the Ledger API

1.5 What You Will Build

This book follows a progressive path:

Part I: Quick Start gets you from zero to a running Canton network with deployed contracts in four chapters. You will install the tooling, understand the local development environment, write your first Daml contract, and deploy it.

Part II: The Daml Language is a systematic tour of the language. Each chapter introduces a concept—templates, choices, authorization, patterns—with standalone examples you can build and test.

Part III: Building Applications moves beyond the ledger. You will connect a backend service to Canton via the JSON Ledger API, use the Participant Query Store for efficient reads, and test the full stack.

Part IV: Canton Deep Dive goes inside Canton's architecture. You will learn how synchronizers work, how identity and topology are managed, and what changes when you move from local development to production.

1.6 Prerequisites

You will need:

- A machine running macOS or Linux (Windows via WSL2 works)
- Docker Desktop (for running Canton via cn-quickstart)
- Basic command-line proficiency
- Familiarity with functional programming concepts (pattern matching, immutability, algebraic data types) is helpful but not required—we introduce them as needed

The next chapter walks you through installing everything.

Chapter 2

Environment Setup

This chapter gets your development environment running. By the end, you will have a local Canton network with three participant nodes, a Daml SDK for writing and testing contracts, and a full-stack reference application to explore.

2.1 Installing DPM

DPM (Digital Asset Package Manager) is the CLI tool for building, testing, and managing Daml projects. It replaces the legacy `daml` CLI.

Install DPM by following the official instructions[1]. The cn-quickstart repository (next section) also includes a `make install-daml-sdk` target. Once installed, set up an SDK version and verify:

```
dpm install 3.4.11        # install a specific SDK version
dpm install package       # or install the version from daml.yaml
dpm version               # verify: should show * 3.4.11
```

2.1.1 What DPM Provides

Here are the commands you will use throughout this book:

Command	Purpose
`dpm build`	Compile Daml source into a DAR file
`dpm test`	Run all Daml Script tests in the project
`dpm new`	Create a new Daml package from a template
`dpm init`	Configure an existing folder as a Daml package
`dpm sandbox`	Run a full Canton node in a single process (for local testing)
`dpm canton-console`	Launch an interactive Canton console
`dpm daml-shell`	Start the Daml Shell (interactive contract REPL)
`dpm studio`	Launch Daml Studio (VS Code integration)
`dpm codegen-java`	Generate Java bindings from a DAR
`dpm codegen-js`	Generate JavaScript bindings from a DAR
`dpm inspect-dar`	Inspect the contents of a DAR archive
`dpm clean`	Remove build artifacts

Table 2.1: Key DPM commands

[1] https://docs.digitalasset.com/build/3.4/dpm/dpm.html

Legacy daml CLI

You may encounter the `daml` CLI in older documentation and tutorials. DPM is its replacement. The commands map directly: `daml build` becomes `dpm build`, `daml test` becomes `dpm test`. If you see deprecation warnings when using `daml`, switch to `dpm`.

2.2 The cn-quickstart Environment

`cn-quickstart` is Digital Asset's official local development scaffold. It runs a complete Canton network inside Docker containers, pre-configured with authentication, observability, and a reference application.

2.2.1 Cloning and Setup

```
git clone https://github.com/digital-asset/cn-quickstart.git
cd cn-quickstart/quickstart
make install-daml-sdk
make setup
```

The `make setup` command is interactive. It asks you to configure:

- **Observability** — enable for Grafana/Prometheus/Loki dashboards
- **OAuth2 authentication** — enable this (uses Keycloak locally)
- **Party hint** — an identifier for your participant in the network (accept the default)
- **TEST MODE** — disable for now (we will use it later for integration tests)

Then build and start:

```
make build    # Builds frontend, backend, Daml DARs, Docker images
make start    # Launches all containers (~60 seconds to fully healthy)
```

2.2.2 What Is Running

Once started, cn-quickstart runs 20+ containers. Here is what matters:

Component	Container	Purpose
Canton node	`canton`	Runs 3 participant nodes in one process
Splice node	`splice`	Simulates the Global Synchronizer
Keycloak	`keycloak`	OAuth2 identity provider
PostgreSQL	`postgres`	Backing store for PQS and services
Nginx	`nginx`	Reverse proxy with auth for each participant
Backend	`backend-service`	Spring Boot REST API (reference app)
PQS	`pqs-app-provider`	Participant Query Store (contract indexing)
Grafana	`grafana`	Metrics and logging dashboards

Table 2.2: Key cn-quickstart containers

2.2.3 The Three Participants

The Canton node hosts three participant nodes, each representing a different role:

Participant	Port Prefix	Nginx Port	Role
App User	2xxx	2000	The customer/end-user
App Provider	3xxx	3000	The business providing the application
Super Validator (SV)	4xxx	4000	Network validator

Table 2.3: Participant nodes and their ports

Each participant exposes three APIs:

gRPC Ledger API (ports `x901`) — the primary programmatic interface for submitting and reading transactions. Used by backends.

Admin API (ports `x902`) — for administrative operations like DAR uploads and party management.

JSON Ledger API (ports `x975`) — an HTTP/JSON wrapper around the gRPC API, convenient for quick interaction and frontend use.

For example, the App Provider's JSON Ledger API is at `localhost:3975`, its gRPC API at `localhost:3901`, and its Admin API at `localhost:3902`.

2.2.4 Local URLs

Once running, you can access these UIs:

URL	What
`app-provider.localhost:3000`	Application UI (provider view)
`wallet.localhost:3000`	Wallet UI (provider)
`wallet.localhost:2000`	Wallet UI (user)
`keycloak.localhost:8082`	Keycloak admin console
`localhost:3030`	Grafana dashboards

Table 2.4: Local development URLs

2.2.5 Key Make Targets

Command	What It Does
`make start`	Start all containers
`make stop`	Stop all containers
`make restart`	Stop then start
`make canton-console`	Interactive Canton REPL (Scala-based)
`make shell`	Daml Shell for contract interaction
`make start-vite-dev`	Start app with hot-reload frontend dev server
`make clean-all`	Full reset (destroys all data)

Table 2.5: Common make targets

2.3 Verifying Your Setup

Run the following to confirm everything is healthy.

All containers should show "healthy" or "running":

```
docker ps --format "table {{.Names}}\t{{.Status}}" \
  | head -10
```

Canton participant should respond:

```
curl -s http://localhost:3975/v2/version | jq .
```

Keycloak should be reachable:

```
KC=http://keycloak.localhost:8082/realms/AppProvider
curl -s "$KC/.well-known/openid-configuration" \
  | jq .issuer
```

If any of these fail, check `make capture-logs` output or run `docker logs canton` to diagnose.

2.4 Your First Daml Project (Outside cn-quickstart)

While cn-quickstart is great for running a full network, you will also want standalone Daml projects for learning. DPM can scaffold these for you with built-in templates.

2.4.1 Creating a Project with dpm new

List available templates:

```
dpm new --list
```

This shows templates for each tutorial topic (`daml-intro-contracts`, `daml-intro-parties`, etc.), as well as general-purpose templates like `skeleton`.

Create a project from the default `skeleton` template:

```
dpm new my-first-project
cd my-first-project
```

This generates a `daml.yaml` and a `daml/` source directory with a starter module. The `daml.yaml` defines the project:

```
sdk-version: 3.4.11
name: my-first-project
source: daml
version: 1.0.0
dependencies:
  - daml-prim
  - daml-stdlib
  - daml-script
```

Build and test it:

```
dpm build    # Compiles to .daml/dist/my-first-project-1.0.0.dar
dpm test     # Runs all Daml Script tests
```

If both commands succeed, your environment is ready.

2.4.2 Tutorial Templates

DPM ships with templates matching the tutorial chapters in this book. You can create any of them directly:

```
# Create the intro-parties project (Chapter 3)
dpm new intro-parties --template daml-intro-parties

# Create other tutorial projects
dpm new intro-choices --template daml-intro-choices
dpm new intro-compose --template daml-intro-compose
```

In the next chapter, we will examine the `daml-intro-parties` template and learn how Daml's authorization model works.

Chapter 3

Your First Contract

This chapter walks through a complete Daml module that builds up an IOU (I Owe You) contract step by step, demonstrating why each design decision matters. By the end, you will understand templates, signatories, choices, and the propose-accept pattern—the core building blocks of every Daml application.

The code in this chapter comes from the `daml-intro-parties` template. Create it with:

```
dpm new intro-parties --template daml-intro-parties
cd intro-parties
```

We will go through it piece by piece.

3.1 A Naive First Attempt

Let us start with the simplest possible IOU:

```
data Cash = Cash with
  currency : Text
  amount : Decimal
    deriving (Eq, Show)

template SimpleIou
  with
    issuer : Party
    owner : Party
    cash : Cash
  where
    signatory issuer
```

This template has one signatory: the `issuer`. The `owner` is just a field—they have no authority over the contract.

Here is a test that shows why this is dangerous:

```
simple_iou_test = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  -- Alice creates an IOU promising Bob 100 USD.
  iou <- submit alice do
    createCmd SimpleIou with
      issuer = alice
      owner = bob
      cash = Cash with amount = 100.0, currency = "USD"

  passTime (days 1)
```

```
  -- Bob delivers the goods, trusting Alice's IOU.

  passTime (minutes 10)
  -- Alice just deletes the payment.
  submit alice do
    archiveCmd iou
```

The Problem

Alice created the IOU, so she is the sole signatory. That means she can unilaterally archive it—effectively deleting her debt to Bob. Bob has no say. In the real world, this would be fraud. In Daml, it is a design bug.

The lesson: **if a party has an obligation, they must be a signatory**. Otherwise they can walk away from it.

3.2 Adding Dual Signatories

The fix is to make both parties signatories:

```
template Iou
  with
    issuer : Party
    owner : Party
    cash : Cash
  where
    signatory issuer, owner
```

Now neither party can unilaterally create or archive the contract. Both must agree. But this creates a new problem:

```
iou_test = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  -- Alice can't create an Iou with Bob as signatory.
  -- Bob hasn't agreed to anything yet.
  submitMustFail alice do
    createCmd Iou with
      issuer = alice
      owner = bob
      cash = Cash with amount = 100.0, currency = "USD"

  -- She can issue one to herself (she's both signatories).
  iou <- submit alice do
    createCmd Iou with
      issuer = alice
      owner = alice
      cash = Cash with amount = 100.0, currency = "USD"

  -- But she can't transfer it to Bob either.
  submitMustFail alice do
    exerciseCmd iou Transfer with newOwner = bob
```

Authorization Rule

A contract can only be created if *all* signatories authorize the transaction. You cannot commit someone to an obligation without their consent. This is enforced by the Daml runtime and the Canton protocol—there is no way to bypass it.

So how do we get Bob to agree? We need the propose-accept pattern.

3.3 The Propose-Accept Pattern

The standard solution in Daml: create a *proposal* contract that only requires one party's authority, then let the other party accept it.

```
template IouProposal
  with
    iou : Iou
  where
    signatory iou.issuer
    observer iou.owner

    choice IouProposal_Accept
        : ContractId Iou
        controller iou.owner
        do
          create iou
```

The workflow:

1. Alice creates an `IouProposal`. Only her authority is needed (she is the sole signatory).
2. Bob can see the proposal (he is an observer).
3. Bob exercises `IouProposal_Accept`. This archives the proposal and creates the real `Iou`—with both Alice and Bob as signatories.

```
iouProposal <- submit alice do
  createCmd IouProposal with
    iou = Iou with
      issuer = alice
      owner = bob
      cash = Cash with amount = 100.0, currency = "USD"

submit bob do
  exerciseCmd iouProposal IouProposal_Accept
```

Why Propose-Accept Works

When Bob exercises `IouProposal_Accept`, the Daml runtime collects authority from two sources: Alice (signatory of the proposal being archived) and Bob (controller of the choice). Together, they satisfy the two-signatory requirement of the `Iou` template. No party is committed without their consent.

This pattern appears everywhere in Daml. Any time you need multiple parties to agree on something, you will use a variation of propose-accept.

3.4 Choices: Transforming Contracts

Choices let parties take actions on existing contracts. Let us add a `Transfer` choice to our IOU:

```
template Iou
  with
    issuer : Party
    owner : Party
    cash : Cash
  where
    signatory issuer, owner

    choice Transfer
      : ContractId Iou
      with
        newOwner : Party
      controller owner
      do
        assertMsg "newOwner cannot be equal to owner."
          (owner /= newOwner)
        create this with owner = newOwner
```

When the owner exercises `Transfer`:

1. The original contract is **archived** (consumed).
2. A new contract is **created** with the `newOwner`.

But wait—the new contract has `issuer` and `newOwner` as signatories. The `newOwner` has not agreed to anything. This means `Transfer` as written here would fail at runtime if `newOwner` is not already a signatory.

The solution is another proposal:

```
    choice ProposeTransfer
      : ContractId IouTransferProposal
      with
        newOwner : Party
      controller owner
      do
        assertMsg "newOwner cannot be equal to owner."
          (owner /= newOwner)
        create IouTransferProposal with
          iou = this
          newOwner
```

And the transfer proposal template:

```
template IouTransferProposal
  with
    iou : Iou
    newOwner : Party
  where
    signatory (signatory iou)
    observer (observer iou), newOwner

    choice IouTransferProposal_Cancel
      : ContractId Iou
      controller iou.owner
      do create iou

    choice IouTransferProposal_Reject
      : ContractId Iou
```

```
      controller newOwner
      do create iou

    choice IouTransferProposal_Accept
      : ContractId Iou
      controller newOwner
      do
        create iou with owner = newOwner
```

Now transfers work safely: Bob proposes, Charlie accepts (or rejects), and the IOU changes hands with everyone's consent.

3.5 Role Contracts and Delegation

Sometimes you want pre-authorized actions. For example: "Bob allows Alice to send him IOUs without asking each time." This is the *role contract* pattern:

```
template IouSender
  with
    sender : Party
    receiver : Party
  where
    signatory receiver
    observer sender

    nonconsuming choice Send_Iou
      : ContractId Iou
      with
        iouCid : ContractId Iou
      controller sender
      do
        iou <- fetch iouCid
        assert (iou.cash.amount > 0.0)
        assert (sender == iou.owner)
        exercise iouCid Mutual_Transfer with
          newOwner = receiver
```

Key points:

- The **receiver** is the signatory—they are the one granting permission.
- The choice is **nonconsuming**—see the explanation below.
- The choice calls `Mutual_Transfer`, which requires both `owner` and `newOwner` authority. The role contract provides the receiver's authority; the sender provides theirs by being the controller.

Consuming vs Nonconsuming Choices

By default, exercising a choice **consumes** (archives) the contract. The contract is gone after the choice executes. This is what happens with `Transfer`, `ProposeTransfer`, and `IouProposal_Accept`—each one is a one-shot action.
A **nonconsuming** choice leaves the contract active. The choice executes, but the contract remains on the ledger and can be exercised again. This is essential for role contracts like `IouSender`: Bob creates one `IouSender` contract granting Alice permission to send him IOUs, and Alice can use it repeatedly—ten, a hundred, a thousand times—without Bob having to create a new permission each time.
The distinction matters for modeling:

- Use **consuming** choices when the action should happen exactly once (transferring an IOU, accepting a proposal).
- Use **nonconsuming** choices when the contract represents a standing permission or capability that should survive repeated use (role contracts, oracles, registries).

3.6 Non-Transitivity of Authority

One subtle but critical property of Daml: **authority is not transitive**.

```
template NonTransitive
  with
    partyA : Party
    partyB : Party
  where
    signatory partyA
    observer partyB

    choice TryA
      : ContractId NonTransitive
      controller partyA
      do
        create NonTransitive with
          partyA = partyB
          partyB = partyA

    choice TryB
      : ContractId NonTransitive
        with
          other : ContractId NonTransitive
      controller partyB
      do
        exercise other TryA
```

If Alice creates a `NonTransitive` contract with Bob, and Bob tries to exercise `TryB` (which internally calls `TryA` on another contract), **it fails**. Why? Because `TryA` requires `partyA`'s authority (Alice's), and Bob exercising `TryB` does not grant Alice's authority—only Bob's.

```
  nt1 <- submit alice do
    createCmd NonTransitive with partyA = alice, partyB = bob
  nt2 <- submit alice do
    createCmd NonTransitive with partyA = alice, partyB = bob

  -- This fails! Bob cannot use Alice's authority transitively.
  submitMustFail bob do
    exerciseCmd nt1 TryB with other = nt2
```

No Authority Laundering

You cannot gain a party's authority by chaining exercises through contracts they signed. Authority flows from the direct controller of the choice being exercised and the signatories of the contract being consumed—nothing more. This prevents a class of attacks where adversaries chain delegations to escalate privileges.

3.7 Building and Testing

The complete code is in the `intro-parties` project. To build and test:

```
cd intro-parties
dpm build
dpm test
```

The `daml.yaml` file defines the project:

```
sdk-version: 3.4.11
name: intro-parties
source: daml
version: 1.0.0
dependencies:
  - daml-prim
  - daml-stdlib
  - daml-script
```

`dpm test` runs every top-level function of type `Script` in your project. It simulates a ledger in-process—no Canton needed. This is your primary feedback loop during development.

3.8 Summary

This chapter covered the fundamental concepts:

- **Templates** define contract types with data, signatories, observers, and choices.
- **Signatories** must authorize contract creation. If a party has an obligation, make them a signatory.
- **Choices** are actions on contracts. Exercising a choice archives the contract (unless `nonconsuming`).
- **Propose-accept** is the standard pattern for getting multiple parties to agree.
- **Role contracts** enable pre-authorized delegation.
- **Authority is not transitive**—you cannot chain exercises to gain unauthorized access.

In the next chapter, we will take our compiled DAR and deploy it to the running Canton network.

Chapter 4

Deploying to Canton

So far we have written and tested Daml contracts against a simulated ledger. In this chapter, we deploy them to a real Canton network—the one running in your cn-quickstart environment.

4.1 Understanding the Deployment Target

Recall from Chapter 2 that cn-quickstart runs three participant nodes inside a single `canton` Docker container. Each participant has its own Ledger API and Admin API, exposed on different ports:

Participant	gRPC Ledger API	Admin API	JSON Ledger API
App User	2901	2902	2975
App Provider	3901	3902	3975
Super Validator	4901	4902	4975

Table 4.1: Canton participant API ports

These ports are exposed directly from the `canton` container. Nginx also fronts them on ports 2000, 3000, and 4000 with Keycloak authentication—but for deployment, we will use the direct ports.

4.2 Method 1: Canton Console

The Canton Console is an interactive Scala REPL connected to all participants. Start it from the quickstart directory:

```
make canton-console
```

You will see the Canton banner. The console has pre-configured references to all participants.

4.2.1 Listing Participants

```
participants.all              // Seq(RemoteParticipantReference(...), ...)
participants.all.map(_.name)  // participant names
```

Remote vs Local

In cn-quickstart, participants are `RemoteParticipantReference`—the console connects to them over the network. In a standalone Canton process, they would be `LocalParticipantReference`. The API is the same; the difference is how the console connects.

4.2.2 Uploading a DAR

The console runs inside a Docker container, so it cannot see your host filesystem. First, copy the DAR into the container:

```
# From your host terminal (not the console)
docker cp \
  intro-parties/.daml/dist/intro-parties-1.0.0.dar \
  canton-console:/tmp/intro-parties-1.0.0.dar
```

Then in the console:

```
// Upload to the App Provider participant
val appProvider = participants.all.find(_.name == "app-provider").get
appProvider.dars.upload("/tmp/intro-parties-1.0.0.dar")
```

A successful upload returns the DAR hash. You can verify:

```
// List uploaded packages
appProvider.packages.list()
```

4.3 Method 2: JSON Ledger API

You can upload DARs directly from your host machine via HTTP. Since cn-quickstart uses Keycloak for authentication, you need an OAuth2 token first.

4.3.1 Getting an Auth Token

The token request uses client credentials from the Keycloak configuration:

```
KC=http://keycloak.localhost:8082
TOKEN_PATH=realms/AppProvider/protocol/openid-connect/token

TOKEN=$(curl -fsS "$KC/$TOKEN_PATH" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "client_id=app-provider-validator" \
  -d "client_secret=AL8648b9SfdTFImq7FV56Vd0KHifHBuC" \
  -d "grant_type=client_credentials" \
  -d "scope=openid" \
  | jq -r .access_token)

echo "Token obtained: $TOKEN"
```

Development Only

The client secret shown here is from the default cn-quickstart Keycloak configuration. It is fine for local development. Never use hardcoded secrets in production.

4.3.2 Uploading the DAR

With the token, upload directly to the App Provider's JSON Ledger API:

```
DAR="intro-parties/.daml/dist/intro-parties-1.0.0.dar"

curl -v -X POST http://localhost:3975/v2/packages \
  -H "Content-Type: application/octet-stream" \
  -H "Authorization: Bearer $TOKEN" \
  --data-binary @"$DAR"
```

A successful upload returns an empty JSON object:

```
{}
```

4.3.3 Verifying the Upload

Query the packages endpoint to confirm:

```
curl -s http://localhost:3975/v2/packages \
  -H "Authorization: Bearer $TOKEN" \
  | jq ".packageIds[:3]"
```

You should see your package ID in the list.

4.4 What Happens After Upload

When you upload a DAR to a participant:

1. The participant **validates** the DAR contents (checks Daml-LF version compatibility, type-checks templates).
2. It **stores** the packages locally.
3. If vetting is enabled, it **announces** the packages to the synchronizer, making them available for transaction processing.
4. Other participants that need to process transactions involving these templates will **also need the DAR** uploaded to them.

Upload to All Participants

If the App Provider creates contracts from the uploaded DAR and the App User needs to exercise choices on them, the App User's participant must also have the DAR. Upload it to each participant that will interact with these contract types.

4.5 Next Steps

You now have a deployed DAR on your Canton network. From here you can:

- Use the **Daml Shell** (`make shell`) to create and exercise contracts interactively.
- Use the **JSON Ledger API** to create contracts programmatically.
- Build a **backend service** that submits transactions and reads contract state.

Part II of this book goes deeper into the Daml language. We will explore each concept—data types, choices, authorization, composition—with standalone examples that you can build, test, and deploy using the techniques from this chapter.

Part II

The Daml Language

Chapter 5

Templates and Signatories

In Chapter 3 we used templates, signatories, and observers to build IOUs. This chapter examines each concept in depth, introduces contract keys, and covers visibility rules.

Create the tutorial project for this chapter:

```
dpm new intro-contracts --template daml-intro-contracts
cd intro-contracts
```

5.1 Anatomy of a Template

A template is the blueprint for a contract. Here is the minimal form:

```
template Token
  with
    owner : Party
  where
    signatory owner
```

Every template has three parts:

`with` block defines the contract's data—its fields and types. This is the payload stored on the ledger when a contract is created.

`where` block defines the contract's meaning—who the signatories and observers are, what choices exist, and what constraints apply.

`signatory` one or more parties who must authorize creation and who guarantee the contract's validity.

A template is a *type*. A contract is an *instance* of that type living on the ledger. Think of it like a class and an object, but immutable—once created, a contract's data never changes. To "update" a contract, you archive the old one and create a new one.

5.2 Signatories

Signatories are the parties who are bound by the contract. They must authorize its creation, and they are notified of every action on it.

```
template Agreement
  with
    partyA : Party
    partyB : Party
```

```
    terms : Text
  where
    signatory partyA, partyB
```

Rules:

1. A contract cannot be created unless *all* signatories authorize the transaction.
2. Signatories can see the contract and all actions on it.
3. A contract can only be archived if all signatories authorize it (directly or through a choice).
4. Every template must have at least one signatory.
5. Signatories must be computed from the contract's `with` fields—they cannot depend on external state.

Obligation = Signatory

If a contract represents an obligation on a party, that party must be a signatory. Otherwise they can walk away. We saw this in Chapter 3 with `SimpleIou`: Alice could delete her debt because Bob was not a signatory.

5.3 Observers

Observers can see the contract but did not sign it. They have no obligations.

```
template Asset
  with
    issuer : Party
    owner : Party
    symbol : Text
    quantity : Decimal
    observers : [Party]
  where
    signatory issuer, owner
    observer observers
```

Why use observers?

- **Visibility**: observers can query and see the contract on the ledger.
- **Choice controllers**: a party must be able to see a contract to exercise a choice on it. Making them an observer grants that visibility.
- **Dynamic**: notice that `observers` is a list field. The contract creator decides who can see it. A choice can create a new contract with different observers.

Signatories Are Automatic Observers

Signatories always see the contract—you do not need to list them as observers separately.

5.4 Visibility Rules

Canton's privacy model means a party only sees contracts they are entitled to see. The rules:

Role	What they can see
Signatory	The contract, all choices exercised on it, all consequences
Observer	The contract, choices exercised on it
Controller	Must be a signatory or observer to see the contract in the first place
Neither	Cannot see the contract at all

Table 5.1: Visibility by role

This is enforced at the protocol level by Canton. A participant node only receives transaction data for contracts involving its hosted parties. The sequencer and mediator never see contract payloads.

5.5 The ensure Clause

You can add preconditions to a template with `ensure`:

```
template Asset
  with
    issuer : Party
    owner : Party
    symbol : Text
    quantity : Decimal
    observers : [Party]
  where
    signatory issuer, owner
    observer observers
    ensure quantity > 0.0
```

If the `ensure` condition is `False` at creation time, the transaction aborts. This is a hard invariant—no contract of this type can ever exist on the ledger with a non-positive quantity.

ensure vs assertMsg

Use `ensure` for template-level invariants that must hold for every instance. Use `assertMsg` inside choices for action-specific validation. The difference: `ensure` applies at creation; `assertMsg` applies at exercise time.

5.6 Contract IDs

Every contract on the ledger has a unique **contract ID** (`ContractId`). When you create a contract, you get back its ID:

```
test_create = do
  alice <- allocateParty "Alice"

  -- createCmd returns a ContractId
  tokenCid <- submit alice do
    createCmd Token with owner = alice

  -- Use the ContractId to exercise a choice
  submit alice do
    exerciseCmd tokenCid Archive
```

Contract IDs are:

- **Opaque**: a hash, not a human-readable string. You cannot construct one manually.

- **Unique**: no two contracts share an ID, ever.
- **Ephemeral**: once a contract is archived, its ID is dead. You cannot reuse it.
- **Unpredictable**: you do not know a contract's ID until it is created. If you need to refer to a contract by a known identifier, use application-level lookups via `queryFilter` or PQS (see Chapter 16).

The problem with contract IDs becomes clear in multi-step workflows. If Alice creates a contract and wants to tell Bob about it off-ledger, she has to send him the contract ID. If the contract is archived and recreated (e.g., through a choice that updates a field), the ID changes. Bob's reference is now stale.

5.7 Finding Contracts: The Business Identifier Pattern

Contract IDs are opaque and ephemeral—not something you can derive from business data. In earlier SDK versions, *contract keys* solved this by letting you declare a stable, human-meaningful identifier at the template level. Contract keys were removed in SDK 3.x. This section covers the patterns that replace them.

5.7.1 Add a Business Identifier Field

The standard approach: add a field to your template that serves as a human-readable, searchable identifier.

```
template Account
  with
    bank : Party
    owner : Party
    accountNumber : Text  -- business identifier
    balance : Decimal
  where
    signatory bank, owner
    ensure accountNumber /= ""
```

The `accountNumber` is not enforced as unique by the ledger—that is now your responsibility. Enforce it *before* creating the contract, in a Daml Script or in your backend service:

```
-- In a Daml Script (off-ledger check before creation):
openAccount : Party -> Party -> Text -> Script (ContractId Account)
openAccount bank newOwner number = do
  -- Check no duplicate exists
  existing <- queryFilter @Account bank
    (\a -> a.accountNumber == number)
  assertMsg "Account number already exists"
    (null existing)
  let parties = actAs bank <> actAs newOwner
  submit parties do
    createCmd Account with
      bank
      owner = newOwner
      accountNumber = number
      balance = 0.0
```

Uniqueness Is Now Application-Level

Without contract keys, the ledger does not enforce that only one contract has a given business identifier. You must check for duplicates yourself—in a Daml Script, in your backend service, or via a PQS constraint. Note that `queryFilter` is a Script action, not an Update action—it cannot be used inside a choice body. Uniqueness checks happen off-ledger. This is the most important difference from the old contract key model.

5.7.2 Discovering Contracts

SDK 3.x provides several ways to find contracts without knowing their ID:

Method	Where	How it works
`query`	Daml Script	All active contracts of a type visible to a party
`queryFilter`	Daml Script	Same, but filtered by a predicate on contract fields
`queryContractId`	Daml Script	Lookup by exact ContractId
`queryInterface`	Daml Script	Query by interface view
Active contracts	Ledger API	`POST /v2/state/active-contracts` with template filter
PQS	SQL	`SELECT * FROM active('Account')` with SQL predicates

Table 5.2: Contract discovery methods

5.7.3 The Find-Then-Exercise Pattern

Without `exerciseByKey`, the standard pattern is to find the contract first, then exercise on it:

```
-- Find the account by business identifier
[(accCid, _)] <- queryFilter @Account bank
  (\a -> a.accountNumber == "ACC-001")

-- Now exercise using the contract ID
submit bank do
  exerciseCmd accCid Deposit with amount = 50.0
```

In a backend application, you do the same via PQS or the active contracts endpoint:

```
# Find via PQS
psql -c "SELECT contract_id FROM active('Main:Account')
  WHERE payload->>'accountNumber' = 'ACC-001'"

# Then exercise via Ledger API
curl -X POST http://localhost:3975/v2/commands/submit-and-wait \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "applicationId": "my-app",
    "commands": [{
      "ExerciseCommand": {
        "templateId": "Main:Account",
        "contractId": "'$CONTRACT_ID'",
        "choice": "Deposit",
        "choiceArgument": {"amount": "50.0"}
      }
    }]
  }'
```

Performance

`queryFilter` scans all active contracts of the template type—fine for hundreds or thousands, but use PQS with indexed columns for larger datasets. See Chapter 16 for indexing strategies.

5.7.4 What You Lose Without Contract Keys

	Contract Keys (SDK 2.x)	Business Identifiers (SDK 3.x)
Uniqueness	Enforced by the ledger	Enforced by application logic
Lookup	`fetchByKey` (atomic)	`queryFilter` or PQS (two-step)
Exercise	`exerciseByKey` (one step)	Find ID, then `exerciseCmd` (two steps)
Stale refs	Key survives archive/recreate	ID changes; re-query needed
Simplicity	Declared in template	Requires application-level coordination

Table 5.3: Contract keys vs business identifiers

The trade-off is clear: more flexibility but more responsibility. In practice, adding a business identifier field plus `queryFilter` covers the vast majority of use cases.

5.8 Putting It Together: An Asset Template

Here is a complete example combining everything from this chapter—templates, signatories, observers, `ensure`, and contract IDs:

```
module SimpleAsset where

import Daml.Script

template Asset
  with
    issuer : Party
    owner : Party
    symbol : Text
    quantity : Decimal
    observers : [Party]
  where
    signatory issuer, owner
    observer observers
    ensure quantity > 0.0

    choice Transfer : ContractId Asset
      with newOwner : Party
      controller owner, newOwner
      do create this with
           owner = newOwner
           observers = []

test_asset = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
```

```
-- Create an asset (bank is sole owner initially)
cid <- submit bank do
  createCmd Asset with
    issuer = bank
    owner = bank
    symbol = "USD"
    quantity = 100.0
    observers = [alice]

-- Transfer to Alice (both must authorize)
let parties = actAs bank <> actAs alice
aliceCid <- submit parties do
  exerciseCmd cid Transfer with newOwner = alice

-- Verify the transfer
Some asset <- queryContractId alice aliceCid
assert (asset.owner == alice)
assert (asset.quantity == 100.0)

-- Look up by contract ID
result <- queryContractId bank cid
assert (result == None)  -- original is archived

-- Query all assets visible to Alice
assets <- queryFilter @Asset alice
  (\a -> a.symbol == "USD")
assert (length assets == 1)
```

5.9 Summary

- **Templates** define contract types with data (`with`) and meaning (`where`).
- **Signatories** must authorize creation and are bound by the contract. If a party has an obligation, make them a signatory.
- **Observers** can see the contract but have no obligations.
- **Visibility** is enforced by Canton at the protocol level—parties only see what they are entitled to.
- **`ensure`** enforces invariants at creation time.
- **Contract IDs** are unique but ephemeral—they change when contracts are archived and recreated.
- **Business identifier fields** replace the removed contract keys feature. Add a meaningful field (e.g., `accountNumber`), use `queryFilter` to find contracts by it, and enforce uniqueness in your application logic.
- **Find-then-exercise** is the standard pattern: discover a contract via `queryFilter` or PQS, then `exerciseCmd` using the returned ID.

Chapter 6

Data Types

Every Daml contract stores structured data. So far we have used simple fields—`Party`, `Text`, `Decimal`—without looking closely at the type system that underlies them. This chapter examines Daml's type system in depth by building a token registry: a contract that tracks digital tokens with rich metadata, account statuses, and optional fields. Along the way, you will learn records, variants, enums, type aliases, tuples, lists, and `Optional`—the building blocks you will use in every Daml project.

Create the project for this chapter:

```
dpm new token-registry
cd token-registry
```

Replace the contents of `daml/Main.daml` with the code we build below.

6.1 Records

A record groups named fields into a single type. We have already seen records in template `with` blocks. Outside templates, you define them with `data`:

```
data TokenInfo = TokenInfo with
    symbol : Text
    name : Text
    decimals : Int
    maxSupply : Decimal
  deriving (Eq, Show)
```

This creates a type called `TokenInfo` with four fields. The `deriving (Eq, Show)` clause tells the compiler to automatically generate equality comparison and string conversion.

Create a value:

```
exampleToken : TokenInfo
exampleToken = TokenInfo with
  symbol = "ACME"
  name = "Acme Corp Token"
  decimals = 8
  maxSupply = 1000000.0
```

Access fields with dot notation:

In a Daml Script or choice body, access fields with dot notation:

```
-- exampleToken.symbol   evaluates to "ACME"
-- exampleToken.maxSupply evaluates to 1000000.0
```

Records Are Immutable

Once created, a record value cannot be modified. To "change" a field, you create a new record. The `with` syntax makes this concise: `exampleToken with decimals = 6` creates a copy with only `decimals` changed.

6.1.1 Nested Records

Records can contain other records:

```
data Issuer = Issuer with
    party : Party
    name : Text
    country : Text
  deriving (Eq, Show)

data TokenConfig = TokenConfig with
    info : TokenInfo
    issuer : Issuer
  deriving (Eq, Show)
```

Access nested fields by chaining dots: `config.info.symbol` or `config.issuer.country`.

6.2 Variants (Sum Types)

A variant defines a type that can be one of several alternatives. This is Daml's most powerful modeling tool—it lets you represent states, outcomes, and choices as types that the compiler can check exhaustively.

```
data Status
  = Active
  | Frozen
  | Closed
    deriving (Eq, Show)
```

A value of type `Status` is exactly one of `Active`, `Frozen`, or `Closed`. There is no fourth option, no null, no "unknown." The compiler knows all possibilities.

Preventing Invalid States

In most languages, you might model status as a string or integer—`"active"`, `"frozen"`, `1`, `2`. This allows typos (`"actve"`), invalid values (`"deleted"`), and runtime crashes. With Daml variants, invalid states are *compile-time errors*. If you add a new variant, the compiler flags every place that needs updating.

6.2.1 Variants with Data

Each variant constructor can carry data:

```
data TransferResult
  = Success (ContractId TokenHolding)
  | InsufficientBalance Decimal
  | AccountFrozen Text
    deriving (Eq, Show)
```

`Success` carries a contract ID. `InsufficientBalance` carries the available balance. `AccountFrozen` carries an error message.

6.2.2 Pattern Matching

Use `case` to inspect a variant value. The compiler ensures you handle every case:

```
describeStatus : Status -> Text
describeStatus s = case s of
  Active -> "Account is active and operational"
  Frozen -> "Account is frozen by administrator"
  Closed -> "Account has been permanently closed"
```

If you remove a branch, the compiler warns about a non-exhaustive pattern match.

6.3 Enums

An enum is a variant where no constructor carries data. `Status` above is already an enum. Here is another common example:

```
data Priority = Low | Medium | High | Critical
  deriving (Eq, Show, Ord)
```

Note the `Ord` derivation—this lets you compare priorities: `High > Low` evaluates to `True`. The ordering follows the declaration order.

6.4 Type Aliases

A type alias gives a new name to an existing type. It does not create a new type—it is purely for readability:

```
type Amount = Decimal
type Symbol = Text
type TokenKey = (Party, Symbol)
```

Use aliases to make template signatures self-documenting:

```
template TokenHolding
  with
    issuer : Party
    owner : Party
    symbol : Symbol
    balance : Amount
  where
    signatory issuer, owner
```

Aliases Are Transparent

A `Symbol` is literally `Text`. You can pass a `Text` where a `Symbol` is expected and vice versa. If you need enforcement—preventing a raw `Text` from being used as a symbol—wrap it in a `newtype` instead.

6.5 Tuples

Tuples are anonymous fixed-size collections of values:

```
type Pair = (Text, Int)
type Triple = (Party, Text, Decimal)

examplePair : Pair
```

```
examplePair = ("ACME", 8)

-- Access with ._1, ._2:
-- examplePair._1 evaluates to "ACME"
-- examplePair._2 evaluates to 8
```

We saw tuples already in Chapter 5. They are useful for quick groupings but become hard to read beyond three elements. Prefer named records for anything with more than two or three fields.

6.6 Lists

A list holds zero or more values of the same type:

```
symbols : [Text]
symbols = ["ACME", "BETA", "GAMA"]

balances : [Decimal]
balances = [100.0, 250.5, 0.0]
```

Common list operations (`head` and `tail` require `import DA.List`):

```
import DA.List (head, tail)

test_lists : Script ()
test_lists = do
  let symbols = ["ACME", "BETA", "GAMA"]
  let balances : [Decimal] = [100.0, 250.5, 0.0]

  assert (length symbols == 3)
  assert (not (null symbols))
  assert (head symbols == "ACME")
  assert (tail symbols == ["BETA", "GAMA"])

  -- Map: apply a function to each element
  let upper = map (\s -> s <> "!") symbols
  assert (head upper == "ACME!")

  -- Filter: keep elements matching a predicate
  let positive = filter (> 0.0) balances
  assert (length positive == 2)

  -- Fold: reduce a list to a single value
  let total = foldl (+) 0.0 balances
  assert (total == 350.5)
```

Lists are used frequently in Daml contracts for observer lists, batch operations, and audit trails.

Lists in Contracts

Large lists in contract payloads can cause performance problems. Each time a contract is created or archived, the entire payload is serialized and transmitted. If you need to store thousands of items, consider splitting them across multiple contracts.

6.7 Optional

`Optional` represents a value that may or may not exist. It has two constructors: `Some value` and `None`.

```
data TokenMetadata = TokenMetadata with
    symbol : Text
    name : Text
    website : Optional Text
    description : Optional Text
  deriving (Eq, Show)

-- A token with all fields
fullMeta : TokenMetadata
fullMeta = TokenMetadata with
  symbol = "ACME"
  name = "Acme Corp Token"
  website = Some "https://acme.example.com"
  description = Some "Utility token for Acme services"

-- A token with missing optional fields
minimalMeta : TokenMetadata
minimalMeta = TokenMetadata with
  symbol = "BETA"
  name = "Beta Token"
  website = None
  description = None
```

Work with `Optional` values using pattern matching or helper functions from `DA.Optional`:

```
import DA.Optional

-- Pattern matching
showWebsite : TokenMetadata -> Text
showWebsite meta = case meta.website of
  Some url -> "Website: " <> url
  None     -> "No website listed"
```

Inside a `do` block, use the helper functions from `DA.Optional`:

```
test_optional : Script ()
test_optional = do
  let meta = TokenMetadata with
        symbol = "ACME"
        name = "ACME Token"
        website = Some "acme.com"
        description = None
  let url = fromOptional "N/A" meta.website
  let hasWeb = isSome meta.website
  let noWeb = isNone meta.website
  assert (url == "acme.com")
  assert hasWeb
  assert (not noWeb)
```

Optional vs Null

There is no `null` in Daml. If a field can be absent, you declare it as `Optional`. The compiler forces you to handle both the `Some` and `None` cases. This eliminates null pointer errors entirely—they are caught at compile time, not at runtime.

6.8 The deriving Clause

When you define a record or variant, `deriving` auto-generates common type class instances:

Type Class	What It Provides
`Eq`	Equality: `==` and `/=`
`Show`	String conversion: `show value` produces a readable string
`Ord`	Ordering: `<`, `>`, `<=`, `>=`, `compare`

Table 6.1: Common derivable type classes

Any type used in a template's `with` block must derive `Eq` and `Show`—the Daml runtime needs these for contract comparison and logging. Always include `deriving (Eq, Show)` on your data types.

6.9 Putting It Together: Token Registry

Here is the complete token registry module combining everything from this chapter:

```
module Main where

import Daml.Script

-- Type aliases for readability
type Symbol = Text
type Amount = Decimal
```

```
-- Account status as a variant
data Status
  = Active
  | Frozen
  | Closed
    deriving (Eq, Show)

-- Token metadata with optional fields
data TokenInfo = TokenInfo with
    symbol : Symbol
    name : Text
    decimals : Int
    website : Optional Text
  deriving (Eq, Show)
```

```
-- The registry contract
template TokenRegistry
  with
    admin : Party
    info : TokenInfo
    status : Status
    holders : [Party]
  where
    signatory admin
    observer holders

    ensure info.decimals >= 0
      && info.decimals <= 18
```

```
    -- Freeze the token (admin only)
    choice Freeze : ContractId TokenRegistry
      controller admin
      do
```

```
        assertMsg "Can only freeze Active tokens"
          (status == Active)
        create this with status = Frozen

    -- Unfreeze back to Active
    choice Unfreeze : ContractId TokenRegistry
      controller admin
      do
        assertMsg "Can only unfreeze Frozen tokens"
          (status == Frozen)
        create this with status = Active
```

```
    -- Close permanently
    choice CloseToken : ContractId TokenRegistry
      controller admin
      do
        assertMsg "Cannot close: already Closed"
          (status /= Closed)
        create this with
          status = Closed
          holders = []

    -- Add an observer
    nonconsuming choice AddHolder
      : ContractId TokenRegistry
      with
        newHolder : Party
      controller admin
      do
        assertMsg "Token must be Active"
          (status == Active)
        assertMsg "Already a holder"
          (notElem newHolder holders)
        archive self
        create this with
          holders = newHolder :: holders
```

Now the test script:

```
test_registry : Script ()
test_registry = do
  admin <- allocateParty "Admin"
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  -- Create a token registry entry
  regCid <- submit admin do
    createCmd TokenRegistry with
      admin
      status = Active
      holders = []
      info = TokenInfo with
        symbol = "ACME"
        name = "Acme Corp Token"
        decimals = 8
        website = Some "https://acme.example.com"

  -- Add holders
  regCid <- submit admin do
    exerciseCmd regCid AddHolder with
      newHolder = alice
  regCid <- submit admin do
```

```
    exerciseCmd regCid AddHolder with
      newHolder = bob

  -- Freeze the token
  regCid <- submit admin do
    exerciseCmd regCid Freeze

  -- Cannot add holders while frozen
  submitMustFail admin do
    exerciseCmd regCid AddHolder with
      newHolder = alice
```

```
  -- Unfreeze and close
  regCid <- submit admin do
    exerciseCmd regCid Unfreeze
  submit admin do
    exerciseCmd regCid CloseToken

  -- Verify: create a minimal token (no optional fields)
  submit admin do
    createCmd TokenRegistry with
      admin
      status = Active
      holders = []
      info = TokenInfo with
        symbol = "BETA"
        name = "Beta Token"
        decimals = 0
        website = None

  pure ()
```

Build and test:

```
dpm build
dpm test
```

The type system does the heavy lifting. You cannot create a `TokenRegistry` with `status = "maybe"` or `decimals = "eight"`—the compiler rejects it. You cannot forget to handle a `Status` variant in a `case` expression. And `Optional` fields force you to explicitly decide what happens when a value is absent.

6.10 Summary

- **Records** (`data ... with`) group named fields. They are the primary data structure in Daml templates.
- **Variants** (`data ... = A | B | C`) represent one-of-many choices. They can carry data and are checked exhaustively by the compiler.
- **Enums** are variants with no associated data—use them for simple state machines.
- **Type aliases** (`type Symbol = Text`) improve readability without creating new types.
- **Tuples** are anonymous fixed-size groupings. Use them for grouping related values and quick returns; prefer records beyond two or three elements.
- **Lists** hold zero or more values. Use `map`, `filter`, and `foldl` to process them. Keep contract lists small.

- **Optional** replaces null. The compiler forces you to handle the `None` case, eliminating null pointer errors.
- **deriving (Eq, Show)** is required for any type used in a template's `with` block.
- Daml's type system prevents invalid states at compile time. Model your domain with records and variants, and the compiler becomes your first line of defense.

Chapter 7

Choices and Controllers

Chapter 3 introduced choices as actions on contracts. This chapter goes deeper: we build a multi-step expense approval workflow covering consuming and nonconsuming choices, flexible controllers, return types, and choice chaining.

Create the project for this chapter:

```
dpm new expense-approval
cd expense-approval
```

Replace the contents of `daml/Main.daml` with the code we build below.

7.1 Anatomy of a Choice

Every choice has four parts:

```
choice ChoiceName : ReturnType
  with
    param1 : Type1
    param2 : Type2
  controller controllerExpr
  do
    -- choice body: Update actions
    create ...
    exercise ...
    pure result
```

`ChoiceName : ReturnType` — the name and what the choice returns when exercised. The return type can be anything: a `ContractId`, a record, a list, `()`, or a custom variant.

`with` block — parameters passed by the party exercising the choice. These are in addition to the contract's own fields (available as `this`).

`controller` — an expression evaluating to one or more parties who may exercise this choice. The controller must be visible (signatory or observer) on the contract.

`do` block — the body. Runs in the `Update` monad, which allows creating contracts, exercising choices, fetching contracts, and asserting conditions.

7.2 The Expense Approval Domain

Our workflow models company expense reports:

1. An **employee** submits an expense report.

2. A **manager** reviews it: approve, reject, or request more information.
3. If approved above a threshold, a **finance officer** gives final approval.
4. The approved expense creates a **payment** record.

We need several contract types and choices that chain together.

7.3 Data Types

```
module Main where

import Daml.Script

data Category
  = Travel
  | Meals
  | Equipment
  | Software
  | Other
    deriving (Eq, Show)

data ExpenseStatus
  = Pending
  | Approved
  | Rejected
  | NeedsInfo
    deriving (Eq, Show)
```

```
data LineItem = LineItem with
    description : Text
    amount : Decimal
    category : Category
  deriving (Eq, Show)
```

7.4 The ExpenseReport Template

```
template ExpenseReport
  with
    employee : Party
    manager : Party
    finance : Party
    title : Text
    items : [LineItem]
    status : ExpenseStatus
    notes : Text
  where
    signatory employee
    observer manager, finance

    -- Only valid with at least one item
    ensure not (null items)
```

The employee is the sole signatory—they are submitting the report. The manager and finance officer are observers so they can see and act on it.

7.5 Consuming Choices

A consuming choice archives the contract when exercised. This is the default. It models one-shot actions: once taken, the contract is gone.

```
-- Manager approves the expense
choice Approve : ContractId ExpenseReport
  controller manager
  do
    assertMsg "Can only approve Pending reports"
      (status == Pending)
    create this with
      status = Approved
      notes = "Approved by manager"
```

When the manager exercises `Approve`:

1. The original `ExpenseReport` (with `status = Pending`) is archived.
2. A new `ExpenseReport` (with `status = Approved`) is created.
3. The choice returns the new contract's ID.

This is the "archive and recreate" pattern from Chapter 3. The contract is immutable—you create a new version rather than mutating the old one.

```
-- Manager rejects the expense
choice Reject : ContractId ExpenseReport
  with
    reason : Text
  controller manager
  do
    assertMsg "Can only reject Pending reports"
      (status == Pending)
    create this with
      status = Rejected
      notes = reason
```

```
-- Manager requests more information
choice RequestInfo : ContractId ExpenseReport
  with
    question : Text
  controller manager
  do
    assertMsg "Can only request info on Pending"
      (status == Pending)
    create this with
      status = NeedsInfo
      notes = question
```

```
-- Employee responds to info request
choice ProvideInfo : ContractId ExpenseReport
  with
    response : Text
  controller employee
  do
    assertMsg "Can only respond to NeedsInfo"
      (status == NeedsInfo)
    create this with
      status = Pending
      notes = response
```

Notice how each choice enforces valid state transitions. You cannot approve a rejected report or request info on an already-approved one. The variant type `ExpenseStatus` combined with runtime assertions creates a state machine.

Consuming Choices as State Transitions

Each consuming choice archives the old contract and creates a new one with updated fields. This models a state machine where each transition is an explicit, authorized action. The ledger records the full history: every state the contract passed through and who triggered each transition.

7.6 Nonconsuming Choices

A nonconsuming choice leaves the contract active after execution. Mark it with the `nonconsuming` keyword:

```
-- Calculate total (read-only, no state change)
nonconsuming choice GetTotal : Decimal
  controller employee
  do
    pure (foldl (\acc i -> acc + i.amount) 0.0 items)
```

Exercising `GetTotal` returns the sum of all line items but does not archive the contract. You can call it repeatedly.

Recall the key concept from Chapter 3: use consuming choices for one-shot actions and nonconsuming choices for standing permissions or read operations.

Nonconsuming Choices and Double-Spend

Be careful with nonconsuming choices that create contracts or transfer value. Since the source contract is not archived, the choice can be exercised multiple times. A nonconsuming `Transfer` choice would allow sending the same funds repeatedly. Only make a choice nonconsuming when repeated exercise is intentional and safe.

7.7 Flexible Controllers

The controller expression can reference choice parameters, enabling flexible authorization:

```
-- Any party in a list can add a comment
nonconsuming choice AddComment : ContractId Comment
  with
    commenter : Party
    body : Text
  controller commenter
  do
    assertMsg "Not authorized to comment"
      (commenter == employee
        || commenter == manager
        || commenter == finance)
    create Comment with
      report = self
      author = commenter
      body
```

Here the controller is the `commenter` parameter. The `assertMsg` inside the choice body adds an additional authorization check—the controller must be one of the involved parties.

```
-- Supporting template for comments
template Comment
  with
    report : ContractId ExpenseReport
    author : Party
    body : Text
  where
    signatory author
```

Controller vs Authorization

The `controller` keyword determines who *can* exercise the choice at the protocol level. Only the controller's participant can submit the exercise command. But you can add further restrictions in the choice body with `assertMsg`. Think of `controller` as the lock on the door and `assertMsg` as the security guard inside.

7.8 Return Types from Choices

Choices can return any type. This is crucial for chaining workflows:

```
-- A payment record created after final approval
template Payment
  with
    payer : Party
    payee : Party
    amount : Decimal
    reference : Text
  where
    signatory payer
    observer payee
```

```
-- Final approval with payment creation
template ApprovedExpense
  with
    employee : Party
    manager : Party
    finance : Party
    title : Text
    totalAmount : Decimal
  where
    signatory employee, manager
    observer finance
```

```
    -- Finance issues payment (return type is a tuple)
    choice IssuePayment
      : (ContractId Payment, Decimal)
      controller finance
      do
        let pmtRef = "EXP-" <> title
        pmtCid <- create Payment with
          payer = finance
          payee = employee
          amount = totalAmount
          reference = pmtRef
        pure (pmtCid, totalAmount)
```

The caller receives both the payment contract ID and the amount. This is useful when a

downstream step needs information produced by the choice.

7.9 Chaining Choices in a Transaction

A single `submit` block can exercise multiple choices, and one choice can call another. This all happens atomically—either everything succeeds or nothing does.

```
-- Manager approval that auto-creates ApprovedExpense
template ExpenseWorkflow
  with
    employee : Party
    manager : Party
    finance : Party
    reportCid : ContractId ExpenseReport
  where
    signatory employee
    observer manager
```

```
    choice ManagerApprove
      : ContractId ApprovedExpense
      with
        approvedTotal : Decimal
      controller manager
      do
        -- Archive the original report
        exercise reportCid Archive

        -- Create the approved record
        create ApprovedExpense with
          employee
          manager
          finance
          title = "Approved expense"
          totalAmount = approvedTotal
```

Inside `ManagerApprove`, we exercise `Archive` on the report and create an `ApprovedExpense`—two ledger operations in one atomic transaction. If either fails, neither takes effect.

Atomicity

Everything inside a `submit` block (and everything inside a choice body) executes as a single atomic transaction. If any operation fails—an assertion, an authorization check, a missing contract—the entire transaction is rolled back. There is no partial execution. This is a fundamental guarantee of the Daml/Canton runtime.

7.10 Complete Test Script

```
test_expense : Script ()
test_expense = do
  emp <- allocateParty "Employee"
  mgr <- allocateParty "Manager"
  fin <- allocateParty "Finance"

  -- Employee submits an expense report
  reportCid <- submit emp do
    createCmd ExpenseReport with
      employee = emp
```

```
      manager = mgr
      finance = fin
      title = "Q1 Travel"
      items =
        [ LineItem "Flight" 450.0 Travel
        , LineItem "Hotel" 320.0 Travel
        , LineItem "Dinner" 85.0 Meals
        ]
      status = Pending
      notes = ""
```

```
  -- Manager requests more info
  reportCid <- submit mgr do
    exerciseCmd reportCid RequestInfo with
      question = "Was the dinner client-facing?"

  -- Employee responds
  reportCid <- submit emp do
    exerciseCmd reportCid ProvideInfo with
      response = "Yes, dinner with client ABC Corp"
```

```
  -- Manager approves
  reportCid <- submit mgr do
    exerciseCmd reportCid Approve

  -- Verify the report is now Approved
  Some report <- queryContractId emp reportCid
  assertMsg "Should be Approved"
    (report.status == Approved)

  pure ()
```

```
test_rejection : Script ()
test_rejection = do
  emp <- allocateParty "Employee"
  mgr <- allocateParty "Manager"
  fin <- allocateParty "Finance"

  reportCid <- submit emp do
    createCmd ExpenseReport with
      employee = emp
      manager = mgr
      finance = fin
      title = "Gaming Setup"
      items =
        [ LineItem "PS5" 499.0 Equipment ]
      status = Pending
      notes = ""
```

```
  -- Manager rejects
  reportCid <- submit mgr do
    exerciseCmd reportCid Reject with
      reason = "Not a business expense"

  -- Verify rejection
  Some report <- queryContractId emp reportCid
  assertMsg "Should be Rejected"
    (report.status == Rejected)

  -- Cannot approve a rejected report
```

```
  submitMustFail mgr do
    exerciseCmd reportCid Approve

  pure ()
```

```
test_nonconsuming : Script ()
test_nonconsuming = do
  emp <- allocateParty "Employee"
  mgr <- allocateParty "Manager"
  fin <- allocateParty "Finance"

  reportCid <- submit emp do
    createCmd ExpenseReport with
      employee = emp
      manager = mgr
      finance = fin
      title = "Office Supplies"
      items =
        [ LineItem "Notebooks" 25.0 Other
        , LineItem "Pens" 15.0 Other
        ]
      status = Pending
      notes = ""
```

```
  -- GetTotal is nonconsuming: can call it twice
  total1 <- submit emp do
    exerciseCmd reportCid GetTotal
  total2 <- submit emp do
    exerciseCmd reportCid GetTotal
  assertMsg "Totals should match" (total1 == total2)
  assertMsg "Total should be 40" (total1 == 40.0)

  pure ()
```

Build and test:

```
dpm build
dpm test
```

7.11 Summary

- **Consuming choices** (the default) archive the contract when exercised. Use them for state transitions and one-shot actions.
- **Nonconsuming choices** leave the contract active. Use them for read operations and standing permissions. Be cautious about double-spend risks.
- **Controllers** determine who can exercise a choice. They can be contract fields, choice parameters, or expressions involving both.
- **Return types** let choices pass results to callers. Return contract IDs, records, tuples, or custom types.
- **Choice chaining** within a single transaction is atomic—all operations succeed or all fail. One choice can `exercise` another, `create` contracts, or `fetch` data.
- **State machines** emerge naturally: each consuming choice is a transition that archives the old state and creates the new one. The type system and assertions enforce valid

transitions.

Chapter 8

Authorization Model

This is the "try to break things" chapter. We will deliberately write contracts that fail authorization, examine the error messages, and then fix them. The goal is to build an intuition for how Daml's authorization model works—not by reading rules, but by hitting walls and understanding why they exist.

The primary teaching tool is `submitMustFail`: a Daml Script command that asserts a transaction *should* fail. If the transaction unexpectedly succeeds, the test fails. This lets us write executable specifications of what is and is not allowed.

Create the project:

```
dpm new auth-lab
cd auth-lab
```

Replace the contents of `daml/Main.daml` with the code we build below.

8.1 The Authorization Rules

Before we start breaking things, here are the rules we will test. Every Daml transaction must satisfy all of them:

1. **Create**: all signatories of the new contract must authorize the transaction.
2. **Exercise**: the controller of the choice must authorize the transaction.
3. **Archive**: all signatories of the archived contract must authorize (archiving is exercising the built-in `Archive` choice).
4. **Authority sources**: authority comes from (a) the submitting party, and (b) the signatories of contracts being consumed in the same transaction.
5. **No laundering**: you cannot gain a party's authority by chaining through contracts they signed. Authority flows from direct sources only.

We covered rule 5 in Chapter 3. Now we will systematically test all five.

8.2 Setup: The Lab Templates

```
module Main where

import Daml.Script
```

```
-- A simple contract: one signatory
template Note
  with
    writer : Party
    text : Text
  where
    signatory writer
```

```
-- Two signatories: both must agree
template Agreement
  with
    partyA : Party
    partyB : Party
    terms : Text
  where
    signatory partyA, partyB
    observer partyA, partyB
```

```
-- A proposal for an agreement (single signatory)
template AgreementProposal
  with
    proposer : Party
    accepter : Party
    terms : Text
  where
    signatory proposer
    observer accepter

    choice Accept : ContractId Agreement
      controller accepter
      do
        create Agreement with
          partyA = proposer
          partyB = accepter
          terms
```

```
    choice Reject : ()
      controller accepter
      do pure ()

    choice Withdraw : ()
      controller proposer
      do pure ()
```

8.3 Test 1: Cannot Create Without All Signatories

The most fundamental rule: you cannot create a contract unless every signatory authorizes it.

```
test_create_auth : Script ()
test_create_auth = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  -- FAIL: Alice tries to create an Agreement
  --       where Bob is also a signatory.
  --       Bob has not authorized this.
  submitMustFail alice do
    createCmd Agreement with
      partyA = alice
```

```
      partyB = bob
      terms = "Alice wrote this without Bob"

  -- OK: Alice can create a Note (she is sole signatory)
  submit alice do
    createCmd Note with
      writer = alice
      text = "Alice's note"

  pure ()
```

The first `submitMustFail` confirms that Alice cannot unilaterally bind Bob to an agreement. The runtime rejects the transaction because Bob's authority is missing.

The Consent Principle

No party can be obligated without their consent. This is enforced at the deepest level of the Daml runtime and the Canton protocol. There is no admin override, no superuser mode, no backdoor. If a party is a signatory, they must authorize.

8.4 Test 2: Cannot Exercise Without Controller Authority

Only the controller (or a party with delegated authority) can exercise a choice.

```
test_exercise_auth : Script ()
test_exercise_auth = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  charlie <- allocateParty "Charlie"

  -- Alice proposes an agreement to Bob
  propCid <- submit alice do
    createCmd AgreementProposal with
      proposer = alice
      accepter = bob
      terms = "Partnership deal"
```

```
  -- FAIL: Charlie tries to accept Bob's proposal.
  --       Charlie is not the controller.
  submitMustFail charlie do
    exerciseCmd propCid Accept

  -- FAIL: Alice tries to accept her own proposal.
  --       Alice is the proposer, not the accepter.
  submitMustFail alice do
    exerciseCmd propCid Accept

  -- OK: Bob accepts (he is the controller)
  submit bob do
    exerciseCmd propCid Accept

  pure ()
```

Even though Alice created the proposal, she cannot exercise `Accept`—that is Bob's choice. And Charlie, who is not even an observer, certainly cannot.

8.5 Test 3: Cannot Archive Without Signatory Authority

Archiving a contract requires all signatories to authorize. Since the built-in `Archive` choice has `controller (signatory this)`, any signatory can trigger it, but the authorization still requires all signatories.

```
test_archive_auth : Script ()
test_archive_auth = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  -- Create a single-signatory Note
  noteCid <- submit alice do
    createCmd Note with
      writer = alice
      text = "My note"

  -- FAIL: Bob tries to archive Alice's Note.
  --       Bob is not a signatory.
  submitMustFail bob do
    archiveCmd noteCid
```

```
  -- OK: Alice archives her own Note
  submit alice do
    archiveCmd noteCid

  -- Create a dual-signatory Agreement via proposal
  propCid <- submit alice do
    createCmd AgreementProposal with
      proposer = alice
      accepter = bob
      terms = "Joint venture"
  agrCid <- submit bob do
    exerciseCmd propCid Accept

  -- FAIL: Alice alone cannot archive a dual-signatory
  -- contract. All signatories must authorize.
  submitMustFail alice do
    archiveCmd agrCid

  -- OK: Both signatories together can archive
  let parties = actAs alice <> actAs bob
  submit parties do
    archiveCmd agrCid

  pure ()
```

Archive on Dual-Signatory Contracts

The built-in `Archive` choice has `controller (signatory this)`, meaning all signatories must authorize. For a contract with signatories A and B, both must submit together using `submit (actAs a <> actAs b) do`. A single signatory cannot unilaterally archive a multi-signatory contract. To allow one party to archive independently, define a custom consuming choice with that party as the sole controller.

8.6 Test 4: Authority Flows Through Exercise Chains

When a choice body exercises another choice or creates a contract, authority comes from two sources:

1. The **submitting party** (who initiated the top-level transaction).
2. The **signatories of contracts being consumed** in the current transaction.

Let us see this in action:

```
-- A contract that delegates creation authority
template Delegator
  with
    owner : Party
    delegate : Party
  where
    signatory owner
    observer delegate

    choice CreateOnBehalf : ContractId Note
      with
        noteText : Text
      controller delegate
      do
        -- This creates a Note with owner as signatory.
        -- Authority comes from:
        --   (1) delegate (submitter / controller)
        --   (2) owner (signatory of Delegator
        --       being consumed)
        create Note with
          writer = owner
          text = noteText
```

```
test_authority_flow : Script ()
test_authority_flow = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  -- Alice creates a Delegator, giving Bob
  -- the ability to create Notes as Alice
  delCid <- submit alice do
    createCmd Delegator with
      owner = alice
      delegate = bob

  -- Bob exercises CreateOnBehalf.
  -- The Delegator is consumed, contributing
  -- Alice's authority. Combined with Bob's
  -- authority as submitter, the Note creation
  -- succeeds.
  submit bob do
    exerciseCmd delCid CreateOnBehalf with
      noteText = "Written by Bob, signed by Alice"

  pure ()
```

This works because consuming the `Delegator` releases Alice's authority into the transaction. Bob's authority comes from being the submitter. Together, they have enough authority for any operation that needs Alice's or Bob's authorization.

Authority Collection

In a transaction, authority accumulates from two sources: the submitting party and the signatories of consumed contracts. When a choice body creates a contract, the runtime checks whether the accumulated authority covers all required signatories. This is how propose-accept works: consuming the proposal releases the proposer's authority, and the accepter's authority comes from being the controller/submitter.

8.7 Test 5: No Authority Laundering

This is the subtlest rule, introduced in Chapter 3. You cannot gain a party's authority by exercising choices on contracts they signed—only by consuming those contracts.

```
-- A contract with a nonconsuming choice
template Vault
  with
    owner : Party
    guardian : Party
  where
    signatory owner
    observer guardian

    -- Nonconsuming: the Vault is NOT archived
    nonconsuming choice Peek : Text
      controller guardian
      do pure "Vault contents"
```

```
    -- Consuming: the Vault IS archived
    choice Open : Text
      controller guardian
      do pure "Vault opened"

-- Try to use a Vault to create a Note as owner
template VaultExploit
  with
    attacker : Party
    vaultCid : ContractId Vault
  where
    signatory attacker
```

```
    choice TryExploit : ContractId Note
      controller attacker
      do
        -- Exercise a nonconsuming choice on the Vault.
        -- The Vault is NOT consumed, so owner's
        -- authority is NOT released.
        exercise vaultCid Peek
        -- Now try to create a Note as the vault owner.
        -- This WILL FAIL: we never got owner's authority.
        vault <- fetch vaultCid
        create Note with
          writer = vault.owner
          text = "Forged by attacker"
```

```
test_no_laundering : Script ()
test_no_laundering = do
  alice <- allocateParty "Alice"
```

```
  mallory <- allocateParty "Mallory"

  -- Alice creates a Vault with Mallory as guardian
  vaultCid <- submit alice do
    createCmd Vault with
      owner = alice
      guardian = mallory

  -- Mallory creates an exploit contract
  exploitCid <- submit mallory do
    createCmd VaultExploit with
      attacker = mallory
      vaultCid = vaultCid
```

```
  -- FAIL: The exploit cannot forge Alice's authority.
  -- Peeking at the Vault (nonconsuming) does NOT
  -- release Alice's signatory authority.
  submitMustFail mallory do
    exerciseCmd exploitCid TryExploit

  -- Even with a consuming choice, the authority
  -- only flows within that choice's sub-transaction.
  -- It does not leak to sibling operations.

  pure ()
```

The key insight: exercising a nonconsuming choice on a contract does *not* release the signatories' authority. And even with consuming choices, the authority flows within the choice's execution scope—it does not propagate arbitrarily up or sideways.

8.8 Test 6: Common Pitfalls and Fixes

Let us look at patterns that commonly trip up Daml developers.

8.8.1 Pitfall: Creating Contracts for Other Parties

```
-- WRONG: Alice tries to give Bob a gift
template Gift
  with
    giver : Party
    receiver : Party
    item : Text
  where
    signatory giver, receiver  -- Both must agree!
```

```
test_gift_pitfall : Script ()
test_gift_pitfall = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  -- FAIL: Alice cannot make Bob a signatory
  --       without his consent
  submitMustFail alice do
    createCmd Gift with
      giver = alice
      receiver = bob
      item = "A book"

  pure ()
```

Fix: use the propose-accept pattern.

```
template GiftOffer
  with
    giver : Party
    receiver : Party
    item : Text
  where
    signatory giver
    observer receiver

    choice AcceptGift : ContractId Gift
      controller receiver
      do
        create Gift with giver, receiver, item

    choice DeclineGift : ()
      controller receiver
      do pure ()
```

```
test_gift_fixed : Script ()
test_gift_fixed = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  -- Alice offers the gift
  offerCid <- submit alice do
    createCmd GiftOffer with
      giver = alice
      receiver = bob
      item = "A book"

  -- Bob accepts
  submit bob do
    exerciseCmd offerCid AcceptGift

  pure ()
```

8.8.2 Pitfall: Modifying Contracts You Observe

```
template Bulletin
  with
    poster : Party
    readers : [Party]
    message : Text
  where
    signatory poster
    observer readers

    nonconsuming choice ViewMessage : Text
      with
        viewer : Party
      controller viewer
      do
        assertMsg "Not a reader"
          (viewer `elem` readers || viewer == poster)
        pure message
```

```
test_observer_limits : Script ()
test_observer_limits = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  bCid <- submit alice do
    createCmd Bulletin with
      poster = alice
      readers = [bob]
      message = "Hello world"

  -- Bob can read (nonconsuming, no authority needed)
  msg <- submit bob do
    exerciseCmd bCid ViewMessage with viewer = bob
  assertMsg "Should see message" (msg == "Hello world")

  -- FAIL: Bob cannot archive Alice's Bulletin.
  -- Archiving requires Alice's authority (signatory).
  submitMustFail bob do
    archiveCmd bCid

  -- OK: Alice (the signatory) can archive
  submit alice do
    archiveCmd bCid

  pure ()
```

The lesson: an observer can *see* a contract and exercise nonconsuming choices, but cannot archive it. Archiving requires all signatories' authority, which observers do not have.

8.8.3 Pitfall: Forgetting Observer Visibility

```
template Secret
  with
    owner : Party
    content : Text
  where
    signatory owner
    -- No observers!

    choice Reveal : ContractId Secret
      with
        viewer : Party
      controller owner
      do
        create this
```

```
test_visibility_pitfall : Script ()
test_visibility_pitfall = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  secretCid <- submit alice do
    createCmd Secret with
      owner = alice
      content = "Top secret"

  -- Bob cannot see or act on this contract at all.
  -- queryContractId returns None for non-stakeholders.
  result <- queryContractId @Secret bob secretCid
```

```
  assertMsg "Bob should not see the secret"
    (result == None)

  pure ()
```

The `Reveal` choice accepts a `viewer` party but does nothing with it—it just recreates the same contract with no observers. The viewer still cannot see it. This is a common mistake: naming a parameter `viewer` does not grant visibility.

Fix: add an `observers` field and populate it in the choice:

```
template Secret
  with
    owner : Party
    content : Text
    readers : [Party]
  where
    signatory owner
    observer readers

    choice Reveal : ContractId Secret
      with viewer : Party
      controller owner
      do create this with
           readers = viewer :: readers
```

8.9 Summary

- **All signatories must authorize creation.** You cannot bind a party to a contract without their consent. Use propose-accept when multiple signatories are needed.
- **Only the controller can exercise a choice.** Other parties are rejected at the protocol level.
- **Archiving requires signatory authority.** Consuming choices implicitly archive the contract, which requires all signatories' authority.
- **Authority flows from two sources**: the submitting party and the signatories of consumed contracts.
- **Authority is not transitive.** Nonconsuming choices do not release signatory authority. You cannot chain exercises to launder authority.
- **Observers can see but not modify.** Visibility does not imply authority.
- `submitMustFail` is your primary tool for testing authorization boundaries. Use it to verify that unauthorized actions are properly rejected.

Chapter 9

The Propose-Accept Pattern

Chapter 3 introduced propose-accept as the solution for getting multiple parties to agree on a contract. This chapter does a deep dive. We will build a job marketplace where employers post jobs, candidates apply, employers make offers, and candidates accept—multiple propose-accept flows interacting in a realistic workflow.

Along the way, we will explore variations of the pattern: propose-accept-reject, propose-with-expiry, and multi-step chains. By the end, you will have a toolkit for modeling any multi-party agreement in Daml.

Create the project:

```
dpm new job-marketplace
cd job-marketplace
```

Replace the contents of `daml/Main.daml` with the code we build below.

9.1 Why Propose-Accept?

Recall the fundamental rule from Chapter 8: a contract cannot be created unless all signatories authorize the transaction. When a contract needs two or more signatories, no single party can create it unilaterally.

The propose-accept pattern solves this in two steps:

1. **Propose**: one party creates a proposal contract. Only the proposer signs it.
2. **Accept**: the other party exercises a choice on the proposal, which creates the final contract with both parties as signatories.

This works because exercising `Accept` consumes the proposal (releasing the proposer's signatory authority) while the accepter provides their authority as the controller. The combined authority satisfies the two-signatory requirement.

The Standard Solution

Propose-accept is not a workaround or a hack. It is the standard, idiomatic way to create multi-party contracts in Daml. Every real-world Daml application uses it. If you need multiple parties to agree on something, this is the pattern.

9.2 The Job Marketplace Domain

Our marketplace has three phases, each using propose-accept:

1. **Job Posting**: an employer posts a job (single signatory). Candidates can see it.
2. **Application**: a candidate applies to a job. The employer sees the application and decides.
3. **Offer and Acceptance**: the employer makes an offer. The candidate accepts or declines. An accepted offer produces an `Employment` contract signed by both.

9.3 Data Types

```
module Main where

import Daml.Script
import DA.List (head)

data JobType
  = FullTime
  | PartTime
  | Contract
    deriving (Eq, Show)
```

```
data OfferTerms = OfferTerms with
    salary : Decimal
    jobType : JobType
    startDate : Text
    notes : Optional Text
  deriving (Eq, Show)
```

9.4 Phase 1: Job Posting

A job posting is a single-signatory contract. The employer creates it unilaterally.

```
template JobPosting
  with
    employer : Party
    title : Text
    description : Text
    jobType : JobType
    salaryRange : (Decimal, Decimal)
    candidates : [Party]
  where
    signatory employer
    observer candidates

    ensure fst salaryRange <= snd salaryRange
      && fst salaryRange > 0.0
```

```
    -- Employer can add visible candidates
    nonconsuming choice AddCandidate
      : ContractId JobPosting
      with
        candidate : Party
      controller employer
      do
        assertMsg "Already a candidate"
          (notElem candidate candidates)
        archive self
        create this with
```

```
        candidates = candidate :: candidates
```

Why nonconsuming + archive self?

You might wonder why `AddCandidate` is declared `nonconsuming` yet immediately calls `archive self`. A `nonconsuming` choice does not automatically archive the contract—but the body does it manually, then creates a replacement with the updated candidate list.

This achieves the same result as a regular consuming choice, which archives automatically:

```
    choice AddCandidate : ContractId JobPosting
      with
        candidate : Party
      controller employer
      do
        assertMsg "Already a candidate"
          (notElem candidate candidates)
        create this with
          candidates = candidate :: candidates
```

Both versions are valid Daml. The consuming version is more concise because the archive is implicit. The `nonconsuming` version makes the archive-and-recreate steps explicit, which can be clearer when reading the code for the first time. We use the explicit form here so you can see exactly what happens; in production code, prefer the consuming version.

```
  -- Employer closes the posting
  choice ClosePosting : ()
    controller employer
    do pure ()
```

Notice that the employer is the sole signatory. They can create, modify, and close the posting without anyone else's consent. Candidates are observers—they can see the posting but cannot change it.

9.5 Phase 2: Application (Propose)

When a candidate applies, they *propose* to the employer. The application is a proposal contract: the candidate is the sole signatory, and the employer is an observer who can accept or reject.

```
template Application
  with
    candidate : Party
    employer : Party
    jobTitle : Text
    coverLetter : Text
    experience : Int
  where
    signatory candidate
    observer employer
```

```
  -- Employer accepts the application
  -- (moves candidate to interview stage)
  choice Shortlist
    : ContractId ShortlistedCandidate
    controller employer
```

```
      do
        create ShortlistedCandidate with
          employer
          candidate
          jobTitle
          experience

    -- Employer rejects
    choice RejectApplication : ()
      with
        reason : Text
      controller employer
      do pure ()
```

```
    -- Candidate withdraws
    choice WithdrawApplication : ()
      controller candidate
      do pure ()
```

This is the propose-accept-reject variation. The candidate proposes (creates the `Application`), and the employer has three options: shortlist, reject, or ignore. The candidate can also withdraw.

```
template ShortlistedCandidate
  with
    employer : Party
    candidate : Party
    jobTitle : Text
    experience : Int
  where
    signatory employer
    observer candidate
```

```
    -- Employer makes a formal offer
    choice MakeOffer : ContractId JobOffer
      with
        terms : OfferTerms
      controller employer
      do
        create JobOffer with
          employer
          candidate
          jobTitle
          terms

    -- Employer decides not to proceed
    choice PassOnCandidate : ()
      controller employer
      do pure ()
```

9.6 Phase 3: Offer and Acceptance

The job offer is the second propose-accept cycle. This time the employer proposes (makes the offer) and the candidate decides.

```
template JobOffer
  with
    employer : Party
    candidate : Party
```

```
    jobTitle : Text
    terms : OfferTerms
  where
    signatory employer
    observer candidate

    ensure terms.salary > 0.0
```

```
    -- Candidate accepts: creates Employment contract
    choice AcceptOffer : ContractId Employment
      controller candidate
      do
        create Employment with
          employer
          employee = candidate
          jobTitle
          terms

    -- Candidate declines
    choice DeclineOffer : ()
      controller candidate
      do pure ()
```

```
    -- Candidate negotiates (counter-propose)
    choice NegotiateOffer
      : ContractId JobOffer
      with
        counterSalary : Decimal
        counterNotes : Text
      controller candidate
      do
        -- Archive the current offer, create a new one
        -- with updated terms. Now the employer is the
        -- observer and must accept the counter.
        create CounterOffer with
          employer
          candidate
          jobTitle
          originalTerms = terms
          counterSalary
          counterNotes
        -- The old offer is consumed (archived)
        -- Return value is the CounterOffer CID
        -- but we declared JobOffer return type,
        -- so we adjust the flow:
        create JobOffer with
          employer
          candidate
          jobTitle
          terms = terms with
            salary = counterSalary
            notes = Some counterNotes
```

```
    -- Employer withdraws the offer
    choice WithdrawOffer : ()
      controller employer
      do pure ()
```

```
-- Counter-offer record (for audit trail)
template CounterOffer
```

```
  with
    employer : Party
    candidate : Party
    jobTitle : Text
    originalTerms : OfferTerms
    counterSalary : Decimal
    counterNotes : Text
  where
    signatory candidate
    observer employer

    choice AcceptCounter : ContractId Employment
      controller employer
      do
        create Employment with
          employer
          employee = candidate
          jobTitle
          terms = originalTerms with
            salary = counterSalary
            notes = Some counterNotes
```

```
    choice RejectCounter : ()
      controller employer
      do pure ()
```

9.7 The Final Contract

`Employment` is the dual-signatory contract that both parties must agree to. It can only be created through the propose-accept flow—never directly.

```
template Employment
  with
    employer : Party
    employee : Party
    jobTitle : Text
    terms : OfferTerms
  where
    signatory employer, employee

    -- Either party can initiate termination
    -- (but both signatures mean both agreed
    -- to the employment in the first place)
    choice Terminate : ()
      with
        reason : Text
        initiator : Party
      controller initiator
      do
        assertMsg "Must be employer or employee"
          (initiator == employer
            || initiator == employee)
        pure ()
```

Dual Signatories from Propose-Accept

The `Employment` contract has both employer and employee as signatories. Neither can create it alone. It comes into existence only when the candidate exercises `AcceptOffer` on a `JobOffer`. At that moment, the employer's authority flows from the consumed `JobOffer` (where the employer is signatory), and the candidate's authority comes from being the controller. This is the same mechanism as the IOU propose-accept in Chapter 3, applied to a richer domain.

9.8 Variation: Propose-Accept-Reject

We already built this into the `Application` template. The pattern has three choices on the proposal:

1. **Accept** (controller: accepter) — creates the target contract.
2. **Reject** (controller: accepter) — archives the proposal with a reason.
3. **Withdraw** (controller: proposer) — the proposer cancels before the other party acts.

This is the most common variation. Almost every proposal in a production Daml application has all three options.

9.9 Variation: Propose-with-Expiry

Sometimes an offer should not stay open forever. We can model expiry with a date field and an assertion:

```
template TimedOffer
  with
    employer : Party
    candidate : Party
    terms : OfferTerms
    expiryDate : Text
  where
    signatory employer
    observer candidate
```

```
    choice AcceptTimedOffer
      : ContractId Employment
      with
        acceptDate : Text
      controller candidate
      do
        assertMsg "Offer has expired"
          (acceptDate <= expiryDate)
        create Employment with
          employer
          employee = candidate
          jobTitle = "Timed position"
          terms
```

```
    choice ExpireOffer : ()
      controller employer
      do pure ()
```

Time in Daml

Daml has a `getTime` function that returns the ledger effective time. In production, you would compare against ledger time rather than passing dates as parameters. We use text dates here for simplicity. Chapter 10 covers time handling in detail.

9.10 The Pattern as Architecture

Propose-accept is more than a coding pattern—it is an architectural principle. In a Daml application:

- Every multi-party agreement starts as a single-party proposal.
- The proposal makes the terms visible to the counterparty.
- The counterparty exercises their agency: accept, reject, negotiate.
- The final contract carries the authority of all parties who agreed.

This mirrors real-world contract formation: offer, consideration, acceptance. Daml makes this explicit in the code and enforceable at the protocol level.

9.11 Complete Test Script

```
test_full_workflow : Script ()
test_full_workflow = do
  acme <- allocateParty "Acme Corp"
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  -- Employer posts a job
  jobCid <- submit acme do
    createCmd JobPosting with
      employer = acme
      title = "Senior Engineer"
      description = "Build Daml applications"
      jobType = FullTime
      salaryRange = (100000.0, 150000.0)
      candidates = [alice, bob]
```

```
  -- Alice applies
  appAlice <- submit alice do
    createCmd Application with
      candidate = alice
      employer = acme
      jobTitle = "Senior Engineer"
      coverLetter = "Experienced Daml developer"
      experience = 5

  -- Bob applies
  appBob <- submit bob do
    createCmd Application with
      candidate = bob
      employer = acme
      jobTitle = "Senior Engineer"
      coverLetter = "Blockchain enthusiast"
      experience = 3
```

```
  -- Employer shortlists Alice, rejects Bob
  shortAlice <- submit acme do
    exerciseCmd appAlice Shortlist

  submit acme do
    exerciseCmd appBob RejectApplication with
      reason = "Insufficient experience"

  -- Employer makes Alice an offer
  offerCid <- submit acme do
    exerciseCmd shortAlice MakeOffer with
      terms = OfferTerms with
        salary = 120000.0
        jobType = FullTime
        startDate = "2026-04-01"
        notes = None
```

```
  -- Alice negotiates
  counterCid <- submit alice do
    exerciseCmd offerCid NegotiateOffer with
      counterSalary = 135000.0
      counterNotes = "Requesting market rate"

  -- Verify the counter-offer exists
  counters <- query @CounterOffer acme
  assertMsg "Should have a counter-offer"
    (not (null counters))
```

```
  -- Employer accepts the counter-offer
  let (counterOfferCid, _) = DA.List.head counters
  emplCid <- submit acme do
    exerciseCmd counterOfferCid AcceptCounter

  -- Verify employment contract exists
  Some empl <- queryContractId acme emplCid
  assertMsg "Salary should be 135000"
    (empl.terms.salary == 135000.0)
  assertMsg "Both parties are signatories"
    (empl.employer == acme
      && empl.employee == alice)

  pure ()
```

```
test_withdrawal : Script ()
test_withdrawal = do
  acme <- allocateParty "Acme Corp"
  alice <- allocateParty "Alice"

  -- Alice applies
  appCid <- submit alice do
    createCmd Application with
      candidate = alice
      employer = acme
      jobTitle = "Engineer"
      coverLetter = "Interested in the role"
      experience = 2

  -- Alice changes her mind and withdraws
  submit alice do
    exerciseCmd appCid WithdrawApplication
```

```
  -- The application is archived: employer
  -- can no longer act on it
  submitMustFail acme do
    exerciseCmd appCid Shortlist

  pure ()
```

```
test_offer_decline : Script ()
test_offer_decline = do
  acme <- allocateParty "Acme Corp"
  bob <- allocateParty "Bob"

  -- Skip straight to an offer for brevity
  offerCid <- submit acme do
    createCmd JobOffer with
      employer = acme
      candidate = bob
      jobTitle = "Junior Dev"
      terms = OfferTerms with
        salary = 70000.0
        jobType = FullTime
        startDate = "2026-05-01"
        notes = Some "Entry-level position"
```

```
  -- Bob declines
  submit bob do
    exerciseCmd offerCid DeclineOffer

  -- Offer is archived
  result <- queryContractId @JobOffer acme offerCid
  assertMsg "Offer should be archived"
    (result == None)

  pure ()
```

```
test_employer_withdraw : Script ()
test_employer_withdraw = do
  acme <- allocateParty "Acme Corp"
  charlie <- allocateParty "Charlie"

  offerCid <- submit acme do
    createCmd JobOffer with
      employer = acme
      candidate = charlie
      jobTitle = "Intern"
      terms = OfferTerms with
        salary = 40000.0
        jobType = Contract
        startDate = "2026-06-01"
        notes = None

  -- Employer withdraws before candidate responds
  submit acme do
    exerciseCmd offerCid WithdrawOffer

  -- Charlie can no longer accept
  submitMustFail charlie do
    exerciseCmd offerCid AcceptOffer

  pure ()
```

Build and test:

```
dpm build
dpm test
```

9.12 Summary

- **Propose-accept is the standard pattern** for creating multi-signatory contracts. One party proposes, the other accepts, and the combined authority satisfies all signatory requirements.
- **Propose-accept-reject** adds a rejection choice and a withdrawal choice, giving both parties full agency over the proposal lifecycle.
- **Propose-with-expiry** adds time limits to proposals, preventing stale offers from lingering on the ledger.
- **Negotiation** is modeled as a chain of proposals: the original offer, a counter-offer, and an acceptance of the counter.
- **Multiple propose-accept flows** can interact in a single application. Our job marketplace chains three: application, shortlisting, and offer acceptance.
- The pattern mirrors real-world contract formation: offer, consideration, acceptance. Daml makes this explicit, enforceable, and auditable.
- Build on the IOU examples from Chapter 3: the mechanics are identical, only the domain differs. Every multi-party workflow in Daml is a variation of propose-accept.

Chapter 10

Composing Choices

In Chapter 3 we saw that a choice body can create new contracts. But choice bodies can do much more—they can exercise other choices, fetch contracts, and chain multiple operations into a single atomic transaction. This chapter builds an atomic swap: Alice trades USD for Bob's EUR in one indivisible step.

Create the project for this chapter:

```
dpm new composing-choices \
  --template daml-intro-composing
cd composing-choices
```

If the template is not available, create a new empty project and follow along:

```
dpm new composing-choices
cd composing-choices
```

10.1 The Asset Template

We need a fungible asset that supports splitting and merging. Here is the complete template:

```
module Asset where

import DA.Assert

template Asset
  with
    issuer   : Party
    owner    : Party
    symbol   : Text
    quantity : Decimal
    observers : [Party]
  where
    signatory issuer, owner
    observer observers
    ensure quantity > 0.0
```

The `ensure` clause guarantees no zero-quantity assets exist on the ledger. Now we add choices for splitting and merging.

10.1.1 The Split Choice

Splitting divides one asset into two pieces. The original contract is consumed and two new ones are created:

```
choice Split : (ContractId Asset, ContractId Asset)
  with
    splitQuantity : Decimal
  controller owner
  do
    assertMsg "Split quantity must be positive"
      (splitQuantity > 0.0)
    assertMsg "Split quantity must be less than total"
      (splitQuantity < quantity)
    cid1 <- create this with
      quantity = splitQuantity
    cid2 <- create this with
      quantity = quantity - splitQuantity
    return (cid1, cid2)
```

The return type is a tuple of two contract IDs—one for each piece. The caller decides what to do with them.

10.1.2 The Merge Choice

Merging combines two assets of the same type into one:

```
choice Merge : ContractId Asset
  with
    otherCid : ContractId Asset
  controller owner
  do
    other <- fetch otherCid
    assertMsg "Cannot merge: issuer mismatch"
      (other.issuer == issuer)
    assertMsg "Cannot merge: owner mismatch"
      (other.owner == owner)
    assertMsg "Cannot merge: symbol mismatch"
      (other.symbol == symbol)
    archive otherCid
    create this with
      quantity = quantity + other.quantity
```

Fetch Inside a Choice

The `fetch` function retrieves a contract's payload by its ID. It does not archive the contract. It requires the caller to be a stakeholder (signatory or observer) of the fetched contract. Inside a choice body, the submitting party's authority plus the signatories' authority determine what can be fetched.

10.1.3 The Transfer Choice

Transfer changes the owner. We need the new owner to agree, so the choice requires both controllers:

```
choice Transfer : ContractId Asset
  with
    newOwner : Party
  controller owner, newOwner
  do
    assertMsg "New owner must differ from current"
      (newOwner /= owner)
    create this with
      owner = newOwner
```

```
        observers = []
```

10.2 Exercise Chains

The real power of choice composition is calling `exercise` inside a choice body. When you exercise one choice from within another, both operations happen in the same transaction—atomically.

Consider a `SplitAndTransfer` choice that splits off a specific amount and transfers it:

```
  choice SplitAndTransfer
    : (ContractId Asset, ContractId Asset)
    with
      amount : Decimal
      newOwner : Party
    controller owner, newOwner
    do
      -- Step 1: Split the asset
      (splitCid, remainCid) <- exercise self Split
        with splitQuantity = amount
      -- Step 2: Transfer the split portion
      transferredCid <- exercise splitCid Transfer
        with newOwner
      return (transferredCid, remainCid)
```

This is an *exercise chain*: one choice exercises another, which may exercise yet another. The entire chain runs as a single transaction. If any step fails, everything rolls back.

Atomic Transactions

Everything inside a choice body—creates, exercises, fetches, assertions—executes as a single atomic transaction. Either all operations succeed and are committed to the ledger, or none are. There is no partial execution. This is the foundation of safe multi-contract operations.

10.3 Authority Accumulation

When choices are chained, authority accumulates. Understanding how this works is essential for building correct workflows.

```
-- Inside Asset template:
  choice TransferToSelf : ContractId Asset
    with
      newOwner : Party
    controller owner
    do
      -- owner's authority is available here
      -- but newOwner's is NOT
      create this with owner = newOwner
```

This choice would fail at runtime because `create this with owner = newOwner` requires `newOwner` to be a signatory of the new contract. The choice controller (`owner`) does not carry `newOwner`'s authority.

Authority comes from two sources inside a choice body:

1. **Controllers** of the choice being exercised.
2. **Signatories** of the contract the choice belongs to.

When a choice exercises another choice, authority from the outer context flows into the inner exercise. This is how delegation works.

Source	Authority Provided
Choice controller	The controlling party
Contract signatories	All signatories of the consumed contract
Inner exercise	Controllers + signatories of the inner contract

Table 10.1: Sources of authority in a choice body

Authority Is Not Transitive

As we saw in Chapter 3, you cannot gain authority by chaining through contracts a party signed. Authority flows only from the direct controller of the choice and the signatories of the contract being exercised. This prevents privilege escalation.

10.4 Delegation with Role Contracts

A role contract grants standing permission for one party to act on behalf of another. We saw a simple version in Chapter 3 with `IouSender`. Here is a more general pattern:

```
module Delegation where

import Asset

template TransferAgent
  with
    agent : Party
    owner : Party
    issuer : Party
    symbol : Text
  where
    signatory owner
    observer agent

    nonconsuming choice AgentTransfer
      : ContractId Asset
      with
        assetCid : ContractId Asset
        newOwner : Party
      controller agent, newOwner
      do
        asset <- fetch assetCid
        assertMsg "Agent: wrong owner"
          (asset.owner == owner)
        assertMsg "Agent: wrong issuer"
          (asset.issuer == issuer)
        assertMsg "Agent: wrong symbol"
          (asset.symbol == symbol)
        exercise assetCid Transfer
          with newOwner
```

How Delegation Works

The `TransferAgent` contract is a *power of attorney*: the owner signs it once, and from then on the agent can execute transfers without the owner co-signing each one. The owner deliberately steps out of the operational loop by not being a controller—the agent decides *when* and *to whom* to transfer.
When the agent and new owner exercise `AgentTransfer`, authority accumulates from three sources:

Party	Source of Authority	What It Enables
`agent`	Controller of `AgentTransfer`	Initiates the transfer—the party who "pulls the trigger"
`owner`	Signatory of `TransferAgent`	Satisfies `Transfer`'s `controller owner` requirement
`newOwner`	Co-controller of `AgentTransfer`	Satisfies `Transfer`'s `controller ..., newOwner` requirement

Notice that the agent does not directly satisfy any requirement of the inner `Transfer` choice. Their role is purely operational—without them as controller, nobody could trigger the delegation. The owner's authority flows from the role contract's signatory field, and the new owner's authority flows from being a co-controller.

10.5 Building an Atomic Swap

Now we combine everything to build an atomic currency swap. Alice has USD, Bob has EUR. They want to exchange in a single transaction—no trust required.

10.5.1 The Trade Template

```
module Trade where

import Asset

template TradeProposal
  with
    proposer     : Party
    counterparty : Party
    offeredCid   : ContractId Asset
    offeredSymbol : Text
    offeredQty   : Decimal
    requestedSymbol : Text
    requestedQty    : Decimal
  where
    signatory proposer
    observer counterparty

    choice TradeProposal_Cancel
      : ()
      controller proposer
      do return ()
```

The proposer creates the proposal, offering one of their assets and requesting a specific asset in return. The counterparty is an observer so they can see and act on the proposal.

10.5.2 The Accept Choice

The accept choice is where atomic composition happens:

```
choice TradeProposal_Accept
  : (ContractId Asset, ContractId Asset)
  with
    counterpartyCid : ContractId Asset
  controller counterparty
  do
    -- Verify the offered asset
    offered <- fetch offeredCid
    assertMsg "Offered asset: wrong proposer"
      (offered.owner == proposer)
    assertMsg "Offered asset: wrong symbol"
      (offered.symbol == offeredSymbol)
    assertMsg "Offered asset: insufficient qty"
      (offered.quantity >= offeredQty)

    -- Verify the counterparty's asset
    counter <- fetch counterpartyCid
    assertMsg "Counter asset: wrong owner"
      (counter.owner == counterparty)
    assertMsg "Counter asset: wrong symbol"
      (counter.symbol == requestedSymbol)
    assertMsg "Counter asset: wrong qty"
      (counter.quantity >= requestedQty)

    -- Atomic swap: transfer both
    newForCounterparty <- exercise offeredCid
      Transfer with newOwner = counterparty
    newForProposer <- exercise counterpartyCid
      Transfer with newOwner = proposer

    return (newForCounterparty, newForProposer)
```

Atomic Swap

When the counterparty exercises `TradeProposal_Accept`, the entire choice body runs as one atomic transaction. Both transfers happen or neither does. There is no state where Alice has sent her USD but has not yet received Bob's EUR. This eliminates settlement risk without a trusted intermediary.

10.5.3 Authority in the Swap

Let us trace the authority flow:

1. The counterparty exercises `TradeProposal_Accept`. Available authority: counterparty (controller) + proposer (signatory of `TradeProposal`).
2. The first `exercise offeredCid Transfer` needs `controller owner, newOwner`. The proposer is the owner (verified by assertion); the counterparty is the new owner. Both authorities are available.
3. The second `exercise counterpartyCid Transfer` needs the counterparty (owner) and proposer (new owner). Both are available.

Everything checks out. The swap is safe.

10.6 Testing the Swap

```
module TradeTest where

import Daml.Script
import Asset
import Trade

setup : Script (Party, Party, Party)
setup = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  return (bank, alice, bob)
```

First, create assets for both parties:

```
test_atomic_swap = do
  (bank, alice, bob) <- setup

  -- Bank issues USD to Alice (Bob is observer)
  let bankAlice = actAs bank <> actAs alice
  aliceUsd <- submit bankAlice do
    createCmd Asset with
      issuer = bank
      owner = alice
      symbol = "USD"
      quantity = 100.0
      observers = [bob]

  -- Bank issues EUR to Bob (Alice is observer)
  let bankBob = actAs bank <> actAs bob
  bobEur <- submit bankBob do
    createCmd Asset with
      issuer = bank
      owner = bob
      symbol = "EUR"
      quantity = 85.0
      observers = [alice]
```

Now Alice proposes a trade and Bob accepts:

```
  -- Alice proposes: 100 USD for 85 EUR
  proposal <- submit alice do
    createCmd TradeProposal with
      proposer = alice
      counterparty = bob
      offeredCid = aliceUsd
      offeredSymbol = "USD"
      offeredQty = 100.0
      requestedSymbol = "EUR"
      requestedQty = 85.0

  -- Bob accepts, providing his EUR asset
  (bobGotUsd, aliceGotEur) <- submit bob do
    exerciseCmd proposal TradeProposal_Accept
      with counterpartyCid = bobEur

  -- Verify: Bob now has USD, Alice has EUR
  Some usd <- queryContractId bob bobGotUsd
  assertMsg "Bob should have 100 USD"
    (usd.quantity == 100.0)
```

```
  Some eur <- queryContractId alice aliceGotEur
  assertMsg "Alice should have 85 EUR"
    (eur.quantity == 85.0)
```

Build and test:

```
dpm build
dpm test
```

10.7 Partial Swaps with Split

What if Alice only wants to trade 50 USD, not all 100? She can split first and then propose:

```
test_partial_swap = do
  (bank, alice, bob) <- setup

  -- Create assets (same as above, omitted for brevity)
  -- ...assume aliceUsd has 100 USD, bobEur has 85 EUR

  -- Alice splits off 50 USD
  (toTrade, toKeep) <- submit alice do
    exerciseCmd aliceUsd Split
      with splitQuantity = 50.0

  -- Propose trading just the 50 USD portion
  proposal <- submit alice do
    createCmd TradeProposal with
      proposer = alice
      counterparty = bob
      offeredCid = toTrade
      offeredSymbol = "USD"
      offeredQty = 50.0
      requestedSymbol = "EUR"
      requestedQty = 42.5

  -- Bob accepts
  submit bob do
    exerciseCmd proposal TradeProposal_Accept
      with counterpartyCid = bobEur
```

10.8 Multi-Step Composition Patterns

Beyond atomic swaps, choice composition enables several useful patterns.

10.8.1 Batch Operations

Process multiple items in a single transaction using `forA` (the monadic map for `Update`):

```
    choice BatchTransfer : [ContractId Asset]
      with
        cids : [ContractId Asset]
        newOwner : Party
      controller owner, newOwner
      do
        forA cids $ \cid -> do
          exercise cid Transfer
            with newOwner
```

10.8.2 Conditional Composition

Branch logic inside a choice:

```
choice SmartTransfer : ContractId Asset
  with
    amount : Decimal
    newOwner : Party
  controller owner, newOwner
  do
    if amount == quantity
      then exercise self Transfer
        with newOwner
      else do
        (splitCid, _remainCid) <-
          exercise self Split
            with splitQuantity = amount
        exercise splitCid Transfer
          with newOwner
```

If the requested amount equals the full quantity, transfer the whole asset. Otherwise, split first and transfer the portion.

10.9 Summary

- **Exercise chains** let one choice call another. The entire chain runs as a single atomic transaction.
- **Authority accumulates** from choice controllers and contract signatories. Inner exercises inherit outer authority.
- **Authority is not transitive**: you cannot gain a party's authority by chaining through contracts they signed.
- **Role contracts** grant standing permissions through delegation. The role contract's signatories provide authority to the agent's exercises.
- **Atomic swaps** use choice composition to exchange assets in a single indivisible transaction, eliminating settlement risk.
- **Split and Merge** choices make assets fungible, enabling partial trades and batch operations.
- **Conditional composition** lets choice bodies branch based on runtime values while maintaining atomicity.

Chapter 11

Error Handling and Constraints

Contracts will encounter invalid data, violated preconditions, and unexpected states. Daml provides several mechanisms for handling errors—from compile-time constraints to structured runtime failures. This chapter covers when and how to use each one.

Create the project for this chapter:

```
dpm new exceptions-demo
cd exceptions-demo
```

11.1 The Error Handling Spectrum

Daml offers five levels of error handling, from broadest to most targeted:

Mechanism	Scope	When to Use
`ensure`	Template-wide	Invariants that must always hold
`assertMsg`	Choice-level	Quick precondition checks
`abort`	Choice-level	Unconditional failure
`failWithStatus`	Choice-level	Structured errors with categories
`failWithStatusPure`	Pure context	Same, outside `Update`

Table 11.1: Error handling mechanisms

11.2 Template Invariants with ensure

We introduced `ensure` in Chapter 5. It enforces a condition that must be true whenever a contract is created:

```
module ErrorHandling where

import DA.Fail
import DA.TextMap (fromList)
import DA.Action (when)

template Account
  with
    bank    : Party
    owner   : Party
    name    : Text
    balance : Decimal
  where
```

```
    signatory bank, owner
    ensure balance >= 0.0
      && name /= ""
```

If the condition evaluates to `False`, the `create` fails and the entire transaction aborts. This applies everywhere the contract is created—in scripts, in choice bodies, in exercises.

ensure Is a Hard Invariant

An `ensure` clause is a guarantee: no contract of this type can ever exist on the ledger violating the condition. It is checked at creation time and cannot be bypassed. Use it for business rules that must always hold, such as non-negative balances or non-empty identifiers.

Multiple conditions can be combined with `&&`:

```
import qualified DA.Text as T

    ensure balance >= 0.0
      && name /= ""
      && T.length name <= 100
```

ensure Cannot Reference External State

The `ensure` expression can only use the template's own fields. It cannot fetch other contracts, call external services, or depend on anything outside the `with` block. For conditions that require external state, use `assertMsg` or `failWithStatus` in a choice body.

11.3 Choice-Level Validation with assertMsg

`assertMsg` checks a condition at exercise time. If the condition is `False`, the transaction aborts with the given message:

```
    choice Deposit : ContractId Account
      with
        amount : Decimal
        source : Text
      controller bank
      do
        assertMsg "Amount must be positive"
          (amount > 0.0)
        assertMsg "Exceeds deposit limit"
          (amount <= 1000000.0)
        assertMsg "Source must not be empty"
          (source /= "")
        create this with
          balance = balance + amount
```

The difference from `ensure`:

- `ensure` validates the *contract's data* at creation.
- `assertMsg` validates the *choice's arguments and context* at exercise time.

You can also use `assert` without a message, but `assertMsg` is strongly preferred because the message appears in error logs and makes debugging much easier:

```
-- Avoid: no useful error message
assert (amount > 0.0)

-- Prefer: clear error message
assertMsg "Amount must be positive" (amount > 0.0)
```

11.4 Unconditional Failure with abort

`abort` immediately fails the transaction with a message. Use it when you have already determined that the operation cannot proceed:

```
    choice Process : ContractId Account
      with
        action : Text
      controller owner
      do
        case action of
          "deposit" -> create this with
            balance = balance + 10.0
          "withdraw" -> create this with
            balance = balance - 10.0
          _ -> abort
            ("Unknown action: " <> action)
```

`abort` is useful in pattern matches and conditional branches where no valid path exists.

11.5 Structured Error Handling with failWithStatus

The mechanisms above—`assertMsg` and `abort`—produce plain text error messages. They work for simple cases, but callers cannot programmatically distinguish between error types or extract structured data from the failure.

`failWithStatus` solves this. It produces errors with a machine-readable error ID, a failure category, a human-readable message, and key-value metadata. Import the `DA.Fail` module to use it:

```
import DA.Fail
import DA.TextMap (fromList)
```

11.5.1 The FailureStatus Record

`FailureStatus` is a record with four fields:

Field	Type	Purpose
`errorId`	`Text`	Unique error identifier
`category`	`FailureCategory`	Whether the caller can retry
`message`	`Text`	Human-readable description
`meta`	`TextMap Text`	Key-value pairs for context

Table 11.2: FailureStatus record fields

11.5.2 Failure Categories

Daml defines two failure categories that map to different gRPC status codes:

Category	gRPC Code	Meaning
`invalidArgument`	`INVALID_ARGUMENT`	Bad input. Do not retry.
`failedPrecondition`	`FAILED_PRECONDITION`	State conflict. Retry later.

Table 11.3: Failure categories

The distinction matters for client applications: an `invalidArgument` error means the request itself is wrong (e.g., negative amount), while `failedPrecondition` means the request was valid but the ledger state does not allow it right now (e.g., insufficient balance).

11.5.3 Using failWithStatus in Choices

Here is a complete withdrawal choice using `failWithStatus`:

```
choice Withdraw : ContractId Account
  with
    amount : Decimal
  controller owner
  do
    when (amount <= 0.0) do
      failWithStatus FailureStatus with
        errorId =
          "myapp/invalid-amount"
        category = invalidArgument
        message =
          "Amount must be positive"
        meta = fromList
          [("amount", show amount)]
    when (amount > balance) do
      failWithStatus FailureStatus with
        errorId =
          "myapp/insufficient-funds"
        category = failedPrecondition
        message =
          "Insufficient balance"
        meta = fromList
          [ ("balance"
            , show balance)
          , ("requested"
            , show amount)
          ]
    create this with
      balance = balance - amount
```

Two Validations, Two Categories

The `Withdraw` choice validates two things. A non-positive amount is bad input (`invalidArgument`)—the caller should never send it. Insufficient balance is a state problem (`failedPrecondition`)—the caller might retry after the account receives a deposit. Choosing the right category helps downstream systems handle errors correctly.

11.5.4 Error ID Conventions

Error IDs should follow a DNS-like prefix convention so they are globally unique and easy to filter in logs:

```
-- Good: prefixed with your app or domain
```

```
"myapp/invalid-amount"
"myapp/insufficient-funds"
"acme.com/kyc/identity-not-verified"

-- Bad: generic, will collide across modules
"invalid-amount"
"error"
```

11.5.5 Metadata with meta

The `meta` field carries machine-readable context as key-value pairs. Use `DA.TextMap.fromList` to construct it:

```
meta = fromList
  [ ("balance", show balance)
  , ("requested", show amount)
  , ("account_owner", show owner)
  ]
```

For errors with no extra context, use an empty map:

```
meta = fromList []
```

Callers receive these key-value pairs in the gRPC error details or JSON API error response, enabling programmatic error handling without parsing human-readable messages.

11.5.6 failWithStatusPure for Pure Contexts

`failWithStatus` works inside the `Update` monad (choice bodies). For pure helper functions that do not perform ledger operations, use `failWithStatusPure`:

```
validatePositive : Text -> Decimal -> Decimal
validatePositive field val
  | val > 0.0 = val
  | otherwise =
      failWithStatusPure FailureStatus with
        errorId =
          "myapp/non-positive-value"
        category = invalidArgument
        message =
          field <> " must be positive"
        meta = fromList
          [ ("field", field)
          , ("value", show val)
          ]
```

This function can be called from choice bodies or other pure code. If the validation fails, the transaction aborts with the structured error.

11.5.7 How Callers Receive the Error

When `failWithStatus` fires, the error propagates to the caller through the API they used to submit the command:

- **gRPC Ledger API:** The error arrives as a gRPC status with the appropriate code (`INVALID_ARGUMENT` or `FAILED_PRECONDITION`). The `errorId`, `message`, and `meta` fields appear in the error details.
- **JSON API:** The error appears in the HTTP response with a corresponding status code.

The structured fields are included in the JSON error body.

- **Daml Script (`submitMustFail`):** The submission fails as expected. The error message includes the structured information.

11.6 Common Error Patterns

11.6.1 Validate-Then-Act

The most common pattern: check everything before modifying state.

```
    choice Transfer : ContractId Account
      with
        amount : Decimal
        recipientName : Text
      controller owner
      do
        when (amount <= 0.0) do
          failWithStatus FailureStatus with
            errorId =
              "myapp/invalid-amount"
            category = invalidArgument
            message =
              "Amount must be positive"
            meta = fromList
              [("amount", show amount)]
        when (recipientName == "") do
          failWithStatus FailureStatus with
            errorId =
              "myapp/empty-recipient"
            category = invalidArgument
            message =
              "Recipient required"
            meta = fromList []
        when (amount > balance) do
          failWithStatus FailureStatus with
            errorId =
              "myapp/insufficient-funds"
            category = failedPrecondition
            message =
              "Insufficient balance"
            meta = fromList
              [ ("balance"
                , show balance)
              , ("requested"
                , show amount)
              ]
        create this with
          balance = balance - amount
```

11.6.2 Reusable Validation Helpers

Factor common validations into helper functions to reduce repetition:

```
requirePositive
  : Text -> Decimal -> Update ()
requirePositive label val =
  when (val <= 0.0) do
    failWithStatus FailureStatus with
      errorId =
```

```
        "myapp/non-positive-value"
      category = invalidArgument
      message =
        label <> " must be positive"
      meta = fromList
        [ ("field", label)
        , ("value", show val)
        ]

requireNonEmpty
  : Text -> Text -> Update ()
requireNonEmpty label val =
  when (val == "") do
    failWithStatus FailureStatus with
      errorId = "myapp/empty-field"
      category = invalidArgument
      message =
        label <> " must not be empty"
      meta = fromList
        [("field", label)]
```

Then use them in choices:

```
    choice QuickWithdraw : ContractId Account
      with
        amount : Decimal
      controller owner
      do
        requirePositive "amount" amount
        when (amount > balance) do
          failWithStatus FailureStatus with
            errorId =
              "myapp/insufficient-funds"
            category = failedPrecondition
            message =
              "Insufficient balance"
            meta = fromList
              [ ("balance"
                , show balance)
              , ("requested"
                , show amount)
              ]
        create this with
          balance = balance - amount
```

11.7 Choosing the Right Approach

Approach	When to Use	Retryable?
`ensure`	Template invariant, always true	N/A
`assertMsg`	Quick validation in choices	No
`abort`	Unconditional failure	No
`failWithStatus` + `invalidArgument`	Bad input from caller	No
`failWithStatus` + `failedPrecondition`	Ledger state conflict	Yes

Table 11.4: Choosing an error handling approach

When to Use failWithStatus vs assertMsg

Use `assertMsg` for simple, internal precondition checks where no structured error data is needed. Use `failWithStatus` when callers (APIs, UIs, other services) need to programmatically distinguish error types, extract metadata, or decide whether to retry. In production applications, prefer `failWithStatus` for any user-facing validation.

11.8 Testing Error Cases

Always test that your error handling works. Use `submitMustFail` to verify that invalid operations are rejected:

```
module ErrorHandlingTest where

import Daml.Script
import ErrorHandling

test_ensure_rejects_negative = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  submitMustFail bank do
    createCmd Account with
      bank
      owner = alice
      name = "Savings"
      balance = -10.0

test_failWithStatus_bad_amount = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  let parties = actAs bank <> actAs alice
  cid <- submit parties do
    createCmd Account with
      bank
      owner = alice
      name = "Savings"
      balance = 50.0
  -- Zero amount triggers invalidArgument
  submitMustFail alice do
    exerciseCmd cid Withdraw
      with amount = 0.0

test_failWithStatus_overdraw = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  let parties = actAs bank <> actAs alice
  cid <- submit parties do
    createCmd Account with
      bank
      owner = alice
      name = "Savings"
      balance = 50.0
  -- Overdraw triggers failedPrecondition
  submitMustFail alice do
    exerciseCmd cid Withdraw
      with amount = 100.0

test_successful_withdraw = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  let parties = actAs bank <> actAs alice
  cid <- submit parties do
    createCmd Account with
```

```
      bank
      owner = alice
      name = "Savings"
      balance = 50.0
  cid2 <- submit alice do
    exerciseCmd cid Withdraw
      with amount = 25.0
  -- Only 25.0 left; 30.0 should fail
  submitMustFail alice do
    exerciseCmd cid2 Withdraw
      with amount = 30.0
```

Build and test:

```
dpm build
dpm test
```

Legacy: Exceptions (SDK 2.x)

Earlier Daml versions supported custom exception types with `exception`, `throw`, and `try/catch` blocks. These are deprecated in SDK 3.4 and will be removed in a future release. Use `failWithStatus` instead—it provides structured errors with categories, machine-readable metadata, and proper gRPC status code mapping. If you encounter legacy code using exceptions, migrate each `throw` to a `failWithStatus` call with the appropriate category and error ID.

11.9 Summary

- **`ensure`** enforces template-level invariants at creation time. No contract can exist on the ledger that violates its `ensure` clause.
- **`assertMsg`** validates conditions inside choice bodies. Prefer it over `assert` for better error messages. Best for simple internal checks.
- **`abort`** unconditionally fails a transaction with a message. Use it for dead-end code paths.
- **`failWithStatus`** is the primary structured error mechanism. It produces errors with an `errorId`, a `category` (`invalidArgument` or `failedPrecondition`), a human-readable `message`, and key-value `metadata`.
- Use **`invalidArgument`** when the request itself is wrong (bad input). Use **`failedPrecondition`** when the ledger state prevents the operation (retry may succeed).
- **`failWithStatusPure`** works in pure (non-`Update`) contexts for validation helpers.
- Callers receive structured errors through gRPC status codes and JSON API error responses, enabling programmatic error handling.
- Use `submitMustFail` in tests to verify that invalid operations are properly rejected.
- Follow the **validate-then-act** pattern: check all preconditions before modifying ledger state.
- Factor repeated validations into **reusable helper functions** to keep choice bodies clean.

Chapter 12

Daml Script and Testing

Every chapter so far has included small test scripts. This chapter covers Daml Script in depth: how to write proper test suites, organize test code, and verify both happy paths and error cases.

Create the project for this chapter:

```
dpm new testing-demo
cd testing-demo
```

12.1 What Is Daml Script?

Daml Script is a domain-specific language for interacting with a Daml ledger. It runs in two modes:

- **Sandbox mode**: `dpm test` runs scripts against an in-memory simulated ledger. No Canton needed. This is your primary development feedback loop.
- **Live mode**: scripts can run against a real Canton ledger via the JSON Ledger API. We will cover this in later chapters.

Every top-level value of type `Script a` in your project is a test case. When you run `dpm test`, all of them execute.

```
module MyTest where

import Daml.Script

-- This IS a test (type: Script ())
test_something = do
  alice <- allocateParty "Alice"
  debug "Hello from the test"

-- This is NOT a test (type: Int)
someHelper : Int
someHelper = 42
```

12.2 Core Script Functions

12.2.1 Party Allocation

`allocateParty` creates a party in the test environment:

```
test_parties = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  bank <- allocateParty "Bank"
  -- alice, bob, bank are now Party values
  debug (show alice)
```

Each call creates a distinct party. The string argument is a display name—it does not have to be unique, but using unique names makes test output easier to read.

12.2.2 Submitting Commands

`submit` sends commands to the ledger as a specific party:

```
test_submit = do
  alice <- allocateParty "Alice"
  -- submit returns the result of the command
  cid <- submit alice do
    createCmd Token with owner = alice
  debug (show cid)
```

For commands requiring multiple parties, combine `actAs` and `readAs` modifiers with `submit`:

```
test_multi_submit = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  -- actAs: parties that provide authority
  -- readAs: parties that can read, no authority
  let parties = actAs alice <> actAs bob
  cid <- submit parties do
    createCmd Agreement with
      partyA = alice
      partyB = bob
      terms = "We agree on this"
  pure ()
```

submit with actAs and readAs

`submit party do` submits as a single `actAs` party. To submit as multiple parties, combine `actAs` modifiers with `<>`: `submit (actAs alice <> actAs bob) do`. Use `readAs` for parties that need visibility but not authority: `submit (actAs alice <> readAs bob) do`.

12.2.3 Testing Failure

`submitMustFail` asserts that a command *should* fail. The test passes if the command is rejected; it fails if the command succeeds:

```
test_must_fail = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  -- Alice cannot create a contract with Bob as signatory
  submitMustFail alice do
    createCmd Agreement with
      partyA = alice
      partyB = bob
      terms = "Bob didn't agree"
```

This is essential for testing authorization rules and validation logic.

12.3 Command Types

Inside a `submit` block, you use command functions:

Command	Description
`createCmd`	Create a new contract
`exerciseCmd`	Exercise a choice on a contract by ID
`archiveCmd`	Archive a contract (sugar for `exerciseCmd Archive`)
`createAndExerciseCmd`	Create a contract and immediately exercise a choice on it

Table 12.1: Daml Script command types

```
test_commands = do
  alice <- allocateParty "Alice"

  -- createCmd
  cid <- submit alice do
    createCmd Token with owner = alice

  -- exerciseCmd
  submit alice do
    exerciseCmd cid Archive

  -- createAndExerciseCmd: useful for one-shot helpers
  result <- submit alice do
    createAndExerciseCmd
      (Token with owner = alice)
      Archive
  -- result is () because Archive returns ()
  return ()
```

12.4 Querying the Ledger

Scripts can read ledger state to verify outcomes.

12.4.1 queryFilter

Find all active contracts of a given type visible to a party, filtered by a predicate:

```
test_query_filter = do
  alice <- allocateParty "Alice"
  bank <- allocateParty "Bank"
  let parties = actAs bank <> actAs alice
  submit parties do
    createCmd Asset with
      issuer = bank
      owner = alice
      symbol = "USD"
      quantity = 100.0
      observers = []
  submit parties do
    createCmd Asset with
      issuer = bank
      owner = alice
      symbol = "EUR"
      quantity = 50.0
```

```
      observers = []

  -- Find Alice's USD assets
  usdAssets <- queryFilter @Asset alice
    (\a -> a.symbol == "USD")
  assertMsg "Should find 1 USD asset"
    (length usdAssets == 1)
```

`queryFilter` returns a list of `(ContractId t, t)` pairs—the contract ID and the contract payload.

12.4.2 queryContractId

Look up a specific contract by its ID:

```
test_query_contract_id = do
  alice <- allocateParty "Alice"
  cid <- submit alice do
    createCmd Token with owner = alice

  -- Returns Some payload if the contract exists
  mbToken <- queryContractId alice cid
  case mbToken of
    Some token ->
      assertMsg "Owner should be Alice"
        (token.owner == alice)
    None ->
      abort "Contract not found"
```

`queryContractId` returns `Some payload` if the contract is active and visible to the querying party, or `None` if it has been archived or is not visible.

12.4.3 queryFilter for Lookups

Since contract keys are not available in SDK 3.x, use `queryFilter` to find contracts by field values:

```
test_query_by_fields = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  let parties = actAs bank <> actAs alice
  submit parties do
    createCmd Asset with
      issuer = bank
      owner = alice
      symbol = "USD"
      quantity = 100.0
      observers = []

  -- Find by field values (replaces key-based lookups from SDK 2.x)
  usdAssets <- queryFilter @Asset bank
    (\a -> a.issuer == bank
        && a.owner == alice
        && a.symbol == "USD")
  assertMsg "Should find 1 USD asset"
    (length usdAssets == 1)

  -- Non-existent returns empty list
  gbpAssets <- queryFilter @Asset bank
    (\a -> a.symbol == "GBP")
  assertMsg "GBP should not exist"
    (null gbpAssets)
```

Query Visibility

All query functions operate from a specific party's perspective. A party can only see contracts where they are a *stakeholder* (signatory or observer). Consider what happens when Bob—who is neither a signatory nor an observer—tries to query the same asset:

```
test_query_visibility = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  let parties = actAs bank <> actAs alice
  submit parties do
    createCmd Asset with
      issuer = bank
      owner = alice
      symbol = "USD"
      quantity = 100.0
      observers = []  -- Bob is NOT an observer

  -- Bank can see it (signatory)
  bankView <- queryFilter @Asset bank
    (\a -> a.symbol == "USD")
  assertMsg "Bank can see the asset"
    (length bankView == 1)

  -- Bob cannot see it (not a stakeholder)
  bobView <- queryFilter @Asset bob
    (\a -> a.symbol == "USD")
  assertMsg "Bob cannot see the asset"
    (null bobView)
```

Bob's query returns an empty list—not an error, just no results. The asset exists on the ledger, but Bob has no visibility. To let Bob see it, add him to `observers = [bob]`.

12.5 Time Manipulation

`passTime` advances the simulated ledger clock:

```
import DA.Time
import DA.Date

test_time = do
  alice <- allocateParty "Alice"
  now <- getTime
  debug ("Start time: " <> show now)

  passTime (days 30)
  later <- getTime
  debug ("After 30 days: " <> show later)

  passTime (hours 6)
  evenLater <- getTime
  debug ("Plus 6 hours: " <> show evenLater)
```

Time functions from `DA.Time`:

- `days n` — duration of `n` days

- `hours n` — duration of `n` hours
- `minutes n` — duration of `n` minutes
- `seconds n` — duration of `n` seconds

`passTime` is useful for testing time-dependent logic such as expiration dates or vesting schedules.

12.6 Test Organization

12.6.1 Setup Helpers

Extract common setup into reusable functions:

```
module TestHelpers where

import Daml.Script

data TestParties = TestParties with
  bank : Party
  alice : Party
  bob : Party

allocateTestParties : Script TestParties
allocateTestParties = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  return TestParties with ..
```

```
data AssetFixture = AssetFixture with
  parties : TestParties
  aliceUsdCid : ContractId Asset
  bobEurCid : ContractId Asset

setupAssets : Script AssetFixture
setupAssets = do
  parties@TestParties{..} <- allocateTestParties

  let bankAlice = actAs bank <> actAs alice
  aliceUsdCid <- submit bankAlice do
    createCmd Asset with
      issuer = bank
      owner = alice
      symbol = "USD"
      quantity = 100.0
      observers = []

  let bankBob = actAs bank <> actAs bob
  bobEurCid <- submit bankBob do
    createCmd Asset with
      issuer = bank
      owner = bob
      symbol = "EUR"
      quantity = 85.0
      observers = []

  return AssetFixture with ..
```

Now each test starts clean with a single line:

```
module AssetTests where

import Daml.Script
import TestHelpers

test_transfer = do
  AssetFixture{..} <- setupAssets
  let TestParties{..} = parties
  -- Test logic here using alice, bob, etc.
  return ()
```

Each Test Gets Fresh State

Every test function runs against a fresh, empty ledger. There is no shared state between tests. The `setupAssets` function creates new parties and contracts each time it is called. This means tests are independent and can run in any order.

12.6.2 Naming Conventions

Adopt a consistent naming scheme:

```
-- Happy-path tests: test_<feature>
test_transfer = do ...
test_split_and_merge = do ...

-- Error cases: test_<feature>_<failure>
test_transfer_wrong_owner = do ...
test_split_zero_fails = do ...

-- Setup helpers: setup<Context>
setupAssets = do ...
setupWithTrade = do ...
```

12.6.3 Module Organization

For larger projects, organize tests into modules:

```
daml/
  Asset.daml          -- Template code
  Trade.daml          -- Template code
  Test/
    AssetTest.daml    -- Asset tests
    TradeTest.daml    -- Trade tests
    Helpers.daml      -- Shared setup
```

12.7 A Complete Test Suite

Here is a full test suite for the `Asset` template from Chapter 10:

```
module AssetTest where

import Daml.Script
import DA.Optional (isNone, isSome)
import Asset

data Parties = Parties with
  bank : Party
```

```
  alice : Party
  bob : Party

setupParties : Script Parties
setupParties = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  return Parties with ..
```

```
issueAsset : Party -> Party -> Text
  -> Decimal -> Script (ContractId Asset)
issueAsset issuer owner symbol qty = do
  let parties = actAs issuer <> actAs owner
  submit parties do
    createCmd Asset with
      issuer
      owner
      symbol
      quantity = qty
      observers = []
```

12.7.1 Creation Tests

```
test_create_asset = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0
  Some asset <- queryContractId alice cid
  assertMsg "Issuer" (asset.issuer == bank)
  assertMsg "Owner" (asset.owner == alice)
  assertMsg "Symbol" (asset.symbol == "USD")
  assertMsg "Qty" (asset.quantity == 100.0)

test_create_zero_fails = do
  Parties{..} <- setupParties
  submitMustFail bank do
    createCmd Asset with
      issuer = bank
      owner = bank
      symbol = "USD"
      quantity = 0.0
      observers = []

test_create_negative_fails = do
  Parties{..} <- setupParties
  submitMustFail bank do
    createCmd Asset with
      issuer = bank
      owner = bank
      symbol = "USD"
      quantity = -50.0
      observers = []
```

12.7.2 Split Tests

```
test_split = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0
```

```
  (cid1, cid2) <- submit alice do
    exerciseCmd cid Split with splitQuantity = 40.0
  Some a1 <- queryContractId alice cid1
  Some a2 <- queryContractId alice cid2
  assertMsg "Split piece 1" (a1.quantity == 40.0)
  assertMsg "Split piece 2" (a2.quantity == 60.0)
  -- Original is archived
  old <- queryContractId alice cid
  assertMsg "Original archived" (isNone old)

test_split_zero_fails = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0
  submitMustFail alice do
    exerciseCmd cid Split with splitQuantity = 0.0

test_split_full_amount_fails = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0
  submitMustFail alice do
    exerciseCmd cid Split
      with splitQuantity = 100.0
```

12.7.3 Merge Tests

```
test_merge = do
  Parties{..} <- setupParties
  cid1 <- issueAsset bank alice "USD" 60.0
  cid2 <- issueAsset bank alice "EUR" 40.0
  -- Cannot merge different symbols
  submitMustFail alice do
    exerciseCmd cid1 Merge with otherCid = cid2

test_merge_same_symbol = do
  Parties{..} <- setupParties
  -- Need two assets with same key? No--keys collide.
  -- Create one, split it, then merge back.
  cid <- issueAsset bank alice "USD" 100.0
  (cid1, cid2) <- submit alice do
    exerciseCmd cid Split with splitQuantity = 40.0
  merged <- submit alice do
    exerciseCmd cid1 Merge with otherCid = cid2
  Some m <- queryContractId alice merged
  assertMsg "Merged quantity" (m.quantity == 100.0)
```

12.7.4 Transfer Tests

```
test_transfer = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0
  let parties = actAs alice <> actAs bob
  newCid <- submit parties do
    exerciseCmd cid Transfer with newOwner = bob
  Some asset <- queryContractId bob newCid
  assertMsg "New owner is Bob"
    (asset.owner == bob)
  -- Alice can no longer see it
  old <- queryContractId alice cid
  assertMsg "Old contract archived" (isNone old)
```

```
test_transfer_to_self_fails = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0
  submitMustFail alice do
    exerciseCmd cid Transfer
      with newOwner = alice
```

12.7.5 Key Lookup Tests

```
test_field_lookup = do
  Parties{..} <- setupParties
  _cid <- issueAsset bank alice "USD" 100.0
  results <- queryFilter @Asset bank
    (\a -> a.owner == alice && a.symbol == "USD")
  assertMsg "Should find 1 asset"
    (length results == 1)

test_field_not_found = do
  Parties{..} <- setupParties
  results <- queryFilter @Asset bank
    (\a -> a.owner == alice && a.symbol == "GBP")
  assertMsg "GBP should not exist"
    (null results)
```

12.8 Running Tests

```
dpm build
dpm test
```

`dpm test` discovers and runs every `Script` value in the project. Output shows each test name and whether it passed or failed.

To run tests for a specific module:

```
dpm test --files daml/AssetTest.daml
```

Fast Feedback

`dpm test` runs entirely in-process—no network, no Canton, no Docker. A suite of 20–30 tests typically completes in under two seconds. Run it after every change.

12.9 Debugging Tips

12.9.1 Using debug

`debug` records a trace message during script execution:

```
test_debug_example = do
  alice <- allocateParty "Alice"
  cid <- submit alice do
    createCmd Token with owner = alice
  debug ("Created token: " <> show cid)
```

debug Output Is Not Shown in the Terminal

`dpm test` does not display `debug` or `trace` output on the command line. To see trace messages, open the file in **Daml Studio** (VS Code with the Daml extension) and click *Script results* above the test function. The results panel shows all trace messages alongside the ledger state after each step.
This is a common source of confusion: your `debug` calls are executing and recording messages, but `dpm test` only prints the pass/fail summary. If you need to inspect values from the CLI, use `assertMsg` with a descriptive message—a failing assertion will print the message to the terminal.

12.9.2 Using trace

`trace` is a pure function with signature `trace : Text -> a -> a`. It logs a message and returns its second argument unchanged. Use it to annotate expressions without changing their value:

```
    choice TracedTransfer : ContractId Asset
      with
        newOwner : Party
      controller owner, newOwner
      do
        let _ = trace ("Transferring from "
              <> show owner <> " to "
              <> show newOwner) ()
        create this with
          owner = newOwner
          observers = []
```

`debug msg` is shorthand for `trace msg ()`—both record the same trace message. Use `debug` in `Script` and choice bodies when you do not need a return value. Use `trace` when you want to annotate an expression inline: `let x = trace "computing x" (a + b)`.

12.9.3 Inspecting Contract State

When a test fails, add queries to inspect intermediate state:

```
test_debug_state = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0

  -- Check state before exercise
  allAssets <- queryFilter @Asset alice (\_ -> True)
  debug ("Assets before: " <> show allAssets)

  (cid1, cid2) <- submit alice do
    exerciseCmd cid Split with splitQuantity = 40.0

  -- Check state after exercise
  allAfter <- queryFilter @Asset alice (\_ -> True)
  debug ("Assets after: " <> show allAfter)
```

12.10 Summary

- **Daml Script** is the testing and ledger interaction language. Every top-level `Script` value is a test case.

- **`allocateParty`** creates test parties. Each test runs against a fresh, empty ledger.
- **`submit`** sends commands as a party or combination of `actAs`/`readAs` parties.
- **`submitMustFail`** asserts that a command is rejected—essential for testing authorization and validation.
- `createCmd`, `exerciseCmd`, `archiveCmd`, and `createAndExerciseCmd` are the core commands inside `submit` blocks.
- **Query functions**: `queryFilter` finds contracts by predicate, `queryContractId` looks up by ID.
- **`passTime`** advances the simulated clock for testing time-dependent logic.
- **Organize tests** with setup helpers, consistent naming, and separate modules. Keep each test independent.
- **`dpm test`** runs the full suite in-process in seconds. Use it as your primary feedback loop.

Chapter 13

Standard Library and Dependencies

Daml ships with a comprehensive standard library and supports multi-package projects. This chapter covers the most useful standard library modules and shows how to split code into reusable packages.

13.1 The Standard Library

Every Daml project depends on three base packages declared in `daml.yaml`:

```
dependencies:
  - daml-prim
  - daml-stdlib
  - daml-script
```

`daml-prim` provides primitive types (`Int`, `Decimal`, `Text`, `Party`, etc.). `daml-stdlib` provides the standard library modules. `daml-script` provides the testing and scripting framework.

The standard library is organized into `DA.*` modules. Here are the most important ones.

13.2 DA.List

List manipulation functions:

```
import DA.List

test_list : Script ()
test_list = do
  let xs = [3, 1, 4, 1, 5, 9]

  -- sort: ascending order
  debug (show (sort xs))       -- [1,1,3,4,5,9]

  -- dedup: remove consecutive duplicates
  -- (sort first for full dedup)
  debug (show (dedup (sort xs)))  -- [1,3,4,5,9]

  -- head / tail
  debug (show (head xs))       -- 3
  debug (show (tail xs))       -- [1,4,1,5,9]

  -- foldl: left fold
  let total = foldl (+) 0 xs   -- 23
  debug (show total)

  -- partition: split by predicate
```

```
  let (small, big) = partition (< 4) xs
  debug (show small)          -- [3,1,1]
  debug (show big)            -- [4,5,9]
```

Commonly used functions:

Function	Description
`sort`	Sort a list in ascending order
`sortOn f`	Sort by a key function
`dedup`	Remove consecutive duplicates
`unique`	Remove all duplicates
`head`, `tail`	First element / rest of list
`last`, `init`	Last element / all but last
`partition p`	Split into (matching, non-matching)
`foldl f z xs`	Left fold with accumulator
`zip`, `unzip`	Pair up / split apart two lists

Table 13.1: Key DA.List functions

13.3 DA.Optional

Working with values that may or may not exist:

```
import DA.Optional

test_optional : Script ()
test_optional = do
  let x : Optional Int = Some 42
  let y : Optional Int = None

  -- isSome / isNone
  debug (show (isSome x))       -- True
  debug (show (isNone y))       -- True

  -- fromSome: extract value (aborts if None)
  debug (show (fromSome x))     -- 42

  -- fromOptional: extract with default
  debug (show (fromOptional 0 x))  -- 42
  debug (show (fromOptional 0 y))  -- 0

  -- optional: fold over Optional
  let result = optional "empty"
        (\n -> "got " <> show n) x
  debug result                  -- "got 42"
```

Avoid fromSome in Production Code

`fromSome` aborts the transaction if the value is `None`. Use it only in tests where you have just verified the value exists. In choice bodies, prefer `fromOptional` with a sensible default, or pattern match with `case ... of Some v -> ...; None -> ...`.

13.4 DA.Map

Immutable key-value maps (ordered by key):

```
import DA.Map (Map)
import qualified DA.Map as Map

test_map : Script ()
test_map = do
  -- Build a map
  let m : Map Text Decimal = Map.fromList
        [("USD", 100.0), ("EUR", 85.0)]

  -- Lookup
  debug (show (Map.lookup "USD" m))  -- Some 100.0
  debug (show (Map.lookup "GBP" m))  -- None

  -- Insert / update
  let m2 = Map.insert "GBP" 120.0 m
  debug (show (Map.size m2))         -- 3

  -- Delete
  let m3 = Map.delete "EUR" m2
  debug (show (Map.size m3))         -- 2

  -- Convert to list of pairs
  let pairs = Map.toList m
  debug (show pairs)
  -- [("EUR",85.0),("USD",100.0)]

  -- Keys and values
  debug (show (Map.keys m))    -- ["EUR","USD"]
  debug (show (Map.values m))  -- [85.0,100.0]
```

Qualified Imports

`DA.Map` exports many names that conflict with `Prelude` (e.g., `null`, `size`, `filter`). The standard practice is to import the type unqualified and the module qualified: `import DA.Map (Map)` and `import qualified DA.Map as Map`.

Maps are useful for template fields that store variable-length key-value data:

```
template Portfolio
  with
    owner  : Party
    broker : Party
    holdings : Map Text Decimal
  where
    signatory owner, broker
    ensure all (> 0.0) (Map.values holdings)
```

13.5 DA.Text

Text processing functions:

```
import qualified DA.Text as T

test_text : Script ()
test_text = do
  let s = "Hello, Daml World"

  -- Length, take, drop
```

```
  debug (show (T.length s))       -- 17
  debug (T.take 5 s)              -- "Hello"
  debug (T.drop 7 s)              -- "Daml World"

  -- Split and join
  let parts = T.splitOn ", " s    -- ["Hello","Daml World"]
  debug (show parts)
  let joined = T.intercalate "-" parts
  debug joined                    -- "Hello-Daml World"

  -- Trim whitespace
  debug (T.trim "  spaced  ")     -- "spaced"

  -- Substring check
  debug (show (T.isInfixOf "Daml" s))  -- True
```

13.6 DA.Time and DA.Date

Working with time and dates:

```
import DA.Time
import DA.Date

test_time : Script ()
test_time = do
  -- Current ledger time
  now <- getTime
  debug ("Now: " <> show now)

  -- Advance time
  passTime (days 30)
  later <- getTime
  debug ("Later: " <> show later)

  -- Date construction
  let d = date 2025 Jun 15
  debug (show d)

  -- Convert date to time (midnight UTC)
  let t = datetime 2025 Jun 15 0 0 0
  debug (show t)

  -- Time arithmetic
  let tomorrow = addRelTime now (days 1)
  debug (show tomorrow)
```

Durations for `passTime` and `addRelTime`:

- `days n`, `hours n`, `minutes n`, `seconds n`
- These can be combined: `addRelTime now (days 1)`

13.7 DA.Assert

Additional assertion functions beyond `assertMsg`:

```
import DA.Assert

test_assert : Script ()
test_assert = do
```

```
  let x = 42

  -- Equality assertion (shows both values on failure)
  x === 42

  -- Inequality assertion
  x =/= 99

  -- These work in Script and in Update (choice bodies)
  return ()
```

The `===` operator is especially useful because on failure it shows both the expected and actual values, making test failures easier to diagnose than plain `assertMsg`.

13.8 Multi-Package Projects

As your Daml codebase grows, you will want to split it into multiple packages: a library package with shared templates, a test package, maybe an interface package.

13.8.1 Project Structure

```
my-project/
  multi-package.yaml
  asset-lib/
    daml.yaml
    daml/
      Asset.daml
  asset-test/
    daml.yaml
    daml/
      AssetTest.daml
```

13.8.2 The Library Package

```
mkdir -p asset-lib/daml
```

Create `asset-lib/daml.yaml`:

```
sdk-version: 3.4.11
name: asset-lib
source: daml
version: 1.0.0
dependencies:
  - daml-prim
  - daml-stdlib
```

Notice: no `daml-script` dependency. The library package contains only template definitions, no test scripts.

Create `asset-lib/daml/Asset.daml`:

```
module Asset where

template Asset
  with
    issuer   : Party
    owner    : Party
    symbol   : Text
    quantity : Decimal
```

```
    observers : [Party]
  where
    signatory issuer, owner
    observer observers
    ensure quantity > 0.0

    choice Split
      : (ContractId Asset, ContractId Asset)
      with splitQuantity : Decimal
      controller owner
      do
        assertMsg "Split qty must be positive"
          (splitQuantity > 0.0)
        assertMsg "Split qty must be less than total"
          (splitQuantity < quantity)
        cid1 <- create this with
          quantity = splitQuantity
        cid2 <- create this with
          quantity = quantity - splitQuantity
        return (cid1, cid2)

    choice Merge : ContractId Asset
      with otherCid : ContractId Asset
      controller owner
      do
        other <- fetch otherCid
        assertMsg "Issuer mismatch"
          (other.issuer == issuer)
        assertMsg "Owner mismatch"
          (other.owner == owner)
        assertMsg "Symbol mismatch"
          (other.symbol == symbol)
        archive otherCid
        create this with
          quantity = quantity + other.quantity

    choice Transfer : ContractId Asset
      with newOwner : Party
      controller owner, newOwner
      do
        assertMsg "New owner must differ"
          (newOwner /= owner)
        create this with
          owner = newOwner
          observers = []
```

Build the library:

```
cd asset-lib && dpm build
```

This produces `asset-lib/.daml/dist/asset-lib-1.0.0.dar`.

13.8.3 The Test Package

```
mkdir -p asset-test/daml
```

Create `asset-test/daml.yaml`:

```
sdk-version: 3.4.11
name: asset-test
source: daml
version: 1.0.0
```

```
dependencies:
  - daml-prim
  - daml-stdlib
  - daml-script
data-dependencies:
  - ../asset-lib/.daml/dist/asset-lib-1.0.0.dar
```

dependencies vs data-dependencies

`dependencies` are packages compiled together with your code. They must be source-compatible with your SDK version. Use this for `daml-prim`, `daml-stdlib`, and `daml-script`.

`data-dependencies` are pre-compiled DAR files. Daml imports their templates and types but does not recompile them. Use this to depend on your own library packages or third-party DARs.

Create `asset-test/daml/AssetTest.daml`:

```
module AssetTest where

import Daml.Script
import DA.Optional (isNone)
import Asset

data Parties = Parties with
  bank : Party
  alice : Party
  bob : Party

setupParties : Script Parties
setupParties = do
  bank <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  return Parties with ..

issueAsset : Party -> Party -> Text
  -> Decimal -> Script (ContractId Asset)
issueAsset issuer owner symbol qty = do
  let parties = actAs issuer <> actAs owner
  submit parties do
    createCmd Asset with
      issuer; owner; symbol
      quantity = qty
      observers = []
```

```
test_create = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0
  Some a <- queryContractId alice cid
  assertMsg "Qty" (a.quantity == 100.0)

test_split = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0
  (c1, c2) <- submit alice do
    exerciseCmd cid Split
      with splitQuantity = 40.0
  Some a1 <- queryContractId alice c1
  Some a2 <- queryContractId alice c2
```

```
  assertMsg "Piece 1" (a1.quantity == 40.0)
  assertMsg "Piece 2" (a2.quantity == 60.0)

test_transfer = do
  Parties{..} <- setupParties
  cid <- issueAsset bank alice "USD" 100.0
  let parties = actAs alice <> actAs bob
  newCid <- submit parties do
    exerciseCmd cid Transfer
      with newOwner = bob
  Some a <- queryContractId bob newCid
  assertMsg "Owner" (a.owner == bob)
  old <- queryContractId alice cid
  assertMsg "Archived" (isNone old)
```

Build and test:

```
cd asset-test && dpm build && dpm test
```

13.8.4 The multi-package.yaml File

To manage both packages together, create a `multi-package.yaml` in the project root:

```
packages:
  - asset-lib
  - asset-test
```

Now you can build everything at once:

```
dpm build --all
```

This builds packages in dependency order: `asset-lib` first, then `asset-test`.

Dependency Resolution

`dpm build -all` reads `multi-package.yaml` and resolves the build order from `data-dependencies`. If package A depends on package B's DAR, B is built first. Circular dependencies are not allowed.

13.9 Package Versioning

Each package has a `version` field in `daml.yaml`. When you change templates in a library, bump the version:

```
-- asset-lib/daml.yaml
name: asset-lib
version: 1.1.0     -- was 1.0.0
```

Then update the dependent package's `data-dependencies`:

```
-- asset-test/daml.yaml
data-dependencies:
  - ../asset-lib/.daml/dist/asset-lib-1.1.0.dar
```

DAR Path Includes Version

The DAR file path contains the version number from `daml.yaml`. When you bump the version, the DAR filename changes. All packages that reference it in `data-dependencies` must be updated. This is easy to forget—`dpm build -all` will catch it with a clear error.

13.9.1 Versioning Best Practices

- Use semantic versioning: `MAJOR.MINOR.PATCH`.
- Bump `MAJOR` when removing templates or changing template fields (breaking change).
- Bump `MINOR` when adding new templates or choices (backward-compatible).
- Bump `PATCH` for bug fixes that do not change the template structure.
- In production, deployed DARs are identified by their package ID (a hash). Changing any source code produces a new package ID, even without a version bump.

13.10 Depending on Third-Party DARs

You can depend on DARs published by other teams. Place the DAR file in a known location and reference it:

```
data-dependencies:
  - ./lib/payment-lib-2.0.0.dar
  - ./lib/identity-lib-1.3.0.dar
```

Import the templates as if they were local:

```
module MyApp where

import Payment.Types (PaymentRequest)
import Identity.KYC (KycCheck)
```

The module paths come from the DAR's source code—they are baked into the compiled package.

13.11 Summary

- The **standard library** (`daml-stdlib`) provides modules for lists (`DA.List`), optionals (`DA.Optional`), maps (`DA.Map`), text (`DA.Text`), time (`DA.Time`), and assertions (`DA.Assert`).
- Use **qualified imports** for modules with conflicting names: `import qualified DA.Map as Map`.
- **`dependencies`** are source-level packages (`daml-prim`, `daml-stdlib`, `daml-script`).
- **`data-dependencies`** are pre-compiled DAR files. Use them to depend on your own library packages or third-party code.
- **`multi-package.yaml`** lists all packages in a project. `dpm build -all` builds them in dependency order.
- **Version your packages** with semantic versioning. The DAR filename includes the version, so dependents must update their paths when versions change.

- Split code into a **library package** (templates only, no `daml-script`) and a **test package** (depends on the library via `data-dependencies`).

Part III

Building Real Applications

Chapter 14

Project Structure

Chapter 13 introduced multi-package projects with a library and a test package. Real applications need more: a backend service, a frontend, CI pipelines, and a plan for upgrades. This chapter covers the full project layout used by production Daml applications.

14.1 The Full Project Layout

A production Daml project typically has four top-level directories:

```
my-app/
  multi-package.yaml
  daml/                    # Templates (library)
    daml.yaml
    daml/
      Main/
        License.daml
        Types.daml
  daml-test/               # Scripts and tests
    daml.yaml
    daml/
      Test/
        LicenseTest.daml
        Helpers.daml
  backend/                 # Spring Boot / JVM service
    build.gradle
    src/
  frontend/                # React / TypeScript UI
    package.json
    src/
```

The key insight is **separation of concerns**: the `daml/` package contains only templates and data types. It has no dependency on `daml-script`. The `daml-test/` package imports the templates via `data-dependencies` and adds test scripts.

Why Separate Templates from Tests?

If your template package depends on `daml-script`, the Daml compiler emits a warning: "`daml-script is only needed for testing.`" More importantly, a template-only package produces a smaller DAR file with no test code—this is what you deploy to production and distribute to counterparties. Test code stays in a separate package that never gets uploaded to the ledger.

14.2 Configuring multi-package.yaml

The `multi-package.yaml` file at the project root lists all Daml packages:

```
packages:
  - daml
  - daml-test
```

The template package's `daml.yaml`:

```
sdk-version: 3.4.11
name: my-app
source: daml
version: 1.0.0
dependencies:
  - daml-prim
  - daml-stdlib
```

The test package's `daml.yaml`:

```
sdk-version: 3.4.11
name: my-app-test
source: daml
version: 1.0.0
dependencies:
  - daml-prim
  - daml-stdlib
  - daml-script
data-dependencies:
  - ../daml/.daml/dist/my-app-1.0.0.dar
```

Build everything with a single command:

```
dpm build --all
```

This resolves the dependency graph and builds `daml/` first, then `daml-test/`.

14.3 The DAR Dependency Graph

Understanding the difference between `dependencies` and `data-dependencies` is critical for structuring larger projects.

	dependencies	**data-dependencies**
What	SDK packages compiled with your code	Pre-compiled DAR files
When	Always: `daml-prim`, `daml-stdlib`	Your own library DARs, third-party DARs
How	Source-level compilation	Binary import of templates and types
Limit	Must match SDK version	Any compatible Daml-LF version

Table 14.1: dependencies vs data-dependencies

For a three-package project (library, interface, app), the dependency graph looks like this:

```
my-app-interfaces/    # Interface definitions
  daml.yaml
  dependencies: [daml-prim, daml-stdlib]

my-app/               # Templates implementing interfaces
```

```
  daml.yaml
  dependencies: [daml-prim, daml-stdlib]
  data-dependencies:
    - ../my-app-interfaces/.daml/dist/...dar

my-app-test/          # Test scripts
  daml.yaml
  dependencies: [daml-prim, daml-stdlib,
                 daml-script]
  data-dependencies:
    - ../my-app/.daml/dist/...dar
    - ../my-app-interfaces/.daml/dist/...dar
```

Circular Dependencies

Daml does not allow circular dependencies between packages. If package A depends on package B, then B cannot depend on A. If you find yourself wanting this, extract the shared types into a third package that both A and B depend on.

14.4 Package Versioning Strategy

Each package declares its version in `daml.yaml`. Follow semantic versioning:

- **MAJOR** (2.0.0): Breaking changes—removed templates, changed fields.
- **MINOR** (1.1.0): New templates or choices added (backward-compatible).
- **PATCH** (1.0.1): Bug fixes, documentation, no template changes.

When you bump a library version, update all `data-dependencies` paths that reference it:

```
-- daml/daml.yaml: version changed to 1.1.0
-- daml-test/daml.yaml must update:
data-dependencies:
  - ../daml/.daml/dist/my-app-1.1.0.dar
```

Package IDs Are Content Hashes

On the ledger, packages are identified by a cryptographic hash of their compiled content, not by their version string. Changing any source code—even a comment in some cases—produces a new package ID. The version string is metadata for human consumption.

14.5 CI/CD Pipeline Basics

A minimal CI pipeline for a Daml project runs three steps:

```
# Step 1: Build all packages
dpm build --all

# Step 2: Run all tests
dpm test --all

# Step 3: Package artifacts
cp daml/.daml/dist/my-app-*.dar artifacts/
```

For a more complete pipeline, add linting and upgrade checks:

```
#!/bin/bash
set -euo pipefail

echo "=== Building ==="
dpm build --all

echo "=== Testing ==="
dpm test --all

echo "=== Upgrade Check ==="
# Compare new DAR against previously deployed version
dpm upgrade-check old-version.dar \
  daml/.daml/dist/my-app-*.dar

echo "=== Codegen ==="
dpm codegen-java
dpm codegen-js

echo "=== Backend Build ==="
cd backend && ./gradlew build && cd ..

echo "=== Frontend Build ==="
cd frontend && npm ci && npm run build && cd ..

echo "=== Done ==="
```

The `dpm upgrade-check` step verifies that your new DAR is upgrade-compatible with the previously deployed version. We cover this in detail in the upgrade safety section below.

14.6 Daml Interfaces

Daml interfaces enable cross-template polymorphism—writing code that works with any template implementing a given interface, without knowing the concrete template type.

14.6.1 Defining an Interface

An interface declares a view type and a set of methods:

```
module Main.Interfaces where

data AssetView = AssetView
  with
    assetOwner : Party
    assetAmount : Decimal
  deriving (Eq, Show)

interface Asset where
  viewtype AssetView

  assetOwner : Party
  assetAmount : Decimal

  choice TransferAsset : ContractId Asset
    with newOwner : Party
    controller (assetOwner this)
    do
      create this with
        assetOwner = newOwner
```

Simplified Example

This interface example is simplified for illustration. Real interface choices involve additional complexity (e.g., you cannot directly `create this` on an interface). Interface methods are accessed via `methodName this` where `this` refers to the current contract. Consult the Daml documentation for the complete interface machinery.

14.6.2 Implementing an Interface

Templates implement interfaces using the `interface instance` syntax:

```
module Main.Token where

import Main.Interfaces

template Token
  with
    owner : Party
    amount : Decimal
    issuer : Party
  where
    signatory owner, issuer

    interface instance Asset for Token where
      view = AssetView with
        assetOwner = owner
        assetAmount = amount
      assetOwner = owner
      assetAmount = amount
```

14.6.3 Why Interfaces Matter

Interfaces are especially useful in multi-party ecosystems:

- A market operator defines an `Asset` interface.
- Each issuer implements it with their own template.
- Trading logic works with `Asset` regardless of the concrete template.
- New asset types can be added without changing the trading code.

We will not go deeper into interfaces here—they deserve their own chapter in a more advanced treatment. The key takeaway is that interfaces live in their own package at the bottom of the dependency graph, so all template packages can import them.

14.7 Contract Upgrades

When you deploy a new version of a template package, existing contracts on the ledger still use the old package ID. Daml provides upgrade mechanisms to handle this transition safely.

14.7.1 Upgrade Safety Rules

The Daml compiler enforces rules about what changes are upgrade-safe:

Change	Safe?	Notes
Add optional field	Yes	Must have a default value
Add new choice	Yes	Existing contracts gain it
Add new template	Yes	No impact on old contracts
Remove a field	No	Breaks deserialization
Rename a field	No	Fields are positional
Change field type	No	Type mismatch
Remove a template	No	Orphans existing contracts

Table 14.2: Upgrade safety rules

14.7.2 Checking Upgrade Compatibility

Before deploying a new version, run the upgrade check against the old and new DAR files:

```
dpm upgrade-check \
  daml/.daml/dist/my-app-1.0.0.dar \
  daml/.daml/dist/my-app-1.1.0.dar
```

This compares the two packages and reports any breaking changes. If it finds issues, it exits with a non-zero code—making it ideal for CI gates. You can use `-compiler` for compile-time checks, `-participant` for runtime checks, or `-both`.

Upgrade Discipline

Once a DAR is deployed to production and contracts exist, you cannot change the template structure in breaking ways. Plan your data model carefully. Use optional fields (`Optional Text`) for data you might add later.

14.7.3 Adding an Optional Field

The safest upgrade pattern is adding a new optional field with a default:

```
-- Version 1.0.0
template License
  with
    vendor : Party
    customer : Party
    product : Text

-- Version 1.1.0 (upgrade-safe)
template License
  with
    vendor : Party
    customer : Party
    product : Text
    notes : Optional Text  -- New field
```

When the new package processes an old contract, the `notes` field is populated with `None`.

14.8 Summary

- Separate **templates** (`daml/`) from **tests** (`daml-test/`) to avoid the `daml-script` dependency warning and keep production DARs clean.

- Use **`multi-package.yaml`** to manage multiple Daml packages and build them in dependency order with `dpm build -all`.
- **`dependencies`** are SDK packages compiled with your code; **`data-dependencies`** are pre-compiled DARs.
- Follow **semantic versioning** for packages. Remember that DAR filenames include the version number.
- **Daml interfaces** enable cross-template polymorphism. Define them in a separate package at the bottom of the dependency graph.
- Use **`dpm upgrade-check`** with old and new DAR files to verify that template changes are backward-compatible before deploying.
- Only **additive changes** (new optional fields, new choices, new templates) are upgrade-safe. Removing or renaming fields breaks existing contracts.
- A CI pipeline should **build**, **test**, and **upgrade-check** before producing deployment artifacts.

Chapter 15

Ledger API and Backend Services

Until now we have interacted with the ledger through Daml Script (testing) and the Canton Console (deployment). Production applications use the Ledger API—a programmatic interface that backends and frontends use to submit commands and read contract state. This chapter covers both API flavors and shows how to build a backend service.

15.1 Two Ledger APIs

Canton exposes two APIs for application integration:

	gRPC Ledger API	**JSON Ledger API**
Protocol	gRPC / Protocol Buffers	HTTP/1.1 + JSON
Port (cn-quickstart)	2901, 3901, 4901	2975, 3975, 4975
Best for	JVM backends, high throughput	Node.js backends, frontends, scripting
Streaming	Native gRPC streams	Server-Sent Events
Code generation	Java, Scala	TypeScript / JavaScript
Auth	mTLS or OAuth2 tokens	OAuth2 Bearer tokens

Table 15.1: gRPC vs JSON Ledger API

Which API Should I Use?

Use the **JSON Ledger API** for frontends, Node.js backends, quick prototyping, and any language without a gRPC client. Use the **gRPC Ledger API** for JVM-based services where you need maximum throughput. The JSON API is a stateless HTTP wrapper around the same underlying services—and is the natural choice for JavaScript/TypeScript backends.

15.2 JSON Ledger API Endpoints

The JSON Ledger API is organized into resource-oriented endpoints. All requests require an `Authorization: Bearer <token>` header in authenticated environments.

15.2.1 Packages: /v2/packages

Upload and list DAR packages:

```
# Upload a DAR
curl -X POST http://localhost:3975/v2/packages \
  -H "Content-Type: application/octet-stream" \
  -H "Authorization: Bearer $TOKEN" \
  --data-binary @my-app-1.0.0.dar

# List package IDs
curl -s http://localhost:3975/v2/packages \
  -H "Authorization: Bearer $TOKEN" \
  | jq ".packageIds[:5]"
```

15.2.2 Parties: /v2/parties

Manage and query parties on the participant:

```
# List known parties
curl -s http://localhost:3975/v2/parties \
  -H "Authorization: Bearer $TOKEN" \
  | jq ".partyDetails"

# Allocate a new party
curl -X POST http://localhost:3975/v2/parties \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"partyIdHint": "Alice"}'
```

15.2.3 Commands: /v2/commands/submit-and-wait

Submit commands to the ledger and wait for the result. This is the primary write endpoint.

Creating a Contract

```
curl -X POST \
  http://localhost:3975/v2/commands/submit-and-wait \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "commands": [{
      "CreateCommand": {
        "templateId": "Main.License:License",
        "createArguments": {
          "vendor": "'"$VENDOR_PARTY"'",
          "customer": "'"$CUSTOMER_PARTY"'",
          "product": "Enterprise Suite",
          "expiresAt": "2027-01-01T00:00:00Z"
        }
      }
    }],
    "actAs": ["'"$VENDOR_PARTY"'"],
    "commandId": "create-license-001",
    "applicationId": "my-app"
  }'
```

Command IDs

Every command submission should include a unique `commandId`. The ledger uses this for deduplication—if the same `commandId` is submitted twice, the second submission is rejected. Use UUIDs or structured IDs like `create-license-001`.

Exercising a Choice

```
curl -X POST \
  http://localhost:3975/v2/commands/submit-and-wait \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "commands": [{
      "ExerciseCommand": {
        "templateId": "Main.License:License",
        "contractId": "'"$CONTRACT_ID"'",
        "choice": "Renew",
        "choiceArgument": {
          "newExpiry":
            "2028-01-01T00:00:00Z"
        }
      }
    }],
    "actAs": ["'"$VENDOR_PARTY"'"],
    "commandId": "renew-license-001",
    "applicationId": "my-app"
  }'
```

Contract Keys Removed in SDK 3.x

Contract keys (and `ExerciseByKeyCommand`) were removed in SDK 3.x. Use `ExerciseCommand` with a contract ID instead. Discover contract IDs via the active contracts endpoint (`/v2/state/active-contracts`) or PQS.

15.2.4 Active Contracts: /v2/state/active-contracts

Get a snapshot of all currently active contracts visible to a party:

```
curl -X POST \
  http://localhost:3975/v2/state/active-contracts \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "filter": {
      "filtersByParty": {
        "'"$VENDOR_PARTY"'": {
          "cumulative": [{
            "templateFilters": [{
              "templateId":
                "Main.License:License"
            }]
          }]
        }
      }
    }
  }'
```

The response contains a list of active contract entries, each with the contract ID, template ID, and payload.

15.2.5 Transaction Streams: /v2/updates

For real-time updates, use the streaming endpoints. The JSON API provides flat transaction streams and transaction tree streams:

```
curl -N -X POST \
  http://localhost:3975/v2/updates/flats \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "filter": {
      "filtersByParty": {
        "'"$VENDOR_PARTY"'": {
          "cumulative": [{
            "templateFilters": [{
              "templateId":
                "Main.License:License"
            }]
          }]
        }
      }
    },
    "beginExclusive": 0
  }'
```

Use `/v2/updates/trees` instead of `/v2/updates/flats` when you need full transaction tree detail (e.g., seeing sub-transactions from choices that call other choices).

Streaming vs Polling

Streaming (`/v2/updates/flats` or gRPC streams) delivers events as they occur with sub-second latency. Use it for real-time UIs and event-driven backends.
Polling means periodically calling the active contracts endpoint (`/v2/state/active-contracts`). It is simpler but introduces latency equal to your polling interval. Use it for batch jobs or systems that cannot maintain long-lived connections.
For most applications, streaming is preferred.

15.3 Code Generation

Writing JSON payloads by hand is tedious and error-prone. Daml provides code generators that produce type-safe bindings from your templates.

15.3.1 JavaScript/TypeScript Code Generation

Generate TypeScript types for use in backends and frontends:

```
dpm codegen-js
```

This reads your compiled DAR and produces TypeScript files in the configured output directory (typically `src/generated/`). Each template becomes a TypeScript interface with:

- An interface matching the template fields.

- Types for each choice argument.
- Helper constants for template IDs.

We use these generated types in both the backend (this chapter) and the frontend (Chapter 17).

15.3.2 Java Code Generation

Generate Java classes for JVM backends:

```
dpm codegen-java
```

This produces Java source files in the configured output directory. Each Daml template becomes a Java class with typed fields, a builder, and inner classes for each choice argument. The generated code targets Java 17+.

15.4 Building a Backend Service

Two Language Options

This chapter shows backend implementations in both JavaScript (Express.js) and Java (Spring Boot). Choose the one that matches your stack. Both connect to the same Canton JSON Ledger API and expose the same REST endpoints.

Express.js (JavaScript)

We build the backend as a Node.js service using Express.js and the JSON Ledger API. This approach uses a single language (JavaScript/TypeScript) across the full stack and communicates with Canton over HTTP—no gRPC tooling required.

15.4.1 Project Setup

A typical backend structure:

```
backend/
  package.json
  .env
  src/
    index.js
    ledgerClient.js
    licenseService.js
    licenseRoutes.js
```

Initialize the project and install dependencies:

```
cd backend
npm init -y
npm install express dotenv
```

The `package.json` should enable ES modules:

```
{
  "name": "license-backend",
  "type": "module",
  "scripts": {
    "start": "node src/index.js",
```

```
    "dev": "node --watch src/index.js"
  },
  "dependencies": {
    "dotenv": "^16.4.0",
    "express": "^4.21.0"
  }
}
```

15.4.2 Configuration

Store configuration in a `.env` file:

```
# .env
LEDGER_API_URL=http://localhost:3975
KEYCLOAK_URL=http://keycloak.localhost:8082
KEYCLOAK_REALM=AppProvider
CLIENT_ID=app-provider-validator
CLIENT_SECRET=your-client-secret
PORT=8080
```

Load it at startup:

```
// src/index.js
import "dotenv/config";
import express from "express";
import { licenseRoutes } from
  "./licenseRoutes.js";

const app = express();
app.use(express.json());
app.use("/api/licenses", licenseRoutes);

const PORT = process.env.PORT || 8080;
app.listen(PORT, () => {
  console.log(`Backend on port ${PORT}`);
});
```

15.4.3 Ledger Client

A helper module wraps JSON Ledger API calls:

```
// src/ledgerClient.js
const BASE = process.env.LEDGER_API_URL;

export async function submitCommand(
  token, actAs, commands, commandId
) {
  const resp = await fetch(
    `${BASE}/v2/commands/submit-and-wait`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${token}`,
      },
      body: JSON.stringify({
        commands,
        actAs,
        commandId,
        applicationId: "license-app",
      }),
```

```
    }
  );
  if (!resp.ok) {
    const err = await resp.text();
    throw new Error(
      `Ledger error ${resp.status}: ${err}`
    );
  }
  return resp.json();
}
```

```
// src/ledgerClient.js (continued)
export async function getActiveContracts(
  token, party, templateId
) {
  const resp = await fetch(
    `${BASE}/v2/state/active-contracts`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${token}`,
      },
      body: JSON.stringify({
        filter: {
          filtersByParty: {
            [party]: {
              cumulative: [{
                templateFilters: [{
                  templateId,
                }],
              }],
            },
          },
        },
      }),
    }
  );
  if (!resp.ok) {
    const err = await resp.text();
    throw new Error(
      `ACS error ${resp.status}: ${err}`
    );
  }
  return resp.json();
}
```

Why JSON API for the Backend?

A Node.js backend uses the JSON Ledger API over HTTP, which is the natural fit for JavaScript. The JSON API provides the same functionality as the gRPC API—command submission, active contract queries, and event streaming—without requiring gRPC tooling. For most applications, the throughput is more than sufficient.

15.4.4 Command Submission

A service module wraps ledger operations:

```
// src/licenseService.js
import { randomUUID } from "node:crypto";
```

```
import {
  submitCommand,
  getActiveContracts,
} from "./ledgerClient.js";

const TEMPLATE =
  "Main.License:License";

export async function createLicense(
  token, vendor, customer,
  product, expiresAt
) {
  const result = await submitCommand(
    token,
    [vendor],
    [{
      CreateCommand: {
        templateId: TEMPLATE,
        createArguments: {
          vendor,
          customer,
          product,
          expiresAt,
        },
      },
    }],
    `create-license-${randomUUID()}`
  );
  return result;
}
```

15.4.5 Reading Contract State

The backend reads active contracts via the ACS endpoint:

```
// src/licenseService.js (continued)
export async function listLicenses(
  token, party
) {
  const data = await getActiveContracts(
    token, party, TEMPLATE
  );
  return data;
}
```

For real-time updates, subscribe to the flat transaction stream using Server-Sent Events:

```
// src/licenseService.js (continued)
export async function subscribeLicenses(
  token, party, onEvent
) {
  const url =
    `${process.env.LEDGER_API_URL}`
    + "/v2/updates/flats";
  const resp = await fetch(url, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({
      filter: {
```

```
        filtersByParty: {
          [party]: {
            cumulative: [{
              templateFilters: [{
                templateId: TEMPLATE,
              }],
            }],
          },
        },
      },
      beginExclusive: 0,
    }),
  });
  const reader = resp.body.getReader();
  const decoder = new TextDecoder();
  while (true) {
    const { done, value } =
      await reader.read();
    if (done) break;
    const text = decoder.decode(value);
    for (const line of text.split("\n")) {
      if (line.startsWith("data:")) {
        onEvent(JSON.parse(line.slice(5)));
      }
    }
  }
}
```

15.4.6 REST API Layer

Express route handlers expose the REST endpoints:

```
// src/licenseRoutes.js
import { Router } from "express";
import {
  createLicense,
  listLicenses,
} from "./licenseService.js";

export const licenseRoutes = Router();

licenseRoutes.post("/", async (req, res) => {
  try {
    const {
      vendor, customer, product, expiresAt
    } = req.body;
    const token =
      req.headers.authorization?.split(" ")[1];
    const result = await createLicense(
      token, vendor, customer,
      product, expiresAt
    );
    res.json(result);
  } catch (err) {
    res.status(500).json({
      error: err.message,
    });
  }
});

licenseRoutes.get("/", async (req, res) => {
  try {
```

```
    const { party } = req.query;
    const token =
      req.headers.authorization?.split(" ")[1];
    const data =
      await listLicenses(token, party);
    res.json(data);
  } catch (err) {
    res.status(500).json({
      error: err.message,
    });
  }
});
```

Spring Boot (Java)

For JVM-based teams, Spring Boot provides an equivalent backend using the same JSON Ledger API. Modern Java (17+) features—records, `var`, `HttpClient`—keep the code concise.

Java Code Generation

Generate Java bindings from your Daml templates:

```
dpm codegen-java
```

This produces Java classes in `src/generated/` mirroring each template. For example, the `License` template produces a `License.java` class with typed fields and a builder.

Project Setup

A typical Spring Boot project structure:

```
backend-java/
  build.gradle.kts
  src/main/
    java/com/example/license/
      LicenseApplication.java
      LedgerClient.java
      LicenseService.java
      LicenseController.java
    resources/
      application.yml
```

The Gradle build file with dependencies:

```
plugins {
    java
    id("org.springframework.boot")
      version "3.3.0"
    id("io.spring.dependency-management")
      version "1.1.6"
}

java {
    sourceCompatibility =
      JavaVersion.VERSION_17
}

repositories { mavenCentral() }

dependencies {
```

```
    implementation(
      "org.springframework.boot:" +
      "spring-boot-starter-web"
    )
    implementation(
      "com.fasterxml.jackson.core:" +
      "jackson-databind"
    )
}
```

Configuration

The `application.yml` stores connection settings:

```
# application.yml
ledger:
  api-url: http://localhost:3975
keycloak:
  url: http://keycloak.localhost:8082
  realm: AppProvider
  client-id: app-provider-validator
  client-secret: ${CLIENT_SECRET}
server:
  port: 8080
```

A configuration record to bind these properties:

```
// LedgerConfig.java
package com.example.license;

import org.springframework.boot.context
  .properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "ledger")
public record LedgerConfig(
    String apiUrl
) {}
```

Enable the config in the application class:

```
// LicenseApplication.java
package com.example.license;

import org.springframework.boot
  .SpringApplication;
import org.springframework.boot.autoconfigure
  .SpringBootApplication;
import org.springframework.boot.context
  .properties
  .EnableConfigurationProperties;

@SpringBootApplication
@EnableConfigurationProperties(
  LedgerConfig.class
)
public class LicenseApplication {
    public static void main(String[] args) {
        SpringApplication.run(
          LicenseApplication.class, args
        );
    }
}
```

Ledger Client

A helper class wraps HTTP calls to the JSON Ledger API using Java's built-in `HttpClient`:

```
// LedgerClient.java
package com.example.license;

import com.fasterxml.jackson.databind.*;
import java.net.URI;
import java.net.http.*;
import java.net.http.HttpResponse
  .BodyHandlers;
import org.springframework.stereotype
  .Component;

@Component
public class LedgerClient {
    private final HttpClient http =
      HttpClient.newHttpClient();
    private final ObjectMapper mapper =
      new ObjectMapper();
    private final String baseUrl;

    public LedgerClient(LedgerConfig cfg) {
        this.baseUrl = cfg.apiUrl();
    }
```

```
    public JsonNode submitCommand(
        String token,
        String[] actAs,
        Object[] commands,
        String commandId
    ) throws Exception {
        var body = mapper.createObjectNode();
        body.set("commands",
          mapper.valueToTree(commands));
        body.set("actAs",
          mapper.valueToTree(actAs));
        body.put("commandId", commandId);
        body.put("applicationId",
          "license-app");

        var req = HttpRequest.newBuilder()
          .uri(URI.create(baseUrl
            + "/v2/commands/submit-and-wait"))
          .header("Content-Type",
            "application/json")
          .header("Authorization",
            "Bearer " + token)
          .POST(HttpRequest.BodyPublishers
            .ofString(
              mapper.writeValueAsString(body)
            ))
          .build();

        var resp = http.send(
          req, BodyHandlers.ofString()
        );
        if (resp.statusCode() >= 400) {
            throw new RuntimeException(
              "Ledger error "
              + resp.statusCode()
              + ": " + resp.body()
```

```
            );
        }
        return mapper.readTree(resp.body());
    }
```

```
    public JsonNode getActiveContracts(
        String token,
        String party,
        String templateId
    ) throws Exception {
        var filter = mapper
          .createObjectNode();
        var partyFilter = mapper
          .createObjectNode();
        var cumEntry = mapper
          .createObjectNode();
        var tplFilter = mapper
          .createObjectNode();
        tplFilter.put(
          "templateId", templateId
        );
        cumEntry.set("templateFilters",
          mapper.createArrayNode()
            .add(tplFilter));
        partyFilter.set("cumulative",
          mapper.createArrayNode()
            .add(cumEntry));
        var byParty = mapper
          .createObjectNode();
        byParty.set(party, partyFilter);
        filter.set(
          "filtersByParty", byParty
        );
        var body = mapper
          .createObjectNode();
        body.set("filter", filter);

        var req = HttpRequest.newBuilder()
          .uri(URI.create(baseUrl
            + "/v2/state/active-contracts"))
          .header("Content-Type",
            "application/json")
          .header("Authorization",
            "Bearer " + token)
          .POST(HttpRequest.BodyPublishers
            .ofString(
              mapper.writeValueAsString(body)
            ))
          .build();

        var resp = http.send(
          req, BodyHandlers.ofString()
        );
        if (resp.statusCode() >= 400) {
            throw new RuntimeException(
              "ACS error "
              + resp.statusCode()
              + ": " + resp.body()
            );
        }
        return mapper.readTree(resp.body());
    }
```

```
}
```

License Service

The service layer maps business operations to ledger commands:

```
// LicenseService.java
package com.example.license;

import com.fasterxml.jackson.databind.*;
import com.fasterxml.jackson.databind.node.*;
import java.util.UUID;
import org.springframework.stereotype
  .Service;

@Service
public class LicenseService {
    private static final String TEMPLATE =
      "Main.License:License";
    private final LedgerClient ledger;
    private final ObjectMapper mapper =
      new ObjectMapper();

    public LicenseService(
        LedgerClient ledger
    ) {
        this.ledger = ledger;
    }
```

```
    public JsonNode createLicense(
        String token,
        String vendor,
        String customer,
        String product,
        String expiresAt
    ) throws Exception {
        var args = mapper
          .createObjectNode();
        args.put("vendor", vendor);
        args.put("customer", customer);
        args.put("product", product);
        args.put("expiresAt", expiresAt);

        var cmd = mapper
          .createObjectNode();
        var create = mapper
          .createObjectNode();
        create.put("templateId", TEMPLATE);
        create.set(
          "createArguments", args
        );
        cmd.set("CreateCommand", create);

        return ledger.submitCommand(
          token,
          new String[]{ vendor },
          new ObjectNode[]{ cmd },
          "create-license-"
            + UUID.randomUUID()
        );
    }
```

```
    public JsonNode listLicenses(
        String token, String party
    ) throws Exception {
        return ledger.getActiveContracts(
          token, party, TEMPLATE
        );
    }
}
```

REST Controller

The controller exposes the same REST endpoints as the Express.js version:

```
// LicenseController.java
package com.example.license;

import com.fasterxml.jackson.databind.*;
import org.springframework.http.*;
import org.springframework.web.bind
  .annotation.*;

@RestController
@RequestMapping("/api/licenses")
public class LicenseController {
    private final LicenseService svc;

    public LicenseController(
        LicenseService svc
    ) {
        this.svc = svc;
    }
```

```
    @PostMapping
    public ResponseEntity<?> create(
        @RequestBody JsonNode body,
        @RequestHeader("Authorization")
          String auth
    ) {
        try {
            var token =
              auth.replace("Bearer ", "");
            var result = svc.createLicense(
              token,
              body.get("vendor").asText(),
              body.get("customer").asText(),
              body.get("product").asText(),
              body.get("expiresAt").asText()
            );
            return ResponseEntity.ok(result);
        } catch (Exception e) {
            return ResponseEntity
              .status(500)
              .body(java.util.Map.of(
                "error", e.getMessage()
              ));
        }
    }
```

```
    @GetMapping
    public ResponseEntity<?> list(
```

```
        @RequestParam String party,
        @RequestHeader("Authorization")
          String auth
    ) {
        try {
            var token =
              auth.replace("Bearer ", "");
            var data =
              svc.listLicenses(token, party);
            return ResponseEntity.ok(data);
        } catch (Exception e) {
            return ResponseEntity
              .status(500)
              .body(java.util.Map.of(
                "error", e.getMessage()
              ));
        }
    }
}
```

Same Endpoints, Different Language

Both the Express.js and Spring Boot backends expose identical REST endpoints (`POST /api/licenses`, `GET /api/licenses`). The frontend does not need to know which backend is running—it uses the same HTTP requests regardless.

15.5 OAuth2 Authentication with Keycloak

In cn-quickstart, all API access is authenticated through Keycloak. The flow works as follows:

1. The backend obtains a service account token from Keycloak using client credentials.
2. It includes this token in the `Authorization` header of every Ledger API call.
3. Tokens expire periodically; the backend must refresh them.

15.5.1 Obtaining a Token

```
KC=http://keycloak.localhost:8082
REALM=AppProvider
TOKEN_URL="$KC/realms/$REALM"
TOKEN_URL="$TOKEN_URL/protocol/openid-connect"
TOKEN_URL="$TOKEN_URL/token"

TOKEN=$(curl -fsS "$TOKEN_URL" \
  -d "client_id=app-provider-validator" \
  -d "client_secret=$CLIENT_SECRET" \
  -d "grant_type=client_credentials" \
  -d "scope=openid" \
  | jq -r ".access_token")
```

15.5.2 Token Contents

The JWT token contains claims that Canton uses for authorization:

```
{
  "sub": "app-provider-validator",
```

```
  "aud": "https://canton.network",
  "act_as": [
    "app-provider::1220abcd..."
  ],
  "read_as": [
    "app-provider::1220abcd..."
  ],
  "exp": 1735700000
}
```

The `act_as` claim determines which parties the token holder can submit commands as. The `read_as` claim determines which parties' contracts they can read.

Token Security

Never expose Keycloak client secrets in frontend code. The frontend uses a different OAuth2 flow (authorization code with PKCE) that does not require a client secret. Only backend services use client credentials.

15.6 Error Handling

Ledger API calls can fail for several reasons. A robust backend handles them all:

Error	Cause and Response
`INVALID_ARGUMENT`	Malformed command. Fix the request.
`NOT_FOUND`	Contract archived or unknown. Refresh state.
`ALREADY_EXISTS`	Duplicate command ID. Command was already processed.
`PERMISSION_DENIED`	Token lacks required party authority. Check claims.
`ABORTED`	Contention (another transaction modified the contract). Retry.

Table 15.2: Common Ledger API errors

The `ABORTED` error deserves special attention. It occurs when two commands try to exercise on the same contract simultaneously. The standard pattern is to retry with exponential backoff:

```
// src/retry.js
export async function submitWithRetry(
  submitFn, maxRetries = 3
) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      return await submitFn();
    } catch (err) {
      const isAborted =
        err.message?.includes("ABORTED");
      const hasRetries =
        i < maxRetries - 1;
      if (isAborted && hasRetries) {
        const ms = 100 * (i + 1);
        await new Promise(
          (r) => setTimeout(r, ms)
        );
        continue;
      }
      throw err;
    }
  }
```

```
  throw new Error("Retries exhausted");
}
```

15.7 Summary

- Canton exposes two Ledger APIs: **gRPC** (for JVM backends, high throughput) and **JSON/HTTP** (for Node.js backends, frontends, scripting).
- The JSON API has key endpoint groups: **/v2/commands/submit-and-wait** (write), **/v2/state/active-contracts** (read ACS), **/v2/updates/flats** (stream events), **/v2/packages** (DAR management), and **/v2/parties** (party management).
- Use **`dpm codegen-js`** for TypeScript bindings or **`dpm codegen-java`** for Java bindings from your templates.
- A typical backend is either an **Express.js** (Node.js) or **Spring Boot** (Java) service that communicates with the JSON Ledger API over HTTP, submits commands, and reads contract state. Both expose the same REST endpoints.
- Use **streaming** (Server-Sent Events) for real-time contract updates; use **polling** only for batch or simple scenarios.
- **OAuth2/Keycloak** provides authentication. Backend services use client credentials; frontends use authorization code flow.
- Handle **contention errors** (`ABORTED`) with retry logic—concurrent exercises on the same contract are expected in multi-party systems.

Chapter 16

PQS and Efficient Queries

The Ledger API is designed for submitting commands and streaming events—it is not a query engine. If your application needs to filter contracts by arbitrary fields, join data across templates, or run aggregate queries, you need a different tool. That tool is the Participant Query Store (PQS).

16.1 What PQS Is and Why It Exists

The Participant Query Store is a component that subscribes to a participant's transaction stream and projects ledger events into a PostgreSQL database. Once the data is in PostgreSQL, you can query it with standard SQL.

The Query Problem

The Ledger API lets you fetch active contracts by template type and party, but it cannot filter by arbitrary fields. Want all licenses expiring this month? All contracts where the amount exceeds 10,000? The Ledger API has no mechanism for this. PQS solves this by making contract data queryable via SQL.

Without PQS, the alternative is to fetch all active contracts and filter in application code. This works for small datasets but becomes impractical at scale—imagine downloading 100,000 contracts to find the 50 that match your criteria.

16.2 PQS Architecture

PQS sits between the participant and your application:

```
Canton Participant
    |
    | (gRPC transaction stream)
    v
PQS Service
    |
    | (writes)
    v
PostgreSQL Database
    ^
    | (SQL queries)
    |
Your Application / Daml Shell
```

The flow:

1. PQS connects to the participant's gRPC Ledger API.
2. It subscribes to the transaction stream starting from the beginning (or a configured offset).
3. For each transaction, it extracts created and archived events.
4. Created events are inserted into PostgreSQL tables; archived events mark rows as inactive.
5. Your application queries PostgreSQL directly—no Ledger API involved.

Eventually Consistent

PQS is **eventually consistent** with the ledger. There is a small delay (typically milliseconds) between a transaction being committed and the data appearing in PostgreSQL. For most applications this is negligible, but if you need guaranteed consistency, use the Ledger API directly.

16.3 Setting Up PQS

In cn-quickstart, PQS is already running as the `pqs-app-provider` Docker container. It is pre-configured to connect to the App Provider participant and write to a PostgreSQL database.

16.3.1 Verifying PQS Is Running

```
docker ps | grep pqs
```

You should see the `pqs-app-provider` container running. It connects to the `postgres` container on port 5432.

16.3.2 PQS Configuration

PQS (internally called "Scribe") is configured via environment variables. The key ones for the database target:

Variable	Purpose
`SCRIBE_TARGET_POSTGRES_HOST`	PostgreSQL host
`SCRIBE_TARGET_POSTGRES_PORT`	PostgreSQL port
`SCRIBE_TARGET_POSTGRES_DATABASE`	Database name
`SCRIBE_TARGET_POSTGRES_USERNAME`	Database user
`SCRIBE_TARGET_POSTGRES_PASSWORD`	Database password

Table 16.1: PQS configuration variables (Scribe target)

PQS also requires OAuth2 configuration to authenticate with the participant's gRPC Ledger API. In cn-quickstart, all of this is pre-configured in the Docker Compose setup.

16.4 Querying Contracts via SQL

Once PQS has indexed the ledger data, you can connect to PostgreSQL and run queries.

16.4.1 Connecting to the Database

```
# Connect to the PQS PostgreSQL database
docker exec -it postgres \
  psql -U pqs -d pqs
```

16.4.2 The PQS Schema

PQS does not create separate tables per template. Instead, it stores all contracts in partitioned internal tables and exposes them via **table functions**: `active()` for the current active contract set and `creates()` for all historically created contracts. These functions return a `contract` composite type with the following key columns:

Column	Type	Description
`contract_id`	text	Unique contract identifier
`payload`	jsonb	Full contract arguments
`template_fqn`	text	Fully qualified template name
`created_effective_at`	timestamptz	Effective time of creation
`archived_effective_at`	timestamptz	NULL if active, set when archived
`signatories`	text[]	Array of signatory parties
`observers`	text[]	Array of observer parties
`contract_key`	jsonb	Contract key (if defined)
`package_name`	text	Daml package name

Table 16.2: Key columns of the PQS contract type

Table Functions, Not Tables

You query PQS using `active('Package:Module:Template')` to get active contracts or `creates('Package:Module:Template')` for all creates. Pass the fully qualified template name as the argument, or call without arguments to get all templates.

16.4.3 Basic Queries

Find all active licenses:

```
SELECT contract_id,
       payload->>'product' AS product,
       payload->>'customer' AS customer,
       payload->>'expiresAt' AS expires
FROM active('Main.License:License');
```

Find licenses expiring this month:

```
SELECT contract_id,
       payload->>'product' AS product,
       payload->>'expiresAt' AS expires
FROM active('Main.License:License')
WHERE (payload->>'expiresAt')::timestamp
      < NOW() + INTERVAL '30 days';
```

Count active contracts by template type:

```
SELECT template_fqn, COUNT(*) AS cnt
FROM active()
```

```
GROUP BY template_fqn
ORDER BY cnt DESC;
```

16.4.4 JSONB Queries

The `payload` column stores the full contract data as JSONB, so you can use PostgreSQL's JSON operators:

```
-- Nested field access
SELECT payload->'details'->>'region'
FROM active('Main.License:License');

-- Filter by numeric field
SELECT contract_id, payload->>'amount'
FROM active('Main.Payment:Payment')
WHERE (payload->>'amount')::decimal > 10000;

-- Array contains
SELECT contract_id
FROM active('Main.License:License')
WHERE payload->'features' ? 'premium';
```

16.5 When to Use PQS vs Ledger API

Use PQS When	Use Ledger API When
Filtering by arbitrary contract fields	Submitting commands (create, exercise)
Running aggregate queries (COUNT, SUM)	Streaming real-time events
Joining data across template types	Reading a specific contract by ID
Building dashboards or reports	Guaranteed consistent reads
Full-text search on contract data	Low-latency command submission

Table 16.3: PQS vs Ledger API usage

Reads vs Writes

Think of it this way: the **Ledger API** is for writes (commands) and simple reads (by contract ID or template). **PQS** is for complex reads (filtering, joining, aggregating). Most production applications use both: the Ledger API for command submission and event streaming, PQS for queries and reporting.

16.6 Performance and Indexing

PQS stores contract payloads as JSONB, which is flexible but can be slow for large datasets without proper indexing.

16.6.1 Adding Indexes

PQS stores contracts in partitioned internal tables (`__contracts`). You can create indexes on the payload fields of specific partitions for better performance. Consult the PQS documentation for the recommended approach to custom indexing on your deployment.

```
-- Example: GIN index on payload for containment queries
-- (applied to the internal __contracts table)
CREATE INDEX idx_contracts_payload
  ON __contracts
  USING gin (payload);
```

16.6.2 Query Performance Tips

- Use the `active()` function to query only active contracts—it handles filtering automatically.
- Cast JSONB values to their proper types for comparisons: `(payload->>'amount')::decimal`.
- Use `EXPLAIN ANALYZE` to verify that your indexes are being used.
- For high-throughput dashboards, consider creating materialized views that refresh periodically.

16.7 Daml Shell for Interactive Queries

Daml Shell is a REPL that connects to PQS, providing an interactive way to explore contract data without writing raw SQL.

16.7.1 Starting Daml Shell

```
dpm daml-shell
```

Or via cn-quickstart:

```
make shell
```

16.7.2 Basic Commands

```
-- List available template types
\templates

-- Query active contracts of a type
SELECT * FROM license WHERE active = true;

-- Filter by field
SELECT product, customer, expires_at
FROM license
WHERE customer = 'Alice'
  AND active = true;

-- Count contracts
SELECT COUNT(*) FROM license
WHERE active = true;
```

Daml Shell vs Raw SQL

Daml Shell translates template-aware queries into the underlying PQS SQL. It provides a friendlier interface that uses template names and field names directly, without needing to know the JSONB column structure. Under the hood, it generates the same queries

you would write by hand.

16.7.3 Exploring Data

Daml Shell is particularly useful for:

- Quickly checking what contracts exist after a deployment.
- Debugging issues by inspecting contract state.
- Verifying that PQS is properly indexing new template types.
- Ad-hoc reporting during development.

16.8 Summary

- **PQS** (Participant Query Store) projects ledger events into PostgreSQL, making contract data queryable via SQL.
- PQS subscribes to the participant's **transaction stream** and maintains a materialized view of all contracts.
- Use PQS for **complex queries**: filtering by arbitrary fields, aggregations, joins. Use the Ledger API for **command submission** and **real-time streaming**.
- Query contracts via the `active()` and `creates()` table functions. Contract data is stored as **JSONB**—use PostgreSQL's JSON operators for filtering.
- PQS is **eventually consistent**—there is a small delay between ledger commits and query availability.
- **Daml Shell** (`dpm daml-shell`) provides an interactive REPL for PQS queries using template-aware syntax.
- In cn-quickstart, PQS runs as the `pqs-app-provider` container, pre-configured and ready to use.

Chapter 17

Frontend and Wallet Integration

A Daml application is not complete without a user interface. This chapter covers building a React frontend that connects to the JSON Ledger API, authenticates via Keycloak, and integrates with Canton's wallet functionality.

17.1 Frontend Architecture

A typical Daml frontend has three layers:

```
React UI Components
    |
    | (uses generated types)
    v
API Client Layer
    |
    | (HTTP requests)
    v
JSON Ledger API (+ Keycloak auth)
```

The frontend never connects directly to the gRPC API—it uses the JSON Ledger API exclusively. The API client layer handles authentication, request formatting, and response parsing using generated TypeScript types.

Backend Independence

The React frontend works with either the Express.js or Spring Boot backend from Chapter 15. Both backends expose identical REST endpoints (`/api/licenses`), so no frontend changes are needed when switching between them.

17.2 Generated TypeScript Types

The `dpm codegen-js` command generates TypeScript interfaces from your Daml templates:

```
dpm codegen-js
```

This produces files in your configured output directory (typically `frontend/src/generated/`):

```
frontend/src/generated/
  Main/
    License.ts        # License template types
    Types.ts          # Shared data types
  index.ts            # Re-exports
```

Each template file exports TypeScript interfaces matching the Daml template fields:

```
// Generated: Main/License.ts (simplified)
export interface License {
  vendor: string;
  customer: string;
  product: string;
  expiresAt: string;
}

export interface Renew {
  newExpiry: string;
}
```

Regenerate After Template Changes

Every time you modify Daml templates and rebuild the DAR, run `dpm codegen-js` again to update the TypeScript types. Stale types are a common source of runtime errors.

17.3 Connecting to the JSON Ledger API

The frontend communicates with the JSON Ledger API over HTTP. Here is a minimal API client:

```
// src/api/ledgerClient.ts
const BASE_URL = "http://localhost:3975";

export async function submitCommand(
  token: string,
  actAs: string[],
  commands: object[],
  commandId: string
): Promise<Response> {
  return fetch(
    `${BASE_URL}/v2/commands/submit-and-wait`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${token}`,
      },
      body: JSON.stringify({
        commands,
        actAs,
        commandId,
        applicationId: "license-app",
      }),
    }
  );
}
```

```
// Create a license
export async function createLicense(
  token: string,
  party: string,
  license: License
): Promise<Response> {
  return submitCommand(
    token,
```

```
    [party],
    [{
      CreateCommand: {
        templateId: "Main.License:License",
        createArguments: license,
      },
    }],
    `create-license-${Date.now()}`
  );
}
```

```
// Fetch active contracts
export async function getContracts(
  token: string,
  party: string,
  templateId: string
): Promise<object[]> {
  const resp = await fetch(
    `${BASE_URL}/v2/state/active-contracts`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${token}`,
      },
      body: JSON.stringify({
        filter: {
          filtersByParty: {
            [party]: {
              cumulative: [{
                templateFilters: [{
                  templateId,
                }],
              }],
            },
          },
        },
      }),
    }
  );
  const data = await resp.json();
  return data || [];
}
```

17.4 OAuth2 Authentication for SPAs

Browser-based applications cannot safely store client secrets. Instead, they use the OAuth2 Authorization Code flow with PKCE (Proof Key for Code Exchange).

17.4.1 The PKCE Flow

1. The user clicks "Log In."
2. The frontend generates a random code verifier and its SHA-256 hash (code challenge).
3. It redirects the browser to Keycloak's authorization endpoint with the code challenge.
4. The user authenticates in Keycloak.
5. Keycloak redirects back to the frontend with an authorization code.

6. The frontend exchanges the code (plus the original code verifier) for an access token.
7. The frontend stores the token in memory and includes it in all API requests.

Token Storage

Never store access tokens in `localStorage` or `sessionStorage`—they are vulnerable to XSS attacks. Store tokens in memory (JavaScript variables) and use refresh tokens to obtain new access tokens when they expire. Libraries like `oidc-client-ts` handle this automatically.

17.4.2 Keycloak Configuration

In cn-quickstart, Keycloak is pre-configured with a client for the frontend SPA:

Setting	Value
Client ID	`app-user-ui`
Realm	`AppUser`
Access Type	Public (no client secret)
Redirect URI	`http://localhost:3000/*`
PKCE method	S256

Table 17.1: Keycloak SPA client configuration

17.4.3 Integration with React

A typical authentication setup using `oidc-client-ts`:

```
// src/auth/authConfig.ts
import { UserManager } from "oidc-client-ts";

const KC_URL = "http://keycloak.localhost:8082";
const REALM = "AppUser";

export const userManager = new UserManager({
  authority:
    `${KC_URL}/realms/${REALM}`,
  client_id: "app-user-ui",
  redirect_uri:
    "http://localhost:3000/callback",
  response_type: "code",
  scope: "openid",
  automaticSilentRenew: true,
});
```

```
// src/auth/AuthProvider.tsx (simplified)
import React, {
  createContext, useContext,
  useEffect, useState
} from "react";
import { userManager } from "./authConfig";

interface AuthCtx {
  token: string | null;
  party: string | null;
  login: () => void;
```

```
  logout: () => void;
}

const AuthContext =
  createContext<AuthCtx | null>(null);

export function AuthProvider(
  { children }: { children: React.ReactNode }
) {
  const [token, setToken] =
    useState<string | null>(null);
  const [party, setParty] =
    useState<string | null>(null);

  useEffect(() => {
    userManager.getUser().then((user) => {
      if (user && !user.expired) {
        setToken(user.access_token);
        setParty(
          user.profile.act_as?.[0] ?? null
        );
      }
    });
  }, []);

  const login = () =>
    userManager.signinRedirect();
  const logout = () =>
    userManager.signoutRedirect();

  return (
    <AuthContext.Provider
      value={{ token, party, login, logout }}
    >
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () =>
  useContext(AuthContext)!;
```

17.5 Building the UI

With authentication and the API client in place, you can build React components that interact with the ledger.

17.5.1 Listing Contracts

```
// src/components/LicenseList.tsx
import React, {
  useEffect, useState
} from "react";
import { useAuth } from "../auth/AuthProvider";
import { getContracts } from "../api/ledgerClient";

interface LicenseContract {
  contractId: string;
  payload: {
    vendor: string;
```

```
    customer: string;
    product: string;
    expiresAt: string;
  };
}

export function LicenseList() {
  const { token, party } = useAuth();
  const [licenses, setLicenses] =
    useState<LicenseContract[]>([]);

  useEffect(() => {
    if (!token || !party) return;
    getContracts(
      token, party,
      "Main.License:License"
    ).then((data) =>
      setLicenses(data as LicenseContract[])
    );
  }, [token, party]);

  return (
    <table>
      <thead>
        <tr>
          <th>Product</th>
          <th>Customer</th>
          <th>Expires</th>
        </tr>
      </thead>
      <tbody>
        {licenses.map((l) => (
          <tr key={l.contractId}>
            <td>{l.payload.product}</td>
            <td>{l.payload.customer}</td>
            <td>{l.payload.expiresAt}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

17.5.2 Submitting Commands

```
// src/components/CreateLicense.tsx
import React, { useState } from "react";
import { useAuth } from "../auth/AuthProvider";
import { createLicense } from "../api/ledgerClient";

export function CreateLicense() {
  const { token, party } = useAuth();
  const [product, setProduct] = useState("");
  const [customer, setCustomer] = useState("");

  const handleSubmit = async () => {
    if (!token || !party) return;
    await createLicense(token, party, {
      vendor: party,
      customer,
      product,
```

```
      expiresAt: "2027-12-31T00:00:00Z",
    });
    setProduct("");
    setCustomer("");
  };

  return (
    <form onSubmit={(e) => {
      e.preventDefault();
      handleSubmit();
    }}>
      <input
        value={product}
        onChange={
          (e) => setProduct(e.target.value)
        }
        placeholder="Product name"
      />
      <input
        value={customer}
        onChange={
          (e) => setCustomer(e.target.value)
        }
        placeholder="Customer party"
      />
      <button type="submit">
        Create License
      </button>
    </form>
  );
}
```

17.5.3 Real-Time Updates with SSE

For live updates, subscribe to the JSON Ledger API's Server-Sent Events stream:

```
// src/api/eventStream.ts
export function subscribeToEvents(
  token: string,
  party: string,
  templateId: string,
  onEvent: (event: object) => void
) {
  const url =
    "http://localhost:3975/v2/updates/flats";
  const body = JSON.stringify({
    filter: {
      filtersByParty: {
        [party]: {
          cumulative: [{
            templateFilters: [{ templateId }],
          }],
        },
      },
    },
    beginExclusive: 0,
  });

  // Use fetch with streaming response
  fetch(url, {
    method: "POST",
    headers: {
```

```
      "Content-Type": "application/json",
      "Authorization": `Bearer ${token}`,
    },
    body,
  }).then(async (resp) => {
    const reader = resp.body?.getReader();
    if (!reader) return;
    const decoder = new TextDecoder();
    while (true) {
      const { done, value } =
        await reader.read();
      if (done) break;
      const text = decoder.decode(value);
      // Parse SSE events
      for (const line of text.split("\n")) {
        if (line.startsWith("data:")) {
          const data =
            JSON.parse(line.slice(5));
          onEvent(data);
        }
      }
    }
  });
}
```

17.6 Wallet Integration

Canton includes a wallet system for managing Canton Coin—a native token used for transaction fees and value transfer on the Canton Network.

17.6.1 Canton Coin Overview

Canton Coin is a tokenized representation of value on the Canton Network. Key concepts:

- **Canton Coin** is represented as Daml contracts on the ledger.
- Each coin contract has an `owner`, an `amount`, and an `issuer`.
- Coins can be transferred, split, and merged—similar to the `Asset` template from earlier chapters.
- The wallet aggregates a user's coin contracts and presents a unified balance.

17.6.2 Wallet Operations

The wallet provides higher-level operations built on top of the underlying coin contracts:

Operation	Description
Balance	Sum of all active coin contracts owned by the user
Transfer	Send coins to another party (may split/merge internally)
Receive	Accept an incoming transfer
History	List past transactions

Table 17.2: Wallet operations

17.6.3 Token Transfers

A transfer involves exercising choices on coin contracts. The wallet handles the complexity of splitting coins to the exact transfer amount:

```
// Simplified transfer flow
export async function transferCoins(
  token: string,
  senderParty: string,
  receiverParty: string,
  amount: number
) {
  // The wallet backend handles:
  // 1. Finding coins totaling >= amount
  // 2. Splitting if needed
  // 3. Transferring the exact amount
  // 4. Returning change to sender
  return submitCommand(
    token,
    [senderParty],
    [{
      ExerciseCommand: {
        templateId:
          "Splice.Wallet:WalletTransfer",
        contractId: walletContractId,
        choice: "Transfer",
        choiceArgument: {
          receiver: receiverParty,
          amount: amount.toString(),
        },
      },
    }],
    `transfer-${Date.now()}`
  );
}
```

17.7 The cn-quickstart Frontend

The cn-quickstart project includes a complete React frontend that demonstrates all of these patterns. It is located in the `frontend/` directory and includes:

- Keycloak authentication with PKCE.
- Generated TypeScript types from the Daml templates.
- Contract listing and creation via the JSON Ledger API.
- Real-time updates using event streaming.
- Wallet balance display and transfer functionality.

To run the cn-quickstart frontend:

```
cd frontend
npm install
npm start
```

The frontend starts at `http://localhost:3000` and connects to the JSON Ledger API on port 3975.

Use cn-quickstart as a Template

Rather than building a frontend from scratch, copy the cn-quickstart frontend and modify it for your application. It already handles authentication, API communication, and the standard project structure. Replace the template-specific components with your own.

17.8 Frontend Development Tips

- **Environment variables**: Store API URLs and Keycloak settings in `.env` files, not in code. React apps use `REACT_APP_*` prefixed variables.
- **Error handling**: Always handle authentication failures (expired tokens, revoked sessions) gracefully. Redirect to login on 401 responses.
- **Loading states**: Ledger commands can take 1–2 seconds to commit. Show loading indicators and disable submit buttons during command processing.
- **Optimistic updates**: After submitting a command, you can optimistically update the UI before the ledger confirms. Revert if the command fails.
- **Party display names**: Party IDs are long hashes. Maintain a mapping of party IDs to display names for a better user experience.

17.9 Summary

- The frontend connects to the **JSON Ledger API** (not gRPC) for both commands and queries.
- Use `dpm codegen-js` to generate TypeScript types from Daml templates. Regenerate after every template change.
- Browser-based apps authenticate via **OAuth2 Authorization Code flow with PKCE**—no client secrets in the frontend.
- Build a **reusable API client layer** that handles authentication headers, command formatting, and error handling.
- For real-time updates, use the streaming `/v2/updates/flats` endpoint.
- **Canton Coin** and the wallet provide native token functionality. The wallet handles coin splitting and merging internally.
- Use the **cn-quickstart frontend** as a starting point for your own applications.
- Store tokens **in memory only**—never in localStorage or sessionStorage.

Chapter 18

End-to-End Example

This chapter brings together everything from Part III into a complete application: a software licensing system. We will build the Daml model, a backend REST API, a React frontend, deploy it all to Canton, and test the full workflow end-to-end.

18.1 The Application: Software Licensing

Our capstone project is a licensing application where:

- A **vendor** creates software licenses for customers.
- A **customer** can view their licenses and request renewals.
- Licenses have an **expiration date** and can be renewed or archived.
- Both parties have **visibility** into the license state at all times.

This is conceptually similar to cn-quickstart but simplified and fully explained.

18.2 Project Structure

Following the patterns from Chapter 14:

```
license-app/
  multi-package.yaml
  daml/
    daml.yaml
    daml/
      Main/
        License.daml
        Types.daml
  daml-test/
    daml.yaml
    daml/
      Test/
        LicenseTest.daml
  backend/              # Express.js
    package.json
    .env
    src/
      index.js
      ledgerClient.js
      licenseService.js
      licenseRoutes.js
```

```
  backend-java/         # Spring Boot
    build.gradle.kts
    src/main/
      java/com/example/license/
        LicenseApplication.java
        LedgerClient.java
        LicenseService.java
        LicenseController.java
      resources/
        application.yml
  frontend/
    package.json
    src/
      api/ledgerClient.ts
      auth/AuthProvider.tsx
      components/
        LicenseList.tsx
        CreateLicense.tsx
        RenewLicense.tsx
      App.tsx
```

The `multi-package.yaml`:

```
packages:
  - daml
  - daml-test
```

18.3 The Daml Model

18.3.1 Types Module

First, define shared types in a separate module:

```
module Main.Types where

data LicenseStatus
  = Active
  | Expired
  | Revoked
  deriving (Eq, Show)
```

18.3.2 License Template

The core template with a full lifecycle:

```
module Main.License where

import DA.Time
import DA.Date
import Main.Types

template License
  with
    vendor    : Party
    customer  : Party
    product   : Text
    issuedAt  : Time
    expiresAt : Time
  where
    signatory vendor
```

```
    observer customer
    ensure expiresAt > issuedAt
```

```
    -- Renew: extend the expiration date
    choice Renew : ContractId License
      with
        newExpiry : Time
      controller vendor
      do
        now <- getTime
        assertMsg "New expiry must be future"
          (newExpiry > now)
        assertMsg "Cannot shorten license"
          (newExpiry > expiresAt)
        create this with
          expiresAt = newExpiry

    -- Revoke: vendor cancels the license
    choice Revoke : ()
      controller vendor
      do
        return ()

    -- RequestRenewal: customer asks vendor
    nonconsuming choice RequestRenewal
      : ContractId RenewalRequest
      with
        requestedExpiry : Time
        reason : Text
      controller customer
      do
        create RenewalRequest with
          licenseCid = self
          vendor
          customer
          requestedExpiry
          reason
```

18.3.3 Renewal Request Template

A separate template for the renewal workflow:

```
template RenewalRequest
  with
    licenseCid      : ContractId License
    vendor          : Party
    customer        : Party
    requestedExpiry : Time
    reason          : Text
  where
    signatory customer
    observer vendor

    choice ApproveRenewal
      : ContractId License
      controller vendor
      do
        exercise licenseCid Renew with
          newExpiry = requestedExpiry

    choice RejectRenewal : ()
```

```
      controller vendor
      do
        return ()
```

18.3.4 Package Configuration

The template package (`daml/daml.yaml`):

```
sdk-version: 3.4.11
name: license-app
source: daml
version: 1.0.0
dependencies:
  - daml-prim
  - daml-stdlib
```

18.4 Testing the Daml Model

The test package (`daml-test/daml.yaml`):

```
sdk-version: 3.4.11
name: license-app-test
source: daml
version: 1.0.0
dependencies:
  - daml-prim
  - daml-stdlib
  - daml-script
data-dependencies:
  - ../daml/.daml/dist/license-app-1.0.0.dar
```

18.4.1 Test Suite

```
module Test.LicenseTest where

import Daml.Script
import DA.Time
import DA.Optional (isNone)
import Main.License

data Parties = Parties with
  vendor   : Party
  customer : Party

setup : Script Parties
setup = do
  vendor <- allocateParty "Acme"
  customer <- allocateParty "Alice"
  return Parties with ..
```

```
test_create_license = do
  Parties{..} <- setup
  now <- getTime
  let expiry = addRelTime now (days 365)
  cid <- submit vendor do
    createCmd License with
      vendor
```

```
      customer
      product = "Enterprise Suite"
      issuedAt = now
      expiresAt = expiry
  Some lic <- queryContractId vendor cid
  assertMsg "Product"
    (lic.product == "Enterprise Suite")
  assertMsg "Customer"
    (lic.customer == customer)
```

```
test_renew_license = do
  Parties{..} <- setup
  now <- getTime
  let expiry = addRelTime now (days 365)
  let newExp = addRelTime now (days 730)
  cid <- submit vendor do
    createCmd License with
      vendor; customer
      product = "Enterprise Suite"
      issuedAt = now
      expiresAt = expiry
  newCid <- submit vendor do
    exerciseCmd cid Renew with
      newExpiry = newExp
  Some lic <- queryContractId vendor newCid
  assertMsg "Extended"
    (lic.expiresAt == newExp)
  -- Old contract archived
  old <- queryContractId vendor cid
  assertMsg "Archived" (isNone old)
```

```
test_revoke_license = do
  Parties{..} <- setup
  now <- getTime
  let expiry = addRelTime now (days 365)
  cid <- submit vendor do
    createCmd License with
      vendor; customer
      product = "Enterprise Suite"
      issuedAt = now
      expiresAt = expiry
  submit vendor do
    exerciseCmd cid Revoke
  old <- queryContractId vendor cid
  assertMsg "Revoked" (isNone old)
```

```
test_request_renewal = do
  Parties{..} <- setup
  now <- getTime
  let expiry = addRelTime now (days 365)
  let reqExp = addRelTime now (days 730)
  cid <- submit vendor do
    createCmd License with
      vendor; customer
      product = "Enterprise Suite"
      issuedAt = now
      expiresAt = expiry
  -- Customer requests renewal
  reqCid <- submit customer do
    exerciseCmd cid RequestRenewal with
      requestedExpiry = reqExp
```

```
      reason = "Need extended support"
  -- Vendor approves
  newLicCid <- submit vendor do
    exerciseCmd reqCid ApproveRenewal
  Some lic <- queryContractId vendor newLicCid
  assertMsg "Renewed"
    (lic.expiresAt == reqExp)
```

```
test_cannot_shorten_license = do
  Parties{..} <- setup
  now <- getTime
  let expiry = addRelTime now (days 365)
  let shorter = addRelTime now (days 100)
  cid <- submit vendor do
    createCmd License with
      vendor; customer
      product = "Enterprise Suite"
      issuedAt = now
      expiresAt = expiry
  -- Cannot renew to earlier date
  submitMustFail vendor do
    exerciseCmd cid Renew with
      newExpiry = shorter
```

Run the tests:

```
dpm build --all
dpm test --all
```

18.5 The Backend

The backend exposes REST endpoints and translates HTTP requests into JSON Ledger API commands. Following the patterns from Chapter 15, we show both an Express.js and a Spring Boot implementation. Choose the one that matches your stack—both expose the same endpoints.

18.5.1 Express.js Backend

A Node.js/Express.js implementation:

18.5.2 REST Endpoints

Method	Path	Description
POST	/api/licenses	Create a new license
GET	/api/licenses	List active licenses
POST	/api/licenses/renew	Renew a license
POST	/api/licenses/revoke	Revoke a license
POST	/api/renewals	Request a renewal
POST	/api/renewals/approve	Approve a request
POST	/api/renewals/reject	Reject a request

Table 18.1: Backend REST API endpoints

18.5.3 License Service

The service layer maps REST operations to JSON Ledger API calls:

```
// src/licenseService.js
import { randomUUID } from "node:crypto";
import {
  submitCommand,
  getActiveContracts,
} from "./ledgerClient.js";

const TEMPLATE =
  "Main.License:License";

export async function createLicense(
  token, vendor, customer,
  product, expiresAt
) {
  return submitCommand(
    token,
    [vendor],
    [{
      CreateCommand: {
        templateId: TEMPLATE,
        createArguments: {
          vendor, customer, product,
          issuedAt: new Date().toISOString(),
          expiresAt,
        },
      },
    }],
    `create-license-${randomUUID()}`
  );
}

export async function renewLicense(
  token, vendor, contractId, newExpiry
) {
  return submitCommand(
    token,
    [vendor],
    [{
      ExerciseCommand: {
        templateId: TEMPLATE,
        contractId,
        choice: "Renew",
        choiceArgument: {
          newExpiry,
        },
      },
    }],
    `renew-license-${randomUUID()}`
  );
}

export async function listLicenses(
  token, party
) {
  return getActiveContracts(
    token, party, TEMPLATE
  );
}
```

18.5.4 Express Routes

```
// src/licenseRoutes.js
import { Router } from "express";
import {
  createLicense,
  renewLicense,
  listLicenses,
} from "./licenseService.js";

export const licenseRoutes = Router();

licenseRoutes.post("/", async (req, res) => {
  try {
    const {
      vendor, customer, product, expiresAt
    } = req.body;
    const token =
      req.headers.authorization
        ?.split(" ")[1];
    const result = await createLicense(
      token, vendor, customer,
      product, expiresAt
    );
    res.json(result);
  } catch (err) {
    res.status(500).json({
      error: err.message,
    });
  }
});

licenseRoutes.get("/", async (req, res) => {
  try {
    const { party } = req.query;
    const token =
      req.headers.authorization
        ?.split(" ")[1];
    const data = await listLicenses(
      token, party
    );
    res.json(data);
  } catch (err) {
    res.status(500).json({
      error: err.message,
    });
  }
});

licenseRoutes.post(
  "/renew", async (req, res) => {
  try {
    const {
      vendor, contractId, newExpiry
    } = req.body;
    const token =
      req.headers.authorization
        ?.split(" ")[1];
    const result = await renewLicense(
      token, vendor, contractId, newExpiry
    );
    res.json(result);
  } catch (err) {
```

```
    res.status(500).json({
      error: err.message,
    });
  }
});
```

18.5.5 Spring Boot Backend (Java Alternative)

The same REST API implemented with Spring Boot and Java 17+:

Project Structure

```
backend-java/
  build.gradle.kts
  src/main/
    java/com/example/license/
      LicenseApplication.java
      LedgerClient.java
      LicenseService.java
      LicenseController.java
    resources/
      application.yml
```

License Service

```
// LicenseService.java
package com.example.license;

import com.fasterxml.jackson.databind.*;
import com.fasterxml.jackson.databind.node.*;
import java.util.UUID;
import org.springframework.stereotype
  .Service;

@Service
public class LicenseService {
    private static final String TEMPLATE =
      "Main.License:License";
    private final LedgerClient ledger;
    private final ObjectMapper mapper =
      new ObjectMapper();

    public LicenseService(
      LedgerClient ledger
    ) {
        this.ledger = ledger;
    }
```

```
    public JsonNode createLicense(
        String token,
        String vendor,
        String customer,
        String product,
        String expiresAt
    ) throws Exception {
        var args = mapper
          .createObjectNode();
        args.put("vendor", vendor);
```

```
        args.put("customer", customer);
        args.put("product", product);
        args.put("issuedAt",
          java.time.Instant.now()
            .toString());
        args.put("expiresAt", expiresAt);

        var cmd = mapper
          .createObjectNode();
        var create = mapper
          .createObjectNode();
        create.put("templateId", TEMPLATE);
        create.set(
          "createArguments", args
        );
        cmd.set("CreateCommand", create);

        return ledger.submitCommand(
          token,
          new String[]{ vendor },
          new ObjectNode[]{ cmd },
          "create-license-"
            + UUID.randomUUID()
        );
    }
```

```
    public JsonNode renewLicense(
        String token,
        String vendor,
        String contractId,
        String newExpiry
    ) throws Exception {
        var cmd = mapper
          .createObjectNode();
        var exercise = mapper
          .createObjectNode();
        exercise.put(
          "templateId", TEMPLATE
        );
        exercise.put(
          "contractId", contractId
        );
        exercise.put("choice", "Renew");
        var choiceArg = mapper
          .createObjectNode();
        choiceArg.put(
          "newExpiry", newExpiry
        );
        exercise.set(
          "choiceArgument", choiceArg
        );
        cmd.set(
          "ExerciseCommand", exercise
        );

        return ledger.submitCommand(
          token,
          new String[]{ vendor },
          new ObjectNode[]{ cmd },
          "renew-license-"
            + UUID.randomUUID()
        );
```

```
    }

    public JsonNode listLicenses(
        String token, String party
    ) throws Exception {
        return ledger.getActiveContracts(
          token, party, TEMPLATE
        );
    }
}
```

REST Controller

```
// LicenseController.java
package com.example.license;

import com.fasterxml.jackson.databind.*;
import org.springframework.http.*;
import org.springframework.web.bind
  .annotation.*;

@RestController
@RequestMapping("/api/licenses")
public class LicenseController {
    private final LicenseService svc;

    public LicenseController(
      LicenseService svc
    ) {
        this.svc = svc;
    }

    private String extractToken(String auth) {
        return auth.replace("Bearer ", "");
    }
```

```
    @PostMapping
    public ResponseEntity<?> create(
        @RequestBody JsonNode body,
        @RequestHeader("Authorization")
          String auth
    ) {
        try {
            var token = extractToken(auth);
            var result = svc.createLicense(
              token,
              body.get("vendor").asText(),
              body.get("customer").asText(),
              body.get("product").asText(),
              body.get("expiresAt").asText()
            );
            return ResponseEntity.ok(result);
        } catch (Exception e) {
            return ResponseEntity
              .status(500)
              .body(java.util.Map.of(
                "error", e.getMessage()
              ));
        }
    }
```

```
    @GetMapping
    public ResponseEntity<?> list(
        @RequestParam String party,
        @RequestHeader("Authorization")
          String auth
    ) {
        try {
            var token = extractToken(auth);
            var data = svc.listLicenses(
              token, party
            );
            return ResponseEntity.ok(data);
        } catch (Exception e) {
            return ResponseEntity
              .status(500)
              .body(java.util.Map.of(
                "error", e.getMessage()
              ));
        }
    }

    @PostMapping("/renew")
    public ResponseEntity<?> renew(
        @RequestBody JsonNode body,
        @RequestHeader("Authorization")
          String auth
    ) {
        try {
            var token = extractToken(auth);
            var result = svc.renewLicense(
              token,
              body.get("vendor").asText(),
              body.get("contractId").asText(),
              body.get("newExpiry").asText()
            );
            return ResponseEntity.ok(result);
        } catch (Exception e) {
            return ResponseEntity
              .status(500)
              .body(java.util.Map.of(
                "error", e.getMessage()
              ));
        }
    }
}
```

Shared `LedgerClient`

The `LedgerClient` class used here is the same one shown in Chapter 15. It wraps Java's `HttpClient` to call the JSON Ledger API endpoints.

18.6 The Frontend

The React frontend provides three main views:

- **License List**: displays all active licenses with status.
- **Create License**: form for vendors to issue new licenses.

- **Renew License**: allows customers to request and vendors to approve renewals.

The key components reuse the patterns from Chapter 17: the `AuthProvider` for Keycloak authentication, the `ledgerClient` for API calls, and generated TypeScript types for type safety.

18.6.1 Application Shell

```
// src/App.tsx
import React from "react";
import { AuthProvider } from "./auth/AuthProvider";
import { LicenseList } from
  "./components/LicenseList";
import { CreateLicense } from
  "./components/CreateLicense";

function App() {
  return (
    <AuthProvider>
      <div className="app">
        <h1>License Manager</h1>
        <CreateLicense />
        <LicenseList />
      </div>
    </AuthProvider>
  );
}

export default App;
```

The `LicenseList` and `CreateLicense` components follow the same patterns shown in Chapter 17, adapted for the License template.

18.7 Deployment

With all components built, here is the deployment sequence:

18.7.1 Step 1: Build DARs

```
dpm build --all
dpm test --all
```

18.7.2 Step 2: Upload to Canton

Using the JSON Ledger API (as shown in Chapter 4):

```
DAR=daml/.daml/dist/license-app-1.0.0.dar

# Get auth token
KC=http://keycloak.localhost:8082
TOKEN_URL="$KC/realms/AppProvider"
TOKEN_URL="$TOKEN_URL/protocol/openid-connect"
TOKEN_URL="$TOKEN_URL/token"

TOKEN=$(curl -fsS "$TOKEN_URL" \
  -d "client_id=app-provider-validator" \
  -d "client_secret=$CLIENT_SECRET" \
  -d "grant_type=client_credentials" \
```

```
  -d "scope=openid" \
  | jq -r ".access_token")

# Upload to App Provider
curl -X POST \
  http://localhost:3975/v2/packages \
  -H "Content-Type: application/octet-stream" \
  -H "Authorization: Bearer $TOKEN" \
  --data-binary @$DAR
```

Upload to All Participants

If the customer is on a different participant (e.g., App User), upload the DAR to that participant as well. Both participants must have the package to process transactions involving these templates.

18.7.3 Step 3: Start the Backend

Start either the Express.js or Spring Boot backend—both serve the same REST API on port 8080.

Option A: Express.js

```
cd backend
npm install
npm start
```

Option B: Spring Boot

```
cd backend-java
./gradlew bootRun
```

The backend starts on port 8080 and connects to the Canton JSON Ledger API on port 3975.

18.7.4 Step 4: Start the Frontend

```
cd frontend
npm install
npm start
```

The frontend starts on port 3000 and connects to the JSON Ledger API on port 3975.

18.8 End-to-End Testing with Playwright

Automated end-to-end tests verify the full workflow through the browser. Playwright is the recommended tool for Daml application E2E testing.

18.8.1 Test Setup

```
cd frontend
npm install -D @playwright/test
npx playwright install
```

18.8.2 Writing an E2E Test

```
// e2e/license-workflow.spec.ts
import { test, expect } from
  "@playwright/test";

test("full license lifecycle",
  async ({ page }) => {
  // Step 1: Login as vendor
  await page.goto("http://localhost:3000");
  await page.click("text=Log In");
  await page.fill(
    "#username", "vendor-user");
  await page.fill(
    "#password", "vendor-pass");
  await page.click("text=Sign In");

  // Step 2: Create a license
  await page.fill(
    "[name=product]", "Enterprise Suite");
  await page.fill(
    "[name=customer]", "alice-party-id");
  await page.click("text=Create License");

  // Step 3: Verify license appears
  await expect(
    page.locator("text=Enterprise Suite")
  ).toBeVisible();

  // Step 4: Verify in list
  const rows = page.locator("table tbody tr");
  await expect(rows).toHaveCount(1);
  await expect(rows.first()).toContainText(
    "Enterprise Suite");
});
```

18.8.3 Running E2E Tests

```
# Ensure all services are running first
npx playwright test
```

E2E Test Prerequisites

E2E tests require all services running: Canton, Keycloak, the backend, and the frontend. Use a Docker Compose setup or a Makefile to start everything before running tests. cn-quickstart's `make start` handles this.

18.9 Putting It All Together

Here is how the pieces from chapters 14–17 fit together in this application:

Chapter	Concept	Used Here
Ch 14	Project layout	`daml/` + `daml-test/` separation
Ch 14	Multi-package	`multi-package.yaml`, `dpm build -all`
Ch 14	Versioning	`license-app-1.0.0` in DAR path
Ch 15	JSON Ledger API	DAR upload, backend and frontend commands
Ch 15	Code generation	TypeScript (JS) and Java bindings
Ch 15	Dual backends	Express.js or Spring Boot REST API
Ch 15	OAuth2/Keycloak	Both backend and frontend auth
Ch 16	PQS	Query licenses by field
Ch 17	React frontend	License management UI
Ch 17	TypeScript codegen	Type-safe API calls
Ch 17	PKCE auth	Browser authentication

Table 18.2: Concepts applied in the E2E example

18.10 Next Steps

With a working end-to-end application, you can:

- Add more template types (e.g., `Invoice`, `Payment`).
- Implement PQS-based dashboards for license analytics.
- Add wallet integration for license payments.
- Set up a CI/CD pipeline that builds, tests, and deploys automatically.
- Explore the Canton architecture in depth (Part IV of this book).

18.11 Summary

- A complete Daml application has four parts: **Daml model** (templates), **tests** (Daml Script), **backend** (REST API wrapping the JSON Ledger API), and **frontend** (React + JSON Ledger API).
- The **License template** demonstrates a full contract lifecycle: create, renew, revoke, and a multi-party renewal request workflow.
- The **backend** can be built with **Express.js** (Node.js) or **Spring Boot** (Java 17+). Both translate REST requests into JSON Ledger API commands and expose identical endpoints.
- The **frontend** authenticates via Keycloak PKCE and communicates through the JSON Ledger API with generated TypeScript types.
- **Deployment** follows a sequence: build DARs, upload to Canton, `npm start` the backend, `npm start` the frontend.
- **Playwright E2E tests** verify the full workflow through the browser, from login to contract creation.
- This capstone project ties together **project structure** (Ch 14), **Ledger APIs** (Ch 15), **PQS** (Ch 16), and **frontend patterns** (Ch 17).

Part IV

Canton Network & Production

Chapter 19

Canton Architecture Deep Dive

The previous parts of this book focused on Daml—the language, the APIs, and building applications. This chapter shifts to the infrastructure underneath: Canton, the protocol that runs Daml smart contracts across multiple organizations with strong privacy guarantees.

Understanding Canton's architecture is not optional for production work. It determines what your application can and cannot do, how transactions are confirmed, what each party can see, and why certain operations have the latency they do.

19.1 Canton Protocol Overview

Canton is a synchronization protocol for Daml ledgers. Its job is to ensure that all parties who need to agree on a transaction do agree, without revealing data to anyone who should not see it. The protocol achieves this through a pipeline of four steps:

1. **Submission**: A participant submits a transaction to a synchronizer.
2. **Sequencing**: The sequencer assigns a global timestamp and delivers the transaction to the relevant parties.
3. **Validation**: Participants validate the transaction against their local view of the ledger.
4. **Confirmation**: Participants send confirmations (or rejections) through the mediator, which collects them and announces the result.

This pipeline runs for every transaction. The key insight is that no single node ever sees the entire transaction. Each participant sees only the parts relevant to its hosted parties, and the sequencer and mediator see only encrypted envelopes.

Privacy by Design

In Canton, privacy is not an afterthought bolted on top of a shared ledger. It is the starting point. The protocol is designed so that infrastructure operators (sequencer, mediator) never see contract data. Only the participants hosting the involved parties see the relevant parts of a transaction.

19.2 Participant Nodes

A participant node is the component that hosts parties, stores their contracts, and executes transactions on their behalf. Every organization in a Canton network runs at least one participant.

19.2.1 Ledger API

The Ledger API is the primary interface for applications. It exposes:

- **Command submission**: Submit Daml commands (create, exercise) as transactions.
- **Transaction stream**: Subscribe to a stream of committed transactions for a given party.
- **Active contract set**: Query the current set of active contracts visible to a party.
- **Completions**: Track the outcome of submitted commands.

The Ledger API is available in two forms: gRPC (for backend services) and JSON (for web applications). Both expose the same logical operations.

19.2.2 Sync Service

The sync service manages communication with synchronizers. It handles:

- Connecting to one or more synchronizers.
- Submitting transaction requests to the sequencer.
- Receiving sequenced messages and driving the confirmation protocol.
- Sending confirmation responses to the mediator.

A single participant can connect to multiple synchronizers simultaneously. This is how Canton enables cross-network transactions—a topic we cover in Chapter 20.

19.2.3 Contract Store

Each participant maintains a local contract store containing:

- **Active contracts**: Contracts that have been created but not yet archived. This is what the Ledger API queries against.
- **Archived contracts**: Contracts that have been consumed by a choice exercise. Kept for audit and pruned over time.
- **Transaction log**: The ordered sequence of committed transactions, used for the transaction stream API.

The contract store is the participant's local view of the ledger. Different participants have different contract stores because they see different subsets of the global transaction history.

Component	Responsibility
Ledger API	Application-facing interface (gRPC + JSON)
Admin API	Node management, DAR uploads, party allocation
Sync Service	Protocol execution with synchronizers
Contract Store	Local storage of contracts and transactions
Key Management	Signing keys, encryption keys, key rotation

Table 19.1: Participant node components

19.3 Synchronizers

A synchronizer is the coordination layer that enables participants to execute transactions together. It does not store contracts or execute Daml code. Its sole purpose is to order messages and mediate confirmations.

Synchronizers Are Not Blockchains

A synchronizer does not maintain a shared ledger. It does not store contract data. It does not execute smart contracts. It orders encrypted messages and mediates yes/no confirmations. This is a fundamentally different role from a blockchain node, and it is what gives Canton its privacy properties.

A synchronizer consists of two services:

1. **Sequencer**: Orders messages and delivers them to recipients.
2. **Mediator**: Collects confirmations and announces transaction outcomes.

Participants connect to a synchronizer to transact with each other. If two parties are on different synchronizers, Canton can still execute transactions between them using multi-synchronizer protocols.

19.4 The Sequencer

The sequencer is the ordering service. It receives messages from participants, assigns each message a unique timestamp, and delivers messages to their intended recipients. The sequencer guarantees:

- **Total order**: All messages get a globally consistent timestamp.
- **Exactly-once delivery**: Each message is delivered to each recipient exactly once.
- **Recipient-specific delivery**: A message is delivered only to the addresses listed in its envelope.

19.4.1 Encryption Envelopes

Messages sent through the sequencer are encrypted. Each message is wrapped in an *encryption envelope* that contains:

- The encrypted payload (transaction data, confirmation responses).
- A list of recipient addresses.
- Per-recipient encryption: each recipient gets the payload encrypted with their public key.

The sequencer sees the envelope metadata (who the recipients are) but cannot read the payload. This means the sequencer operator cannot see contract data, even though all messages flow through it.

Metadata Visibility

While the sequencer cannot read message payloads, it does see message metadata: who is sending to whom, message sizes, and timing patterns. In high-security deployments,

this metadata can be sensitive. Canton's trust model accounts for this—the sequencer is trusted for ordering and delivery, but not for confidentiality of metadata.

19.4.2 Sequencer Implementations

Canton supports multiple sequencer backends:

Backend	Use Case
Database (PostgreSQL)	Development, single-organization deployments
Block Sequencer (CometBFT)	Multi-organization networks, BFT tolerance

Table 19.2: Sequencer backend options

Historical Sequencer Backends

Earlier versions of Canton (2.x) included experimental Ethereum and Hyperledger Fabric sequencer backends. These have been removed in Canton 3.x. The current supported backends are database (PostgreSQL) and block sequencer (CometBFT).

The cn-quickstart environment uses a database-backed sequencer. The Global Synchronizer (Chapter 20) uses a CometBFT-based block sequencer with a quorum of Super Validators.

19.5 The Mediator

The mediator collects confirmation responses from participants and determines whether a transaction is committed or rejected. Its role in the protocol:

1. Receive the transaction's *informee tree*—a structure that tells the mediator which participants need to confirm, without revealing the contract data.
2. Collect signed confirmation responses (approve or reject) from each required confirming participant.
3. Determine the outcome: if all required confirmers approve, the transaction commits; otherwise it is rejected.
4. Announce the result through the sequencer.

Note that the submitting participant sends the encrypted transaction views directly to the relevant participants through the sequencer. The mediator does not forward the transaction data itself—it only tracks which confirmations are needed and whether they arrive.

The Mediator Cannot See Contracts

The mediator sees confirmation requests and responses, but the transaction payload is encrypted. The mediator knows *that* participants are confirming something, and whether they approved or rejected, but not *what* the contracts contain. This is a critical privacy property.

The mediator also enforces timeout policies. If a participant does not respond within the configured timeout, the transaction is rejected. This prevents a single slow or crashed participant from blocking the network indefinitely.

19.6 Sub-Transaction Privacy

Canton goes beyond per-contract privacy. It provides *sub-transaction privacy*: within a single transaction, different participants see different parts.

19.6.1 Transaction Views

When a Daml transaction exercises choices that create sub-transactions, Canton decomposes the transaction into *views*. Each view corresponds to a sub-tree of the transaction tree, and each participant receives only the views relevant to its hosted parties.

Consider a transaction where Alice transfers an asset to Bob, and Bob's acceptance triggers a payment to Carol. In this transaction:

- Alice's participant sees the transfer view.
- Bob's participant sees both the transfer and payment views.
- Carol's participant sees only the payment view.
- Carol never learns that Alice was involved, or that the payment was triggered by an asset transfer.

19.6.2 Merkle Trees and Blinding

Canton uses Merkle trees to implement sub-transaction privacy. The full transaction is structured as a tree:

1. Each view (sub-transaction) is a node in the tree.
2. Each node contains a cryptographic hash of its contents.
3. Participants receive only their relevant nodes in cleartext; other nodes are *blinded*—replaced by their hash.

A participant can verify that its views are part of the same transaction by checking the Merkle root hash, without seeing the blinded views. This provides both privacy (you cannot see other views) and integrity (no one can tamper with your views without changing the root hash).

Blinding Example

Imagine a transaction tree with three views: V1, V2, V3. A participant that should see only V2 receives:

- V2 in cleartext (the actual contract data).
- V1 and V3 as hashes only (blinded nodes).
- The Merkle root hash covering all three views.

The participant can verify that V2 belongs to this transaction by recomputing the root hash. But it cannot reconstruct V1 or V3 from their hashes.

19.6.3 View Encryption

Each view is encrypted separately for its recipients. The sequencer delivers a bundle of encrypted views, and each participant can decrypt only the views addressed to it. This is implemented using hybrid encryption:

1. A symmetric key is generated for each view.
2. The view is encrypted with the symmetric key (fast, for large payloads).
3. The symmetric key is encrypted with each recipient's public key (one copy per recipient).
4. Only a recipient with the matching private key can decrypt the symmetric key and then the view.

19.7 Transaction Flow: Step by Step

Let us trace a complete transaction through the Canton protocol. Alice exercises a choice on a contract, creating a new contract visible to Bob.

19.7.1 Step 1: Submission

Alice's application calls the Ledger API to exercise a choice. The participant:

1. Interprets the command against the Daml engine, producing a transaction tree.
2. Decomposes the tree into views based on party visibility.
3. Encrypts each view for its recipients.
4. Wraps the encrypted views into a submission request.
5. Sends the request to the synchronizer's sequencer.

19.7.2 Step 2: Sequencing

The sequencer:

1. Assigns a timestamp to the submission.
2. Delivers the encrypted views to the relevant participants and the mediator.
3. The timestamp establishes the transaction's position in the global order.

19.7.3 Step 3: Validation

Each receiving participant:

1. Decrypts its views using its private key.
2. Re-executes the Daml code for its views against its local contract store.
3. Checks that the contracts being consumed are still active (not already archived).
4. Checks authorization: are the right parties signing?
5. Checks the Merkle root: does the view hash match the expected root?

19.7.4 Step 4: Confirmation

Each participant sends a signed confirmation response (approve or reject) to the mediator via the sequencer. The mediator:

1. Collects responses from all required confirmers.
2. If all approve, announces **commit** through the sequencer.
3. If any reject, announces **reject** through the sequencer.
4. If the timeout expires, announces **reject**.

19.7.5 Step 5: Commit

Upon receiving the commit announcement:

1. Each participant updates its local contract store: newly created contracts become active, consumed contracts are archived.
2. The Ledger API transaction stream emits the committed transaction to subscribed applications.
3. Alice's application receives a completion event confirming success.

Confirmation Latency

The end-to-end latency of a transaction is dominated by three network round trips: submission to sequencer, sequencer to participants, and confirmation back through the mediator. In a well-connected network, this is typically 1–3 seconds. Computation time (Daml execution, Merkle verification) is negligible in comparison.

19.8 Comparison with Other Architectures

Canton's architecture is fundamentally different from both public blockchains and traditional permissioned ledgers. Understanding the differences clarifies Canton's design choices.

19.8.1 Canton vs Ethereum

Aspect	Ethereum	Canton
State model	Global state, all nodes store everything	Per-party state, each participant stores only its contracts
Privacy	All transactions public (even on L2, sequencer sees data)	Sub-transaction privacy, encrypted envelopes
Consensus	All nodes validate all transactions	Only involved participants validate their views
Finality	Probabilistic (PoS, 12 sec per slot, 15 min for economic finality)	Deterministic, confirmed in seconds
Smart contract	Solidity, EVM execution	Daml, interpreted by each participant independently
Ordering	Block producers order transactions	Sequencer orders messages, no blocks

Table 19.3: Canton vs Ethereum

The most important difference is state visibility. On Ethereum, every node processes every transaction and stores the complete state. On Canton, a participant only processes and stores data for its own parties. This means Canton can support workflows where competitors transact on the same network without revealing their positions to each other.

19.8.2 Canton vs Hyperledger Fabric

Aspect	Hyperledger Fabric	Canton
Privacy	Channels (separate ledgers per group)	Sub-transaction privacy within a single network
Endorsement	Endorsement policies per chaincode	Signatory-based authorization per template
Ordering	Raft-based orderer	Pluggable sequencer (DB, CometBFT, etc.)
State model	Key-value store, MVCC conflicts	UTXO-like, contracts consumed atomically
Cross-group	Complex, requires relay chains	Multi-synchronizer protocol, native
Smart contract	Go, Java, Node.js chaincode	Daml, platform-independent

Table 19.4: Canton vs Hyperledger Fabric

Fabric's channel model provides privacy by creating separate ledgers for different groups. The problem is that cross-channel transactions are difficult. Canton solves this differently: all parties can be on the same synchronizer, but the protocol ensures each party sees only its own data. Cross-synchronizer transactions are a native protocol feature, not an add-on.

19.9 Canton's Consistency Model

Within a single synchronizer, the sequencer establishes a total order on all messages via timestamps, and transaction processing is deterministic. However, because of sub-transaction privacy, each participant observes only the transactions relevant to its parties. The effective consistency from a participant's perspective is:

- **Causal ordering**: If transaction A happened before transaction B (A's output is B's input), then every participant sees A before B.
- **No globally visible total order**: Although the sequencer assigns a total order to all messages, a participant cannot observe transactions it is not party to. Two unrelated transactions may be observed in different relative orders by different participants. This is acceptable because they operate on disjoint contracts.
- **Conflict resolution**: If two transactions try to consume the same contract, the sequencer's timestamp determines the winner. The first one commits; the second is rejected.
- **No forks**: Unlike blockchains, there are no forks or chain reorganizations. Once a transaction is confirmed, it is final.

Privacy Implies Limited Observability

Canton does not provide linearizability (a global real-time order of all operations visible to all parties). The sequencer does establish a total order on messages, but participants

only see the subset relevant to their parties. This is a deliberate consequence of privacy: linearizability would require all participants to observe all transactions, which leaks information (you learn that *some* transaction happened, even if you cannot see its contents). Canton's model preserves privacy while guaranteeing that causally related events are correctly ordered.

This consistency model has practical implications for application design:

- You can safely build workflows that chain transactions (transfer then settle), because causal ordering guarantees the chain is respected.
- You cannot assume a global "latest state"—two participants may have slightly different views of the ledger at any given moment, as long as their views are causally consistent.
- Contention on hot contracts (many parties competing to exercise on the same contract) causes retries, because only one transaction per contract can win at each timestamp.

19.10 Summary

- Canton's **transaction pipeline** has four phases: submission, sequencing, validation, and confirmation.
- **Participant nodes** host parties, store contracts locally, execute Daml code, and expose the Ledger API to applications.
- **Synchronizers** coordinate transactions without storing contract data. They consist of a sequencer (ordering) and a mediator (confirmation).
- The **sequencer** assigns timestamps and delivers encrypted envelopes. It never sees contract payloads.
- The **mediator** collects confirmations and announces outcomes. It also never sees contract payloads.
- **Sub-transaction privacy** uses Merkle trees and per-view encryption so that each participant sees only the parts of a transaction relevant to its parties.
- A complete transaction flow involves: Daml interpretation, view decomposition, encryption, sequencing, parallel validation, confirmation collection, and commit.
- Compared to **Ethereum**, Canton provides per-party state and sub-transaction privacy instead of a globally replicated ledger.
- Compared to **Hyperledger Fabric**, Canton provides fine-grained privacy within a single network instead of coarse-grained privacy via channels.
- The sequencer provides a **total order** on messages, but each participant observes only the transactions relevant to its parties. Causally related transactions are correctly ordered; unrelated transactions may not be observed at all. This preserves privacy while guaranteeing correctness.

Chapter 20

The Global Synchronizer

Chapter 19 explained how Canton's protocol works within a single synchronizer. This chapter zooms out to the Canton Network—a network of networks—and its backbone: the Global Synchronizer.

20.1 A Network of Networks

The Canton Network is not a single blockchain. It is a collection of synchronizers, each operated by different organizations, connected through a common protocol. Any participant can connect to any synchronizer, and transactions can span multiple synchronizers.

This architecture solves a fundamental tension in enterprise distributed ledgers: organizations want to transact with each other, but they do not want to share infrastructure or trust a single operator. With Canton:

- Each organization can run its own synchronizer for internal workflows.
- Organizations connect to shared synchronizers for inter-company transactions.
- A single participant can be connected to multiple synchronizers simultaneously.
- Transactions that span synchronizers are handled by the protocol, not by the application.

Why Multiple Synchronizers?

Different synchronizers serve different purposes. A bank might run a private synchronizer for internal operations (fast, low-cost, fully controlled), while connecting to an industry synchronizer for inter-bank settlements. The Global Synchronizer provides a universal coordination layer that any Canton participant can connect to.

20.2 The Global Synchronizer (sync.global)

The Global Synchronizer is the flagship shared synchronizer of the Canton Network. It is operated by a decentralized group of Super Validators and provides:

- A globally accessible synchronizer that any validator can connect to.
- BFT (Byzantine Fault Tolerant) consensus via CometBFT.
- An on-ledger token (Canton Coin / Amulet) for paying network fees and rewarding operators.

- A standard identity layer for parties across the network.

The Global Synchronizer is to Canton what a public L1 is to Ethereum rollups—a shared coordination layer. But unlike Ethereum, the Global Synchronizer never sees contract data. It only orders encrypted messages and mediates confirmations.

20.3 Super Validators

Super Validators (SVs) are the operators of the Global Synchronizer. They run the sequencer and mediator infrastructure using CometBFT consensus.

20.3.1 Role and Responsibilities

Each Super Validator:

- Runs a CometBFT node that participates in consensus.
- Operates sequencer and mediator components.
- Votes on governance decisions (fee schedules, protocol upgrades, onboarding new SVs).
- Receives mining rewards in Canton Coin for operating the infrastructure.

20.3.2 CometBFT Consensus

The Global Synchronizer uses CometBFT (formerly Tendermint) for ordering messages. CometBFT provides:

- **BFT tolerance**: The network operates correctly as long as more than 2/3 of Super Validators are honest.
- **Instant finality**: Once a message is sequenced, it is final. No forks, no reorganizations.
- **Deterministic ordering**: All Super Validators agree on the same message order.

The 2/3 Quorum

CometBFT requires more than 2/3 of validators to agree on each block. For example, with N Super Validators, at least $\lfloor 2N/3 \rfloor + 1$ must be online and honest. If more than 1/3 are down or malicious, the network halts (safety is preserved—no incorrect results, but no progress). This is a standard BFT trade-off: safety over liveness. The number of SVs grows over time as new organizations are onboarded through governance votes.

The SV set is governed by a Daml-based governance application running on the Global Synchronizer itself. Adding or removing an SV requires a vote by the existing SVs.

20.4 Validators

Validators are the participants that connect to the Global Synchronizer. Any organization can become a validator by:

1. Running a Canton participant node.
2. Onboarding to the Global Synchronizer (registering topology transactions, obtaining connectivity).
3. Optionally running application-specific services (wallet, automation).

20.4.1 Onboarding Process

The typical onboarding flow for a new validator:

1. **Deploy infrastructure**: Set up a Canton participant node, database, and supporting services.
2. **Generate keys**: Create signing and encryption key pairs.
3. **Register with the network**: Submit topology transactions that register the participant's identity and keys with the Global Synchronizer.
4. **Get sponsored**: An existing SV or validator sponsors the new validator, vouching for its identity.
5. **Connect**: The participant connects to the Global Synchronizer and can begin transacting.

Role	Description
Super Validator	Operates sequencer/mediator, participates in consensus, governs the network
Validator	Connects to the network, hosts parties, submits transactions
Application Provider	Builds applications on top of validators
End User	Uses applications through a validator

Table 20.1: Canton Network roles

20.5 Canton Coin (Amulet)

The Global Synchronizer uses an on-ledger token called Canton Coin (also known as Amulet) to fund network operations.

20.5.1 Tokenomics

Canton Coin serves two primary purposes:

1. **Traffic fees**: Participants pay Canton Coin to submit transactions. Fees are denominated in USD but settled by burning Canton Coin, and are proportional to the size of the transaction (bytes sequenced).
2. **Mining rewards**: Super Validators, validators, and application providers receive Canton Coin for operating infrastructure and driving network usage.

Governance Is On-Chain, Not Coin-Weighted

SV governance operates through a Daml-based governance application where SVs vote on proposals (fee schedules, protocol upgrades, onboarding). Voting power is determined by SV membership status, not by Canton Coin holdings. This is different from token-weighted governance in many public blockchains.

Traffic-Based Pricing

Unlike gas-based pricing (Ethereum), Canton fees are based on the number of bytes sequenced, not computational complexity. Fees are denominated in USD but settled by burning Canton Coin at the prevailing market rate. This is because the sequencer's cost is proportional to message size, not Daml execution cost. Daml execution happens on participants, who bear their own compute costs.

20.5.2 Rewards and Issuance

Canton Coin issuance follows a schedule governed by the SV committee:

- New coins are minted each *mining round* (a 10-minute interval).
- Rewards are distributed to SVs (for operating consensus), validators (for providing user access), and application providers (for driving network usage). The split evolves over time—initially SVs received the majority, but by mid-2025 application providers became eligible for the largest share (approximately 62%), with SVs receiving approximately 20%.
- The issuance rate decreases over time through periodic halvings. Over the first 10 years, a total of approximately 100 billion Canton Coin is expected to be minted, after which annual issuance targets approximately 2.5 billion with an equivalent burn target to stabilize supply.
- Network usage fees are burned (removed from circulation), creating a burn-and-mint equilibrium that ties token supply to actual network usage.

20.6 The Splice Stack

The Global Synchronizer runs on the Splice stack—a set of Daml applications and services built on top of Canton that provide network-level functionality.

20.6.1 Splice Components

Component	Purpose
SV App	Governance, voting, onboarding SVs
Validator App	Validator onboarding, traffic management
Wallet App	Canton Coin transfers, balance management
Scan App	Network-wide queries, reference data
Amulet Rules	Daml contracts governing coin issuance and fees

Table 20.2: Splice stack components

Splice is open source and provides the infrastructure layer that makes the Global Synchronizer a functioning network rather than just a raw synchronizer. Your cn-quickstart environment includes a local Splice deployment.

20.6.2 Relationship to Your Application

Your application sits on top of the Splice stack:

1. Your Daml contracts run on a participant (validator).
2. The participant connects to the Global Synchronizer via Splice.
3. Transaction fees are paid in Canton Coin through Splice's wallet.
4. Your application's parties are registered through Splice's identity layer.

You do not need to interact with Splice directly for most application logic. Splice handles network-level concerns (fees, identity, governance) transparently.

20.7 Token Standard APIs

The Canton Network defines a Token Standard that provides uniform APIs for working with tokenized assets. These APIs abstract over the underlying Daml templates to provide a consistent interface.

20.7.1 Core Concepts

The Token Standard (CIP-0056) defines key abstractions for working with tokenized assets:

Concept	Description
Metadata	Token definition: properties of a token type
Holding	A party's balance of a specific token (UTXO-based)
Transfer Instruction	Moving tokens between parties
Allocation	Reserving tokens for a pending settlement

Table 20.3: Token Standard abstractions (CIP-0056)

20.7.2 Metadata and Holdings

Token metadata defines the properties of a token type. A holding represents a party's token balance as a Daml contract on the ledger. Holdings follow a UTXO model—they are created and consumed rather than mutated in place.

20.7.3 Transfers and Allocations

The Token Standard supports two primary transfer patterns:

- **Free-of-Payment (FOP) transfers**: A direct transfer of tokens from one party to another.
- **Delivery versus Payment (DVP) transfers**: Parties allocate their respective holdings to a settlement, and the settlement executes all transfers atomically in a single Daml transaction. This ensures that either all legs of the trade settle or none do.

Allocations reserve tokens for a pending operation, ensuring they are not double-spent before the settlement completes.

Token Standard Is Evolving

The Token Standard (CIP-0056) is actively developed. The exact Daml interfaces, field names, and API shapes may change between releases. Consult the current specifi-

cation at https://github.com/global-synchronizer-foundation/cips/blob/main/cip-0056/cip-0056.md and the Token Standard section of https://docs.sync.global for the latest details.

20.8 Network Environments

The Canton Network provides a progression of environments for developing and deploying applications:

Environment	Access	Purpose
LocalNet	Local machine	Development and unit testing (cn-quickstart)
DevNet	Shared	Integration testing with other teams
TestNet	Shared	Pre-production testing, realistic conditions
MainNet	Production	Live network with real Canton Coin

Table 20.4: Canton Network environments

20.8.1 LocalNet (cn-quickstart)

This is what you have been using throughout this book. It runs a complete Canton Network locally with Docker Compose:

- Three participants (App User, App Provider, Super Validator).
- A local synchronizer with database-backed sequencer.
- A local Splice deployment with wallet and scan.
- Keycloak for authentication.

LocalNet is free, fast, and fully under your control. The trade-off is that it does not test multi-organization scenarios.

20.8.2 DevNet and TestNet

DevNet and TestNet are shared environments operated by the Canton Network team:

- Real multi-party, multi-organization transactions.
- Test Canton Coin (no real value).
- Shared SVs operating consensus.
- Realistic latency and failure modes.

To connect to DevNet or TestNet, you configure your participant to connect to the shared synchronizer instead of a local one.

20.8.3 MainNet

MainNet is the production Global Synchronizer:

- Real Canton Coin with real value.
- Production-grade SV committee.
- Full BFT consensus.
- SLA guarantees from SV operators.

20.9 Connecting Your Application

To connect an application to the Global Synchronizer, you need to configure your participant node. The key configuration is the synchronizer connection:

```
# Canton participant configuration (canton.conf or environment variables)
canton.participants.myParticipant {
  storage.type = postgres
  storage.config {
    url = "jdbc:postgresql://db:5432/participant"
    user = "canton"
    password = $DB_PASSWORD
  }
  init.auto-init = true
}
```

The synchronizer connection is established through the Admin API:

```
// Connect to a synchronizer (URL depends on target environment)
val syncConfig = SynchronizerConnectionConfig(
  "https://<sequencer-url>:<port>")
myParticipant.synchronizers.connect("global", syncConfig)
myParticipant.synchronizers.list_connected()  // verify
```

Multi-Synchronizer

A participant can be connected to multiple synchronizers simultaneously. Internal transactions use the private synchronizer (fast, free), while cross-organization transactions use the Global Synchronizer (BFT, requires Canton Coin for fees).

20.10 The Scan API

The Scan application provides read-only access to network-wide reference data. It is the "block explorer" of the Canton Network, though what it exposes is fundamentally different from a blockchain explorer (because Canton does not have public transactions).

The Scan API provides:

- **Network status**: Connected SVs, validator count, synchronizer health.
- **Canton Coin data**: Total supply, issuance history, fee schedules.
- **Round information**: Current round number, reward distributions.
- **Validator directory**: Registered validators and their connectivity.

20.10.1 Querying the Scan API

```
# Base URL depends on deployment (proxied through nginx in cn-quickstart)
SCAN_URL="http://scan.localhost:4000"
# Consult the OpenAPI reference at docs.sync.global for all endpoints
curl -s "$SCAN_URL/api/scan/" | jq "."
```

Scan API Endpoints

The Scan API path structure has evolved across Splice releases. The examples in this section show the general pattern. Always consult the OpenAPI reference for your specific Splice version at https://docs.sync.global to confirm exact endpoint paths and request/response formats.

Scan Shows Network Data, Not Contracts

The Scan API shows network-level reference data (rounds, fees, validator lists). It does *not* expose individual contracts or transactions—that would violate Canton's privacy model. To query your application's contracts, use the Ledger API or PQS as shown in earlier chapters.

20.11 Summary

- The **Canton Network** is a network of networks—multiple synchronizers connected through a common protocol.
- The **Global Synchronizer** (sync.global) is a shared, BFT-based synchronizer operated by a decentralized committee of Super Validators.
- **Super Validators** run CometBFT consensus, requiring more than 2/3 agreement for liveness. They govern the network through on-ledger voting.
- **Validators** are participants that connect to the Global Synchronizer to host parties and submit transactions.
- **Canton Coin (Amulet)** funds network operations through traffic fees (bytes sequenced, denominated in USD, settled by burning coins) and mining rewards distributed to SVs, validators, and application providers.
- The **Splice stack** provides governance, wallet, scan, and onboarding services on top of Canton.
- The **Token Standard** (CIP-0056) defines uniform APIs for metadata, holdings, transfer instructions, and allocations.
- Environments progress from **LocalNet** (cn-quickstart) through **DevNet**, **TestNet**, and **MainNet**.
- The **Scan API** provides read-only access to network reference data (rounds, fees, validator lists) without exposing private contract data.

Chapter 21

Identity, Topology and Keys

Every distributed system needs an identity model. Who are the actors? How are they identified? How do you prove you are who you claim to be? Canton's answers to these questions are different from most blockchain platforms—and understanding them is essential for production deployments.

In earlier chapters, `allocateParty "Alice"` was sufficient. In production, party identity involves namespaces, cryptographic keys, topology transactions, and multi-hosting. This chapter covers all of it.

21.1 Party Identity at the Protocol Level

A party in Canton is not a username or an account. It is a globally unique identifier with a specific structure:

```
Alice::12202a8b3f4e5d6c7a8b...    # format: <display-name>::<namespace>
```

Display name A human-readable hint (e.g., "Alice", "Acme-Treasury"). It is *not* unique by itself—two different parties can have the same display name if they are in different namespaces.

Namespace A cryptographic identifier derived from a key. This is what makes the party globally unique.

Parties Are Not Accounts

On Ethereum, an address is derived from a private key. Whoever holds the key controls the address. In Canton, a party is an *abstract identity* that is *mapped* to one or more participants. The party itself does not hold keys. The participant holds keys and acts on the party's behalf. This separation enables multi-hosting, key rotation, and migration.

21.2 Namespaces and Identifiers

Every Canton node (participant, sequencer, mediator) has a *namespace* derived from its root key. The namespace is a hash of the public key:

```
namespace = hash(root-signing-public-key)  # derived from root key
```

When a participant allocates a party, the party's namespace is set to the participant's namespace by default. This means the participant "owns" the party in the sense that it controls the namespace.

21.2.1 Identifier Structure

A full Canton identifier (for any entity) has the form:

```
<type>::<name>::<namespace>              # full form
Alice::12202a8b3f4e5d6c7a8b...          # party (type prefix omitted)
PAR::my-participant::12202a8b3f4e5d6c7a8b...  # participant
global-sync::12209e7f6a5b4c3d2e1f...    # synchronizer (uses alias)
```

Prefix	Entity	Example
(none)	Party	`Alice::1220abc...`
PAR	Participant	`PAR::node1::1220abc...`
MED	Mediator	`MED::med1::1220def...`
SEQ	Sequencer	`SEQ::seq1::1220def...`
(alias)	Synchronizer	`da::1220abc...`

Table 21.1: Canton identifier prefixes

Synchronizer Identifiers

Unlike participants, mediators, and sequencers, synchronizer identifiers use the synchronizer's alias as the human-readable component rather than a standard three-letter prefix. The alias is set during synchronizer initialization and can be customized.

21.3 Party Allocation

In development (Daml Script), party allocation is a one-liner:

```
alice <- allocateParty "Alice"
```

This creates a party in the script's test ledger with a generated namespace. In production, party allocation is a two-step process:

1. **Generate the party identity**: The participant creates a new party ID in its namespace.
2. **Register the party-to-participant mapping**: A topology transaction is broadcast that tells the network "this party is hosted on this participant."

21.3.1 Canton Console Allocation

```
// Allocate a party on a specific participant
val alice = appProvider.parties.enable(
  "Alice"
)

// View the result
alice.toLf  // Full party ID with namespace
```

21.3.2 Admin API Allocation

```
# Allocate via the Admin API (JSON)
curl -X POST \
```

```
  "http://localhost:3902/v2/parties" \
  -H "Content-Type: application/json" \
  -d '{"party_id_hint": "Alice"}'
```

21.3.3 Ledger API Allocation

```
# Allocate via the Ledger API
curl -X POST \
  "http://localhost:3975/v2/parties" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"party_id_hint": "Alice"}'
```

Party ID Hints Are Not Guaranteed

The `party_id_hint` is a suggestion. The participant may modify it to ensure uniqueness (e.g., appending a suffix). Always use the returned party ID, not the hint, in subsequent operations.

21.4 Topology Transactions

Topology transactions are the mechanism by which the Canton network learns about its own structure. They are not Daml transactions—they are protocol-level messages that describe:

- Which participants exist and what keys they use.
- Which parties are hosted on which participants.
- Which synchronizers exist and what sequencers/mediators they run.
- What permissions parties have (submission, confirmation, observation).

21.4.1 Types of Topology Transactions

Type	Purpose
Namespace Delegation	Delegate authority over a namespace to a key
Owner-to-Key Mapping	Associate signing/encryption keys with a node
Party-to-Key Mapping	Associate protocol signing keys with a party (for external parties)
Party-to-Participant	Map a party to one or more participants
Synchronizer Parameters	Set synchronizer-wide configuration
Mediator State	Register or update mediator nodes
Sequencer State	Register or update sequencer nodes
Vetted Packages	Approve packages for use on a synchronizer

Table 21.2: Topology transaction types

21.4.2 How Topology Transactions Propagate

Topology transactions are submitted through the Admin API and propagated through the synchronizer:

1. A node submits a topology transaction to its local topology manager.
2. The topology manager signs it with the node's signing key.
3. The signed transaction is sent to the synchronizer's topology dispatcher.
4. The dispatcher sequences the transaction and delivers it to all participants.
5. Each participant validates and stores the topology state.

```
appProvider.topology.transactions.list()        // all topology transactions
appProvider.topology.party_to_participant.list() // filter by type
```

21.5 Key Management

Canton uses two types of cryptographic keys:

21.5.1 Signing Keys

Signing keys authenticate actions. Every participant, sequencer, and mediator has at least one signing key. Signing keys are used for:

- Signing topology transactions.
- Signing transaction confirmations.
- Authenticating messages to the sequencer.

21.5.2 Encryption Keys

Encryption keys protect transaction data. Each participant has at least one encryption key. Encryption keys are used for:

- Encrypting transaction views (sub-transaction privacy).
- Encrypting messages in sequencer envelopes.

21.5.3 Key Listing and Inspection

```
appProvider.keys.secret.list()  // all keys
appProvider.keys.secret.list()
  .filter(_.purpose == KeyPurpose.Signing)     // signing keys only
appProvider.keys.secret.list()
  .filter(_.purpose == KeyPurpose.Encryption)  // encryption keys only
```

21.5.4 Key Rotation

Key rotation replaces an active key with a new one. This is essential for security hygiene in production:

```
// Generate a new signing key
val newKey = appProvider.keys.secret
  .generate_signing_key("rotation-2026")

// Update the owner-to-key mapping
```

```
appProvider.topology.owner_to_key_mappings
  .rotate_key(
    appProvider,
    appProvider.keys.secret
      .list()
      .find(_.name == "old-key")
      .get
      .id,
    newKey.id
  )
```

Key Rotation Is Not Instant

After rotating a key, the old key remains valid until the topology transaction propagates and takes effect. There is a brief period where both old and new keys are valid. Plan key rotations during low-traffic periods and verify propagation before decommissioning old keys.

21.6 Party-to-Participant Mapping

The party-to-participant mapping is the topology transaction that tells the network where a party is hosted. This mapping determines:

- Which participant(s) can submit transactions on behalf of the party.
- Which participant(s) receive transaction data for the party.
- What permissions the party has on each participant.

21.6.1 Permission Levels

Each party-to-participant mapping includes a permission level:

Permission	Capabilities
Submission	Can submit transactions on behalf of the party
Confirmation	Can confirm transactions involving the party
Observation	Can observe transactions involving the party (read-only)

Table 21.3: Party-to-participant permission levels

A typical setup gives a party Submission and Confirmation on its primary participant, and Observation on a backup participant for disaster recovery.

21.6.2 Viewing Mappings

```
appProvider.topology.party_to_participant.list()  // all mappings
appProvider.topology.party_to_participant
  .list().filter(_.item.partyId == alice)  // specific party
```

21.7 Multi-Hosting

Multi-hosting means a single party is mapped to multiple participants. This enables several important scenarios:

21.7.1 High Availability

A party can be hosted on two participants in an active-passive configuration. If the primary participant goes down, the backup can continue operating:

```
// Map Alice to a second participant
// for disaster recovery
appProvider.topology.party_to_participant
  .propose(
    party = alice,
    participant = backupParticipant.id,
    permission = ParticipantPermission
      .Observation,
    groupAddressing = true
  )
```

21.7.2 Geographic Distribution

A party can be hosted on participants in different regions for latency optimization. Users in Europe connect to the European participant; users in Asia connect to the Asian participant.

21.7.3 Migration

Multi-hosting enables zero-downtime party migration between participants:

1. Add the new participant to the party mapping (with Observation).
2. Wait for the new participant to catch up on contract state.
3. Upgrade the new participant's permission to Submission + Confirmation.
4. Downgrade the old participant to Observation.
5. Remove the old participant from the mapping.

Multi-Hosting Requires Coordination

When a party is hosted on multiple participants with Submission permission, both participants can submit transactions. The Canton protocol handles conflicts (double-spends) through the normal confirmation protocol—the sequencer orders competing submissions, and only the first one commits. However, application-level coordination is recommended to avoid unnecessary rejections.

21.8 Namespace Delegation

Namespace delegation allows one key to authorize actions in another key's namespace. This is the foundation for organizational key hierarchies.

21.8.1 Use Cases

- **Root key protection**: Keep the root namespace key offline. Delegate to an operational key for day-to-day use.
- **Organizational structure**: A parent organization delegates sub-namespaces to subsidiaries.
- **Key recovery**: If an operational key is compromised, the root key can revoke the delegation and delegate to a new key.

21.8.2 Delegation Hierarchy

```
// Delegate from root namespace to an
// intermediate key
appProvider.topology.namespace_delegations
  .propose(
    namespace = appProvider.namespace,
    targetKey = intermediateKey.id,
    isRootDelegation = false
  )
```

Root vs Intermediate Delegation

A *root delegation* gives the target key full authority over the namespace, including the ability to create further delegations. An *intermediate delegation* gives the target key authority to manage topology transactions but not to create new delegations. Use root delegation sparingly—only for trusted backup keys.

21.9 Decentralized Identity

Canton's identity model supports decentralized identity patterns where parties control their own identifiers without depending on a central authority.

21.9.1 Self-Sovereign Namespaces

Because a namespace is derived from a key that the party's operator generates, no central authority assigns or controls party identities. The network's topology protocol ensures that:

- Two parties in different namespaces cannot collide (the namespace hash ensures uniqueness).
- A party's operator can rotate keys, migrate participants, and delegate authority without asking anyone's permission.
- The synchronizer validates topology transactions but does not assign identities.

21.9.2 Cross-Synchronizer Identity

A party's identity is not tied to a specific synchronizer. The same party (same identifier, same namespace) can transact on multiple synchronizers. This is because:

1. The party ID is derived from the participant's root key, not from any synchronizer.
2. Topology transactions are registered per synchronizer, so the party must be registered on each synchronizer it wants to use.
3. The participant manages connections to multiple synchronizers and routes transactions accordingly.

```
// Check which synchronizers a party is
// registered on
appProvider.synchronizers.list_connected()
  .foreach { sync =>
    val mappings = appProvider.topology
      .party_to_participant
```

```
      .list(filterStore = sync.synchronizerId
        .toProtoPrimitive)
    println(s"$sync: ${mappings.size} parties")
  }
```

21.10 Summary

- A **party** is a globally unique identifier in the format `display-name::namespace`, where the namespace is derived from a cryptographic key.
- **Namespaces** are hashes of root signing keys. They ensure global uniqueness without a central authority.
- **Party allocation** in production involves creating the party identity and registering a party-to-participant topology transaction.
- **Topology transactions** describe the network's structure: parties, participants, keys, synchronizers, and their relationships. They propagate through the synchronizer.
- Canton uses **signing keys** (for authentication) and **encryption keys** (for privacy). Both support rotation through topology transactions.
- **Party-to-participant mappings** define where a party is hosted and what permissions it has (submission, confirmation, observation).
- **Multi-hosting** maps one party to multiple participants, enabling high availability, geographic distribution, and zero-downtime migration.
- **Namespace delegation** creates key hierarchies: keep the root key offline, delegate to operational keys, revoke if compromised.
- Canton's identity model is **decentralized**: parties control their own namespaces, identities work across synchronizers, and no central authority assigns identifiers.

Chapter 22

Operations and Deployment

Development with cn-quickstart is convenient: a single `make start` brings up the entire network. Production is different. This chapter covers what it takes to run Canton in production: deployment, monitoring, maintenance, and scaling.

22.1 From Docker Compose to Kubernetes

The cn-quickstart environment runs everything in Docker Compose: three participants, a synchronizer, Keycloak, PostgreSQL, and the Splice services. This is fine for a single developer, but production deployments use Kubernetes.

22.1.1 Why Kubernetes

- **High availability**: Kubernetes restarts failed pods automatically.
- **Scaling**: Add participant replicas to handle more load.
- **Rolling upgrades**: Update components without downtime.
- **Resource management**: CPU and memory limits prevent noisy-neighbor problems.
- **Secret management**: Kubernetes Secrets and external vaults for keys and credentials.

22.1.2 Deployment Architecture

A typical production deployment consists of:

Component	Deployment
Participant	StatefulSet with persistent volume for contract store
PostgreSQL	Managed database (Cloud SQL, RDS) or StatefulSet
Sequencer	StatefulSet or external service (Global Synchronizer)
Mediator	StatefulSet or external service (Global Synchronizer)
JSON API	Deployment (stateless, horizontally scalable)
PQS	StatefulSet with its own PostgreSQL database
Application	Deployment (your backend services)

Table 22.1: Production deployment components

Managed Synchronizer vs Global Synchronizer

If you are connecting to the Global Synchronizer, you do not deploy the sequencer and mediator yourself—the SV committee operates them. You only deploy your participant, database, and application services.

22.1.3 Helm Charts

Canton and Splice provide Helm charts for Kubernetes deployment. The exact repository URL and chart names depend on your deployment scenario (standalone Canton participant vs. Splice-based validator). Consult the deployment documentation for your target environment:

```
# Install a participant via Helm (URL/chart specific to your version)
helm repo add canton <HELM_REPO_URL>
helm repo update
helm install my-participant canton/canton-participant \
  --namespace canton --values participant-values.yaml
```

A minimal `participant-values.yaml`:

```
storage:
  type: postgres
  host: postgres.canton.svc
  port: 5432
  database: participant
  user: canton
  password:
    secretName: canton-db-secret
    secretKey: password

init:
  autoInit: true

resources:
  requests:
    cpu: "2"
    memory: "4Gi"
  limits:
    cpu: "4"
    memory: "8Gi"
```

22.2 Canton Console Operations

The Canton Console is your primary operational tool. It connects to participants via their Admin API and provides a Scala REPL for management tasks.

22.2.1 Health Checks

```
appProvider.health.status()    // NodeStatus(Active, ...)
appProvider.health.active()    // true if the node is active
appProvider.synchronizers.list_connected()
```

22.2.2 Diagnostics

```
appProvider.id                 // participant's ID
```

```
appProvider.parties.hosted()    // hosted parties
appProvider.health.maybe_ping(appProvider.id)  // pending transactions

// Synchronizer connectivity details
appProvider.synchronizers.list_connected()
  .foreach { s =>
    println(s"${s.synchronizerId}: ${s.healthy}")
  }
```

22.2.3 Ping Test

The ping test verifies end-to-end connectivity by sending a transaction through the synchronizer:

```
// Creates and immediately archives a contract
appProvider.health.ping(appUser)  // Duration (round-trip time)
```

If the ping succeeds, you know the full protocol pipeline works: submission, sequencing, validation, confirmation, and commit.

22.3 DAR Lifecycle

Managing Daml packages (DARs) in production requires more care than in development. The lifecycle involves upload, vetting, and eventual retirement.

22.3.1 Upload

Upload a DAR to a participant:

```
appProvider.dars.upload("/path/to/my-app-1.0.0.dar")  // returns DAR hash
appProvider.dars.list()
```

```
# Upload via Admin API
curl -X POST \
  "http://localhost:3902/v2/packages" \
  -H "Content-Type: application/octet-stream" \
  --data-binary @my-app-1.0.0.dar
```

22.3.2 Vetting

Before a package can be used on a synchronizer, it must be *vetted*—approved by the participant for use. Vetting is a topology transaction:

```
appProvider.topology.vetted_packages
  .propose(appProvider.id, Seq(packageId))  // vet a package
appProvider.topology.vetted_packages.list()  // list vetted
```

Vetting Is a Safety Mechanism

Vetting prevents a malicious party from forcing your participant to execute arbitrary Daml code. Your participant will only process transactions involving packages you have explicitly vetted. If someone submits a transaction using an un-vetted package, your participant rejects its part of the transaction.

22.3.3 Unvetting

To retire a package (e.g., after migrating to a new version):

```
// New list without the old package
appProvider.topology.vetted_packages
  .propose(appProvider.id, Seq(newPackageId))
```

Unvet With Care

Unvetting a package means your participant will reject transactions involving that package. If there are still active contracts using the old package, they become unusable from your participant's perspective. Always migrate active contracts to the new package version before unvetting the old one.

22.3.4 Package Dependencies

DARs can depend on other DARs. When you upload a DAR, all its dependencies must also be present on the participant:

```
appProvider.packages.list()
  .filter(_.packageId == myPackageId)
  .head.dependencies  // check package dependencies
```

The upload command automatically uploads transitive dependencies included in the DAR file. However, dependencies from external sources (e.g., the Splice libraries) must be uploaded separately.

22.4 Monitoring

Production Canton deployments should be monitored with industry-standard tools: Prometheus for metrics, Grafana for dashboards, and a log aggregator for structured logs.

22.4.1 Prometheus Metrics

Canton exposes Prometheus metrics on a dedicated metrics port, configured separately from the Admin API and Ledger API. The metrics reporter is configured in Canton's monitoring configuration:

```
# Canton monitoring configuration (monitoring.conf or canton.conf)
canton.monitoring.metrics {
  reporters = [{
    type = prometheus
    address = "0.0.0.0"
    port = 10013  # configurable
  }]
}

# Scrape metrics (port depends on your configuration)
curl -s "http://localhost:10013/metrics" | head -20
```

Canton organizes metrics by category using qualifiers: errors, latency, saturation, traffic, and debug. Metric names follow the pattern `canton_<component>_<metric>`. Consult the Canton documentation for the full list of available metrics for your version.

Metrics Port Is Not the Admin API Port

A common mistake is to assume metrics are served on the Admin API port (e.g., 3902). Canton exposes Prometheus metrics on a separate, configurable port (default 9000, or as configured in `monitoring.conf`). In cn-quickstart, this port is 10013.

22.4.2 Grafana Dashboards

Canton ships with pre-built Grafana dashboards. In cn-quickstart, Grafana is available at `http://localhost:3030` and comes with dashboards organized into folders:

- **Participant**: Commands, Database operations, Caching, Metered events, Received events, Dirty requests.
- **Platform**: Health, gRPC endpoints, JVM Metrics, JVM Executor Services, Network Operation Center Health.
- **Infrastructure**: cAdvisor Docker insights, PostgreSQL.

```
curl -s http://localhost:3030/api/health | jq "."       # verify Grafana
curl -s http://localhost:3030/api/search | jq ".[].title"  # list dashboards
```

22.4.3 Structured Logging

Canton uses structured JSON logging that integrates with log aggregators like Loki, Elasticsearch, or Splunk. In cn-quickstart, Loki collects logs and Grafana provides a "Quickstart - consolidated logs" dashboard for viewing them. A typical log entry looks like:

```
{
  "timestamp": "2026-03-23T10:15:30Z",
  "level": "INFO",
  "logger": "c.d.c.participant.sync",
  "message": "Transaction committed",
  "transaction_id": "tx-abc123",
  "synchronizer": "global",
  "duration_ms": 1250
}
```

Log Format Is Illustrative

The exact field names and structure depend on Canton's logback configuration. The example above shows the general pattern. Canton's logging is configured via `logback.xml` and supports various appenders and formats.

Important log patterns to alert on:

- `level=ERROR`: Any error indicates a problem requiring investigation.
- `message=*rejection*`: Transaction rejections may indicate contention or configuration issues.
- `message=*timeout*`: Confirmation timeouts may indicate network or participant problems.
- `message=*disconnected*`: Synchronizer disconnections require immediate attention.

22.5 Backups and Disaster Recovery

Canton's state is stored in PostgreSQL databases. Backup and recovery follow standard database practices, with some Canton-specific considerations.

22.5.1 What to Back Up

Data	Backup Strategy
Participant database	Regular PostgreSQL backups (pg_dump or continuous WAL archiving)
Cryptographic keys	Backed up to a secure key vault (never in the database backup alone)
Configuration files	Version-controlled in git
PQS database	Regular PostgreSQL backups
Topology state	Included in participant database

Table 22.2: Backup targets

22.5.2 Recovery Procedure

1. **Restore the database** from backup to a new PostgreSQL instance.
2. **Restore cryptographic keys** from the key vault to the participant's key store.
3. **Start the participant** with the restored database. It will reconnect to the synchronizer.
4. **Catch up**: The participant replays missed messages from the sequencer to bring its state up to date.
5. **Verify**: Run health checks and confirm the active contract set is consistent.

Sequencer Replay

When a participant reconnects after a failure, it requests messages from the sequencer starting from its last known timestamp. The sequencer retains messages for a configurable retention period (typically 7–30 days). As long as the downtime is within this window, the participant can fully recover without a backup.

Key Loss Is Catastrophic

If you lose a participant's private keys and have no backup, the participant cannot decrypt messages or sign confirmations. The parties hosted on that participant become permanently inaccessible. Always back up keys to a hardware security module (HSM) or secure key vault, separate from the database backup.

22.6 Pruning

Over time, the participant's database grows as transactions accumulate. Pruning removes old, no longer needed data to keep the database manageable.

22.6.1 What Gets Pruned

- **Archived contracts**: Contracts that have been consumed and are beyond the retention period.
- **Old transaction records**: Transaction log entries older than the retention period.
- **Sequencer messages**: Delivered messages beyond the sequencer's retention window.

22.6.2 Configuring Pruning

```
// Prune the participant's data up to a specific offset
appProvider.pruning.prune(pruneUpTo = offset)

// Enable automatic pruning (Canton configuration):
// canton.participants.myParticipant {
//   pruning.enabled = true
//   pruning.max-pruning-batch-size = 1000
//   pruning.prune-internally-only = false
// }
```

Pruning and PQS

If you run PQS (Participant Query Store), coordinate pruning with PQS's own data. PQS maintains its own copy of transaction data. Prune the participant only after PQS has processed and stored the data you need for historical queries.

22.7 Upgrades

Canton upgrades fall into two categories: participant upgrades (which can be rolling) and synchronizer upgrades (which may require coordinated downtime).

22.7.1 Participant Upgrades

Participant upgrades are the simpler case:

1. **Review the changelog**: Check for breaking changes, required migrations, and new configuration options.
2. **Back up**: Take a database snapshot before upgrading.
3. **Stop the participant**: Gracefully shut down to complete in-flight transactions.
4. **Upgrade the binary**: Replace the Canton binary or Docker image with the new version.
5. **Run migrations**: Canton automatically migrates the database schema on startup.
6. **Start the participant**: It reconnects to the synchronizer and catches up.
7. **Verify**: Run health checks, confirm connectivity, test a ping.

For zero-downtime upgrades with multi-hosting:

1. Upgrade the passive participant first.
2. Verify the passive participant is healthy.

3. Failover: promote the upgraded participant to active.
4. Upgrade the original active participant.
5. Restore the original active/passive roles.

22.7.2 Synchronizer Upgrades

Synchronizer upgrades affect all connected participants:

1. **Announce**: Notify all participants of the upgrade window.
2. **Drain**: Stop accepting new transactions.
3. **Wait**: Allow in-flight transactions to complete.
4. **Upgrade**: Update the sequencer and mediator.
5. **Resume**: Participants reconnect and resume operations.

Synchronizer Upgrades Require Coordination

Unlike participant upgrades, synchronizer upgrades affect all connected participants. On the Global Synchronizer, upgrades are coordinated by the SV committee and announced well in advance. On a private synchronizer, you coordinate with your own participants.

22.8 Scaling

Canton's architecture scales differently from traditional blockchains because work is distributed across participants rather than replicated on every node.

22.8.1 Party Limits per Validator

Each participant (validator) has practical limits on the number of parties it can host:

- **Contract store size**: More parties means more contracts to store and index.
- **Transaction throughput**: Each party's transactions consume participant resources.
- **Memory**: Active contract caches grow with the number of contracts.

For high-scale deployments, distribute parties across multiple participants. Use Canton's multi-hosting capability to ensure parties on different participants can still transact.

22.8.2 Topology Batching

When registering many parties or updating many topology transactions, batch them to reduce overhead:

```
val parties = (1 to 100).map { i =>     // batch party allocation
  appProvider.parties.enable(s"User-$i")
}
appProvider.topology.vetted_packages     // batch topology proposals
  .propose(appProvider.id, allPackageIds)
```

22.8.3 Horizontal Scaling

Component	Scaling Strategy
JSON API	Stateless; add replicas behind a load balancer
PQS	Add read replicas for query-heavy workloads
Participant	Distribute parties across multiple participants
PostgreSQL	Vertical scaling (bigger instance), read replicas for PQS
Application	Standard horizontal scaling

Table 22.3: Scaling strategies by component

22.9 Performance Tuning

22.9.1 Throughput

Transaction throughput depends on several factors:

- **Sequencer latency**: The time to sequence a message. Database-backed sequencers are faster than BFT sequencers.
- **Transaction size**: Larger transactions (more creates, more exercises) take longer to process.
- **Contention**: Transactions competing for the same contracts cause rejections and retries.
- **Database performance**: PostgreSQL query latency directly impacts participant performance.

22.9.2 Latency Optimization

- **Co-locate participant and database**: Network latency between participant and PostgreSQL should be sub-millisecond.
- **Use connection pooling**: Configure the database connection pool size to match your workload.
- **Tune JVM settings**: Canton runs on the JVM. Heap size, garbage collector choice, and thread pool sizes affect performance.
- **Pre-vet packages**: Ensure all packages are vetted before submitting transactions. Unvetted packages cause immediate rejection.

22.9.3 Traffic Management

On the Global Synchronizer, traffic is metered in bytes sequenced. To manage costs:

- **Minimize transaction size**: Fewer contract fields, shorter text values, avoid unnecessary creates.
- **Batch operations**: Combine multiple operations into a single transaction when possible.
- **Use private synchronizers**: Route internal transactions through a private synchronizer (no traffic fees) and use the Global Synchronizer only for cross-organization transactions.
- **Monitor traffic**: Track sequencer traffic metrics (available through Canton's Prometheus endpoint) to understand your traffic patterns.

Cost Model: Private vs Global

Private synchronizers have zero Canton Coin costs—you pay only for infrastructure. The Global Synchronizer charges per byte sequenced. A typical production architecture uses a private synchronizer for high-volume internal workflows and the Global Synchronizer for cross-organization settlement. This hybrid approach optimizes both cost and trust.

22.10 Summary

- Production Canton deployments move from **Docker Compose to Kubernetes**, using StatefulSets for stateful components and Deployments for stateless ones.
- The **Canton Console** is the primary operational tool, providing health checks, diagnostics, and ping tests for verifying end-to-end connectivity.
- The **DAR lifecycle** involves upload, vetting (a topology transaction that approves packages), and careful unvetting when retiring old versions.
- **Monitoring** uses Prometheus metrics (on a dedicated port, not the Admin API), Grafana dashboards (Participant, Platform, and Infrastructure folders in cn-quickstart), and structured JSON logging via Loki.
- **Backups** cover the participant database, cryptographic keys (stored separately in a key vault), and configuration. Recovery relies on database restore plus sequencer replay.
- **Pruning** removes old archived contracts and transaction records. Coordinate with PQS to avoid losing historical query data.
- **Participant upgrades** can be rolling with multi-hosting. **Synchronizer upgrades** require coordinated downtime across all connected participants.
- **Scaling** distributes parties across multiple participants. The JSON API and PQS scale horizontally; the participant scales by hosting fewer parties.
- **Performance tuning** focuses on co-locating participant and database, managing contention, and routing internal traffic through private synchronizers to minimize Global Synchronizer costs.

Part V

Security & Best Practices

Chapter 23

Daml Security Model

Most smart contract vulnerabilities—reentrancy, integer overflow, unauthorized access—exist because the language allows them and relies on developers to prevent them. Daml takes a different approach: the language and runtime eliminate entire categories of bugs by design. This chapter explains what Daml protects against, how the Canton protocol extends those protections, and what remains the developer's responsibility.

23.1 Secure by Design

Daml is not a general-purpose language bolted onto a blockchain. It was designed from the ground up for multi-party workflows on shared ledgers. Every design decision prioritizes correctness and safety:

- **Pure functional core**: no mutable state, no side effects outside the ledger.
- **Strong static type system**: catches errors at compile time.
- **Deterministic execution**: the same inputs always produce the same outputs.
- **Authorization built in**: the signatory model is part of the language, not an afterthought.

These properties mean that many vulnerabilities common in other smart contract platforms simply cannot occur in Daml.

23.2 No Reentrancy

Reentrancy is the most infamous class of smart contract bugs. In Solidity, an external call can re-enter the calling contract before the first invocation completes, leading to drained funds and corrupted state. The 2016 DAO hack exploited exactly this.

Daml eliminates reentrancy entirely:

Atomic Choice Execution

A Daml choice executes atomically within a single transaction. There are no external calls, no callbacks, no interruptions. The choice body runs to completion, and either all its effects are committed or none are. There is no mechanism for re-entering a choice mid-execution.

When a choice body calls `exercise` on another contract, that exercise happens within the same atomic transaction. The callee cannot "call back" into the caller in any way that would create a reentrancy window. The entire transaction tree is computed first, then validated and

committed as a unit.

```
-- In Daml, this is safe. In Solidity, it would be
-- a classic reentrancy vulnerability.
template Vault
  with
    owner : Party
    balance : Decimal
  where
    signatory owner
    ensure balance >= 0.0

    choice Withdraw : ContractId Vault
      with amount : Decimal
      controller owner
      do
        assertMsg "Insufficient funds"
          (balance >= amount)
        -- No external call can re-enter here.
        -- The state update is atomic.
        create this with
          balance = balance - amount
```

In Solidity, you would need to follow the checks-effects-interactions pattern and add a reentrancy guard. In Daml, the problem does not exist.

23.3 No Silent Integer Overflow

Integer overflow has caused millions of dollars in losses in Solidity contracts (particularly before Solidity 0.8, which added built-in overflow checks). When a `uint256` wraps around from its maximum value to zero (or vice versa), balances become nonsensical.

Daml handles arithmetic safely, though differently from languages with arbitrary-precision integers:

- `Int` is a **signed 64-bit integer** with a range of -2^{63} to $2^{63} - 1$. Arithmetic operations that exceed this range **raise a runtime error** rather than silently wrapping around. This means overflow is detected and rejected, not ignored.
- `Decimal` (a synonym for `Numeric 10`) is a fixed-point number with 38 digits total and 10 decimal places. The `Numeric n` family supports scales from 0 to 37. Overflow during arithmetic or casting also raises a runtime error.

```
-- Daml detects overflow at runtime. Unlike Solidity
-- (pre-0.8), values do not silently wrap around.
test_overflow_detected = do
  let maxInt = 9223372036854775807  -- max Int
  -- maxInt + 1 would raise an ArithmeticError
  -- at runtime, not wrap to a negative number.
  -- The transaction would be rejected.
  pure ()
```

Int Is Not Arbitrary Precision

Daml's `Int` is a 64-bit signed integer, not arbitrary precision. However, unlike Solidity's pre-0.8 behavior where overflow wraps silently, Daml raises a runtime error on overflow. This means overflow bugs cause transaction failures (which are safe) rather than corrupted state (which is dangerous). For values that may exceed 64-bit range, use `Numeric` with

an appropriate scale, or the `BigNumeric` type from the `DA.BigNumeric` standard library for arbitrary-precision decimal arithmetic.

23.4 No Unauthorized Access

In many smart contract platforms, authorization is implemented as ad-hoc `require(msg.sender == owner)` checks sprinkled throughout the code. Forget one check, and anyone can call the function.

In Daml, authorization is structural. The signatory model enforces it at the runtime level:

1. **Creation**: all signatories must authorize. No party can be bound to a contract without their consent.
2. **Exercise**: only the designated controller can exercise a choice. The runtime rejects all other parties.
3. **Archive**: requires signatory authority. A contract cannot be destroyed without authorization.

```
template BankAccount
  with
    bank : Party
    owner : Party
    balance : Decimal
  where
    signatory bank, owner

    -- Only the owner can withdraw. This is enforced
    -- by the runtime, not by an if-check.
    choice Withdraw : ContractId BankAccount
      with amount : Decimal
      controller owner
      do
        assertMsg "Positive amount" (amount > 0.0)
        assertMsg "Sufficient funds"
          (balance >= amount)
        create this with
          balance = balance - amount

    -- Only the bank can freeze. Again, enforced
    -- structurally.
    choice Freeze : ()
      controller bank
      do pure ()
```

There is no way for the owner to call `Freeze` or the bank to call `Withdraw`. The controller declaration is not a suggestion—it is enforced by the Daml runtime and verified by the Canton protocol.

23.5 Front-Door Security: The Daml Model

Think of Daml's security model as the "front door" of your application. It governs what happens when parties interact with contracts through the ledger:

Front-Door Guarantees

The Daml model guarantees:

1. **Consent**: no party is obligated without agreeing.
2. **Authorization**: every action requires the correct parties' authority.
3. **Validity**: `ensure` clauses and `assertMsg` checks prevent invalid state.
4. **Atomicity**: transactions succeed or fail as a unit.
5. **Determinism**: the same inputs always produce the same outputs.

These are enforced by the Daml interpreter and cannot be bypassed by any party, including infrastructure operators.

The front-door model means your business logic is the security boundary. If your templates correctly model who can do what, the runtime enforces it. There is no gap between the specification and the enforcement.

23.6 Back-Door Security: The Canton Protocol

The Canton protocol provides "back-door" security—protection against infrastructure-level attacks that the Daml model alone cannot address:

23.6.1 Sub-Transaction Privacy

Canton does not broadcast transactions to all participants. Each participant sees only the parts of a transaction involving their hosted parties. This is fundamentally different from public blockchains where every node sees every transaction.

Component	What It Sees	What It Cannot See
Participant	Contract data for its hosted parties	Contracts of other participants' parties
Sequencer	Encrypted message envelopes	Any contract payload or party identity
Mediator	Involved participants and confirmation verdicts	Contract payloads (sees hashes, not content)

Table 23.1: Canton component visibility

23.6.2 Integrity Without Global Visibility

Canton achieves consensus without revealing data to all parties. The protocol uses:

- **Cryptographic commitments**: participants commit to transaction results without revealing them.
- **Confirmation by stakeholders**: only the parties involved in a sub-transaction confirm it.
- **Mediator validation**: the mediator verifies that all required confirmations are present without seeing the data itself.

This means that even a compromised synchronizer cannot forge transactions or learn private data. It can only disrupt availability (by refusing to sequence), not integrity or privacy.

23.6.3 Non-Repudiation

Every transaction in Canton is cryptographically signed by the submitting participant. This provides:

- **Proof of submission**: the submitter cannot deny they submitted the transaction.
- **Tamper detection**: any modification to a transaction after submission is detectable.
- **Audit trail**: the full history of who did what is cryptographically verifiable.

23.7 The Authorization Model as a Security Primitive

Chapter 8 covered authorization from a developer's perspective. From a security lens, the authorization model is a formal access control system with properties that most platforms lack:

1. **Mandatory, not discretionary**: authorization is enforced by the runtime. Developers cannot accidentally skip it.
2. **Fine-grained**: each choice has its own controller. Different operations on the same contract can have different access policies.
3. **Composable**: authority flows through exercise chains in a well-defined way. The rules are formal and auditable.
4. **Verifiable**: because authorization is part of the transaction structure, it can be checked by every validating participant.

Authority Is Not a Role System

Daml's authorization model is based on party identity, not roles. There is no "admin" role that automatically gets elevated privileges. If you need role-based access, model it explicitly with templates—for example, an `AdminRole` contract that a party must hold to exercise certain choices.

23.8 Sub-Transaction Privacy as Security

Privacy is not just a compliance feature—it is a security property. Information that is never disclosed cannot be stolen or misused:

- **Minimal disclosure**: parties only learn what they need to know to validate their part of a transaction.
- **Minimal metadata exposure**: unlike public blockchains, Canton does not reveal transaction graphs or party relationships to uninvolved parties. However, transaction metadata such as root hash messages is not end-to-end encrypted, and side-channel information (timing, message size) may still leak (see "Side-Channel Attacks" later in this chapter).
- **Selective transparency**: observers see contracts they are explicitly added to. There is no global "read" capability.

Privacy by Default

In most blockchain platforms, privacy is an afterthought—something you add with zero-knowledge proofs or private channels. In Canton, privacy is the default. You must explicitly choose to share information by adding observers. This "opt-in visibility" model dramatically reduces the attack surface for data exfiltration.

23.9 Comparison: Daml vs Solidity vs Hyperledger Fabric

Property	Daml/Canton	Solidity/EVM	Hyperledger Fabric
Reentrancy	Impossible (atomic choices)	Possible; developer must prevent	Not a typical concern (execute-order-validate)
Integer overflow	Detected (runtime error, no silent wrap)	Detected since 0.8 (reverts); silent wrap pre-0.8	Depends on language
Authorization model	Built into language (signatories)	Ad-hoc `require()` checks	Ad-hoc policy files (MSP)
Privacy	Sub-transaction (default)	None (all data public)	Channel-level (coarse)
Determinism	Guaranteed (pure functional)	Mostly (no randomness)	Not guaranteed (endorsement)
Formal verifiability	High (typed, pure)	Low (mutable state, side effects)	Low (general-purpose languages)
Upgrade safety	Language-level rules	No built-in mechanism	No built-in mechanism

Table 23.2: Security properties across platforms

23.10 What Daml Does NOT Protect Against

Daml's security model is strong, but it has clear boundaries. The following threats are outside its scope:

23.10.1 Off-Ledger Attacks

Daml secures on-ledger interactions. If your application has an HTTP API, a database, or a frontend, those components have their own attack surface. A SQL injection in your backend can compromise data regardless of how secure your Daml contracts are.

23.10.2 Key Compromise

If an attacker obtains a party's private keys, they can submit transactions as that party. Daml cannot distinguish between a legitimate key holder and an attacker with the same key. Key management is covered in Chapter 26.

23.10.3 Social Engineering

Daml ensures that parties must explicitly authorize actions. But if a party is tricked into exercising a choice they do not understand, Daml cannot prevent that. The consent model protects against *technical* unauthorized access, not against human manipulation.

23.10.4 Denial of Service

A compromised synchronizer can refuse to sequence messages, effectively halting the network. Canton's multi-synchronizer architecture mitigates this (parties can move to another synchronizer), but a targeted attack on a single synchronizer can disrupt its participants.

23.10.5 Side-Channel Attacks

While Canton hides contract payloads, certain metadata may leak information:

- **Timing**: when a participant submits transactions may reveal activity patterns.
- **Message size**: the size of encrypted messages may hint at the type of transaction.
- **Network topology**: which participants communicate may reveal business relationships.

These are infrastructure concerns addressed by network hardening, not by the Daml language.

23.10.6 Bugs in Business Logic

Daml eliminates *categories* of bugs but cannot prevent logical errors. If you write a contract that allows withdrawals without checking the balance (omitting the `assertMsg`), Daml will faithfully execute it. The type system catches type errors; it does not catch business logic errors.

Security Is Multi-Layered

Daml and Canton provide strong on-ledger security, but production applications require security at every layer: network, infrastructure, API, key management, and human processes. Chapter 26 covers the operational security layers.

23.11 Summary

- **No reentrancy**: Daml choices execute atomically. There are no external calls, no callbacks, no re-entry windows.
- **No silent integer overflow**: Daml's `Int` is a 64-bit signed integer that raises a runtime error on overflow rather than wrapping silently. `Decimal` (and `Numeric n`) provides fixed-point arithmetic with 38 digits of precision.
- **No unauthorized access**: the signatory/controller model is enforced by the runtime, not by ad-hoc checks.
- **Front-door security**: the Daml model enforces consent, authorization, validity, atomicity, and determinism.
- **Back-door security**: the Canton protocol provides sub-transaction privacy, integrity without global visibility, and non-repudiation.

- **Authorization is mandatory**: unlike platforms where access control is opt-in, Daml enforces it structurally.
- **Privacy by default**: Canton discloses the minimum information needed. Parties must be explicitly added as observers.
- **Daml does not protect against**: off-ledger attacks, key compromise, social engineering, denial of service, side-channel analysis, or bugs in your business logic.
- Security is multi-layered. Daml and Canton handle the on-ledger layer; the rest requires operational security practices.

Chapter 24

Common Pitfalls and Anti-Patterns

Every Daml developer hits the same walls. This chapter catalogs the ten most common pitfalls, shows the buggy code, explains why it fails, and provides the fix. Treat this as a reference—skim it now, return when you hit a problem in your own code.

24.1 Pitfall 1: Missing Signatories

The bug: a party has an obligation but is not a signatory. They can walk away from the contract at any time because they never agreed to it.

```
-- BAD: Bob owes Alice, but Bob is not a signatory.
-- Bob never authorized this debt.
template BadDebt
  with
    creditor : Party
    debtor : Party
    amount : Decimal
  where
    signatory creditor    -- Only creditor signs
    observer debtor
```

Why it is dangerous: Alice can unilaterally create a contract claiming Bob owes her money. Bob never consented. Worse, since Bob is only an observer, he has no obligation on-ledger. The contract is meaningless.

```
-- FIXED: Both parties are signatories.
-- Creation requires both to authorize.
template Debt
  with
    creditor : Party
    debtor : Party
    amount : Decimal
  where
    signatory creditor, debtor
    ensure amount > 0.0
```

Since both parties must be signatories, you need the propose-accept pattern (Chapter 9) to create this contract. That is the point—the debtor must explicitly agree to the debt.

The Rule

If a contract represents an obligation on a party, that party **must** be a signatory. Otherwise the contract is unenforceable.

24.2 Pitfall 2: Visibility Leaks

The bug: making a party an observer when they should not see the data.

```
-- BAD: All employees can see everyone's salary.
template SalaryRecord
  with
    company : Party
    employee : Party
    salary : Decimal
    allEmployees : [Party]
  where
    signatory company
    observer allEmployees  -- Everyone sees everything!
```

Why it is dangerous: every party in `allEmployees` can see every salary record. In a real system, this is a privacy violation. Salary data should be visible only to the employee and their manager.

```
-- FIXED: Only the employee and authorized viewers
-- can see this record.
template SalaryRecord
  with
    company : Party
    employee : Party
    salary : Decimal
    authorizedViewers : [Party]
  where
    signatory company
    observer employee :: authorizedViewers
```

The Rule

Add observers only when there is a legitimate business reason for that party to see the data. Every observer is a potential information leak. Apply the principle of least privilege.

24.3 Pitfall 3: Stale Contract IDs

The bug: holding a `ContractId` that has been archived. Exercising it fails at runtime.

```
-- BAD: The reference goes stale after transfer.
template Inventory
  with
    warehouse : Party
    itemCid : ContractId Item
  where
    signatory warehouse

test_stale_cid = do
  warehouse <- allocateParty "Warehouse"
  supplier <- allocateParty "Supplier"

  itemCid <- submit warehouse do
    createCmd Item with
      owner = warehouse
      name = "Widget"

  -- Archive and recreate the item
  submit warehouse do
    exerciseCmd itemCid UpdateName
```

```
      with newName = "Widget v2"

  -- itemCid is now STALE. The original contract
  -- was archived by UpdateName.
  -- Any code still holding itemCid will fail.
  submitMustFail warehouse do
    exerciseCmd itemCid Archive
```

Why it is dangerous: in multi-step workflows, contract IDs become invalid the moment the contract is consumed by a choice. Code that caches IDs across transactions will break.

```
-- FIXED: Return the new contract ID from choices
-- and use it in subsequent operations.
template Item
  with
    owner : Party
    name : Text
  where
    signatory owner

    choice UpdateName
      : ContractId Item
      with newName : Text
      controller owner
      do create this with name = newName
```

```
-- Always use the returned contract ID, not the old one.
test_use_returned_cid = do
  warehouse <- allocateParty "Warehouse"
  itemCid <- submit warehouse do
    createCmd Item with
      owner = warehouse
      name = "Widget"
  -- Capture the new contract ID from the choice.
  newCid <- submit warehouse do
    exerciseCmd itemCid UpdateName
      with newName = "Widget v2"
  -- Use newCid, not itemCid. newCid is valid.
  submit warehouse do
    exerciseCmd newCid Archive
```

For applications that need to look up contracts without holding a contract ID, use the Participant Query Store (PQS) to query by template fields. The PQS provides a queryable database of active contracts indexed by their field values.

The Rule

Always capture and propagate the contract IDs returned by choices. Never hold stale IDs across transactions. For discovery without a known ID, use the PQS to query active contracts by their fields.

24.4 Pitfall 4: Non-Atomic Operations

The bug: a multi-step workflow that can fail mid-way, leaving the system in an inconsistent state.

```
-- BAD: Two separate submits for a transfer.
-- If the second fails, Alice lost her asset
-- but Bob never got it.
```

```
test_non_atomic_transfer = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  bank <- allocateParty "Bank"

  let bankAlice = actAs bank <> actAs alice
  cid <- submit bankAlice do
    createCmd Asset with
      issuer = bank
      owner = alice
      amount = 100.0

  -- Step 1: Archive Alice's asset
  submit bankAlice do
    archiveCmd cid

  -- Step 2: Create Bob's asset
  -- If THIS fails, Alice's asset is gone
  -- and Bob has nothing!
  let bankBob = actAs bank <> actAs bob
  submit bankBob do
    createCmd Asset with
      issuer = bank
      owner = bob
      amount = 100.0
```

Why it is dangerous: the two `submit` calls are separate transactions. If the second one fails (perhaps Bob is not authorized), the first one has already succeeded. Alice has lost her asset with no recourse.

```
-- FIXED: Use a single choice that does both
-- steps atomically.
template Asset
  with
    issuer : Party
    owner : Party
    amount : Decimal
  where
    signatory issuer, owner

    choice Transfer : ContractId Asset
      with newOwner : Party
      controller owner, newOwner
      do
        -- Both archive (implicit) and create
        -- happen in one atomic transaction.
        create this with owner = newOwner
```

The Rule

Operations that must succeed or fail together should happen in a single choice (single transaction). Never split atomic operations across multiple `submit` calls.

24.5 Pitfall 5: Time-of-Check/Time-of-Use

The bug: fetching a contract to check its state, then exercising it in a separate transaction. Between the two, someone else may have changed it.

```
-- BAD: Check and use in separate transactions.
```

```
test_toctou = do
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"
  bank <- allocateParty "Bank"

  let parties = actAs bank <> actAs alice
  cid <- submit parties do
    createCmd Account with
      bank; owner = alice; balance = 100.0

  -- Step 1: Check balance (query)
  Some acct <- queryContractId alice cid
  let hasEnough = acct.balance >= 50.0

  -- Between here and Step 2, another transaction
  -- could drain Alice's account!

  -- Step 2: Withdraw based on stale check
  if hasEnough then do
    submit alice do
      exerciseCmd cid Withdraw with amount = 50.0
    pure ()
  else pure ()
```

Why it is dangerous: the balance check and the withdrawal happen in different transactions. If Alice submits another withdrawal between the check and the use, the second withdrawal may overdraw the account.

```
-- FIXED: Check and act in the same choice.
template Account
  with
    bank : Party
    owner : Party
    balance : Decimal
  where
    signatory bank, owner

    choice Withdraw : ContractId Account
      with amount : Decimal
      controller owner
      do
        -- Check and use in the same transaction.
        -- No gap for interference.
        assertMsg "Insufficient balance"
          (balance >= amount)
        create this with
          balance = balance - amount
```

The Rule

Always check and act in the same transaction. Use `assertMsg` inside choices, not queries followed by separate exercises.

24.6 Pitfall 6: Unbounded Lists

The bug: using a list field that grows without limit. Every contract operation must read and write the entire contract payload, so large lists kill performance.

```
-- BAD: The history list grows forever.
template Ledger
```

```
  with
    owner : Party
    balance : Decimal
    history : [Text]  -- Grows with every transaction
  where
    signatory owner

    choice RecordTransaction
      : ContractId Ledger
      with entry : Text
      controller owner
      do
        -- After 10,000 transactions, this
        -- contract is enormous and slow.
        create this with
          history = entry :: history
```

Why it is dangerous: Daml contracts are immutable—every archive-and-recreate copies the entire payload. A list with thousands of entries means every operation serializes, transmits, and validates all that data.

```
-- FIXED: Use separate contracts for history.
-- The ledger stays small; entries are independent.
template Ledger
  with
    owner : Party
    balance : Decimal
  where
    signatory owner

    choice RecordTransaction
      : (ContractId Ledger, ContractId LedgerEntry)
      with
        entry : Text
        newBalance : Decimal
      controller owner
      do
        newLedger <- create this with
          balance = newBalance
        entryContract <- create LedgerEntry with
          owner
          entry
        return (newLedger, entryContract)

template LedgerEntry
  with
    owner : Party
    entry : Text
  where
    signatory owner
```

The Rule

Keep contract payloads small. If a collection can grow without bound, store items as separate contracts instead of a list field. Use the PQS (Participant Query Store) for historical queries.

24.7 Pitfall 7: Missing ensure Clauses

The bug: allowing invalid state to be created because there is no `ensure` clause.

```
-- BAD: No validation on creation.
template Invoice
  with
    vendor : Party
    customer : Party
    amount : Decimal
    description : Text
  where
    signatory vendor
    observer customer
    -- No ensure clause!
    -- Allows: amount = -100.0
    -- Allows: description = ""
    -- Allows: vendor == customer
```

Why it is dangerous: without `ensure`, any data that type-checks can be stored on the ledger. Negative invoices, empty descriptions, and self-invoicing all become possible. Every choice that reads this contract must now defensively validate its state.

```
-- FIXED: ensure prevents invalid state at creation.
template Invoice
  with
    vendor : Party
    customer : Party
    amount : Decimal
    description : Text
  where
    signatory vendor
    observer customer
    ensure amount > 0.0
      && description /= ""
      && vendor /= customer
```

The Rule

Every template should have an `ensure` clause that encodes its invariants. If a contract should never have a negative balance, zero quantity, or empty name, enforce it in `ensure`. This is your first line of defense.

24.8 Pitfall 8: Authority Confusion

The bug: misunderstanding how authority flows through exercise chains. A common mistake is assuming that exercising a choice on a contract gives you the signatories' authority for unrelated operations.

```
-- BAD: Assuming nonconsuming exercise
-- grants authority.
template Safe
  with
    owner : Party
    contents : Text
  where
    signatory owner

    nonconsuming choice Peek : Text
```

```
      controller owner
      do pure contents

template Thief
  with
    thief : Party
    safeCid : ContractId Safe
  where
    signatory thief

    choice StealContents : ContractId Safe
      controller thief
      do
        -- Exercise Peek to "get" owner's authority
        _txt <- exercise safeCid Peek
        -- Try to create a new Safe as owner
        safe <- fetch safeCid
        -- FAILS: owner's authority was scoped to
        -- Peek's body, not available here.
        create Safe with
          owner = safe.owner
          contents = "Stolen!"
```

Why it is dangerous: developers coming from other platforms expect that interacting with a contract somehow grants authority over it. In Daml, within a choice body you have the authority of the choice's controllers (actors) plus the signatories of the contract being exercised. However, this authority is *scoped* to that choice body—it does not flow back to the caller. When `StealContents` exercises `Peek`, the `owner`'s authority exists inside `Peek`'s body, but not inside `StealContents`'s body.

```
-- CORRECT: Authority flows from consumed contracts.
template Safe
  with
    owner : Party
    contents : Text
  where
    signatory owner

    -- Consuming choice: authority IS released
    choice Open : (Text, ContractId Safe)
      with
        newContents : Text
      controller owner
      do
        -- owner's authority is available here
        -- because Safe is being consumed.
        newSafe <- create this with
          contents = newContents
        return (contents, newSafe)
```

The Rule

Within a choice body, you have the authority of the choice's controllers plus the signatories of the contract being exercised. This authority is scoped to that choice body—it does not propagate to the caller. If a choice on contract A exercises a choice on contract B, the authority available inside B's choice body comes from B's signatories and B's choice controllers, not from A's.

24.9 Pitfall 9: Upgrade-Unsafe Changes

The bug: making changes to a template that break existing contracts on the ledger.

```
-- VERSION 1: Deployed to production.
template UserProfile
  with
    admin : Party
    user : Party
    name : Text
    email : Text
  where
    signatory admin
    observer user
```

```
-- VERSION 2: BAD upgrade -- removed a field.
-- Existing contracts have 'email' but the new
-- template does not. Deserialization fails.
template UserProfile
  with
    admin : Party
    user : Party
    name : Text
    -- email field removed!
    phone : Text  -- New field added
  where
    signatory admin
    observer user
```

Why it is dangerous: when Canton encounters existing contracts of the old type, it cannot deserialize them into the new type. Removing fields, renaming fields, or changing field types are all breaking changes.

```
-- FIXED: Add new fields as Optional, keep old ones.
template UserProfile
  with
    admin : Party
    user : Party
    name : Text
    email : Text          -- Keep the original field
    phone : Optional Text -- New field, Optional
  where
    signatory admin
    observer user
```

Safe upgrade rules:

1. **Adding optional fields** is safe: existing contracts get `None` for the new field.
2. **Removing fields** is unsafe: existing data cannot be deserialized.
3. **Changing field types** is unsafe: type mismatches cause runtime errors.
4. **Changing signatories** is unsafe: authority requirements change.
5. **Adding choices** is safe: existing contracts gain new capabilities.
6. **Removing choices** is generally unsafe: code that references them breaks.

The Rule

When upgrading templates with active contracts on the ledger, only make additive changes. Never remove or rename fields. Use `Optional` for new fields. Run `dpm upgrade-check` before deploying.

24.10 Pitfall 10: Over-Trusting Observers

The bug: assuming that observers cannot share what they see with unauthorized parties.

```
-- You add Bob as an observer so he can
-- exercise a choice on this contract.
template ConfidentialDeal
  with
    partyA : Party
    partyB : Party
    terms : Text
    confidentialPrice : Decimal
  where
    signatory partyA
    observer partyB
    -- Bob can now see confidentialPrice.
    -- But can he share it?
```

Why it is dangerous: while Canton's privacy model prevents unauthorized parties from seeing contracts on the ledger, there is nothing stopping Bob from copying the data off-ledger and sharing it. Once a party sees data, they can exfiltrate it through non-ledger channels.

```
-- MITIGATION: Separate sensitive data.
template DealTerms
  with
    partyA : Party
    partyB : Party
    terms : Text
  where
    signatory partyA
    observer partyB

-- Only partyA sees the pricing details.
-- partyB is not an observer here.
template DealPricing
  with
    partyA : Party
    dealTermsCid : ContractId DealTerms
    confidentialPrice : Decimal
  where
    signatory partyA
```

The Rule

Treat observers as potential information leaks for the specific data they can see. If a party should not see sensitive fields, do not make them an observer of the contract containing those fields. Split sensitive data into separate contracts with different visibility.

24.11 Quick Reference

#	Pitfall	Fix
1	Missing signatories	Make obligated parties signatories; use propose-accept
2	Visibility leaks	Minimize observers; apply least privilege
3	Stale contract IDs	Capture and propagate returned contract IDs; use PQS for discovery
4	Non-atomic operations	Put related steps in a single choice
5	TOCTOU	Check and act in the same transaction
6	Unbounded lists	Store items as separate contracts
7	Missing ensure	Add invariants to every template
8	Authority confusion	Authority is scoped to the choice body; it does not propagate to the caller
9	Upgrade-unsafe changes	Only make additive changes; use Optional for new fields
10	Over-trusting observers	Split sensitive data into separate contracts

Table 24.1: Pitfalls quick reference

24.12 Summary

- **Missing signatories** allow parties to be bound without consent. If a party has an obligation, they must be a signatory.
- **Visibility leaks** expose data to parties who should not see it. Minimize observers and split sensitive data.
- **Stale contract IDs** cause runtime failures. Always capture and use the contract IDs returned by choices. Use PQS for discovery.
- **Non-atomic operations** can leave the system in an inconsistent state. Use single choices for multi-step workflows.
- **TOCTOU bugs** arise when checking and acting happen in separate transactions. Always check and act together.
- **Unbounded lists** degrade performance. Store growing collections as separate contracts.
- **Missing ensure clauses** allow invalid state on the ledger. Encode invariants in every template.
- **Authority confusion** leads to failed transactions. Authority within a choice body comes from the choice's controllers and the contract's signatories, and is scoped to that choice body.
- **Upgrade-unsafe changes** break existing contracts. Only make additive changes to deployed templates.
- **Over-trusting observers** risks data exfiltration. Once a party sees data, they can share it off-ledger.

Chapter 25

Auditing Daml Contracts

A Daml audit is different from a Solidity audit. You are not hunting for reentrancy, overflow, or gas griefing. Instead, you are verifying that the authorization model is correct, privacy boundaries are appropriate, state transitions are valid, and invariants are complete. This chapter provides a systematic checklist and walks through a full audit of a real contract.

25.1 What a Daml Audit Covers

A Daml audit answers seven questions:

1. Are all signatories correct?
2. Is visibility appropriately restricted?
3. Do all choices validate their inputs?
4. Are contract identifiers correctly managed?
5. Are all state transitions valid?
6. Do ensure clauses cover all invariants?
7. Is the contract safe to upgrade?

The remainder of this chapter provides a detailed checklist for each question, followed by a hands-on audit example.

25.2 Checklist 1: Authorization Review

The authorization model is the foundation of Daml security. Every audit starts here.

25.2.1 Signatory Completeness

For each template, ask: does every party with an obligation appear as a signatory?

Check	Status
Every obligated party is a signatory	_
No party is a signatory without a reason	_
Single-signatory contracts do not represent bilateral agreements	_
Propose-accept is used where multiple signatories are needed	_

Table 25.1: Signatory completeness checklist

25.2.2 Unilateral Action Analysis

Check whether any party can unilaterally take actions that affect others:

```
-- RED FLAG: Bank is sole signatory, but customer
-- has obligations (maintaining minimum balance).
template Account
  with
    bank : Party
    customer : Party
    balance : Decimal
    minBalance : Decimal
  where
    signatory bank     -- Only bank signs!
    observer customer
```

Finding: the bank can unilaterally change the customer's minimum balance, freeze the account, or archive it. The customer has no on-ledger recourse because they are not a signatory.

Recommendation: make the customer a signatory for the account terms. Use a separate `BankAction` template for unilateral bank operations.

25.2.3 Controller Appropriateness

For each choice, verify that the controller is the correct party:

- Does this party have the business authority to perform this action?
- Could this controller abuse the choice to harm other parties?
- Is there a choice that should require multiple controllers but only requires one?

25.3 Checklist 2: Privacy Review

Privacy errors are silent. No test will fail when you add too many observers. You must manually verify that visibility is minimal.

25.3.1 Observer Minimality

For each template, list every observer and justify their presence:

Template	Observer	Justification
`Invoice`	`customer`	Must see the invoice to pay it
`Invoice`	`auditor`	Legitimate business need? Or should auditor use a separate view?

Table 25.2: Observer justification table

25.3.2 Transitive Visibility

Check for unintended visibility through exercise chains:

```
-- A choice that creates a contract visible to
-- a new party. Does the caller realize this?
    choice ShareWithAuditor
      : ContractId AuditableInvoice
      with auditor : Party
      controller vendor
      do
```

```
        create AuditableInvoice with
          vendor; customer; amount
          auditor  -- auditor can now see the amount
```

Ask: when this choice is exercised, does the new observer learn anything they should not? Trace the visibility chain from root to leaf.

25.3.3 Divulgence Awareness

When a contract is used (fetched or exercised) within another party's transaction, that party may learn about the contract's existence. Review each `fetch` and `exercise` call for unintended divulgence.

25.4 Checklist 3: Choice Analysis

Every choice is an entry point into your system. Each one must validate its inputs and enforce its preconditions.

25.4.1 Input Validation

For each choice parameter, ask:

- What values are valid? Are all invalid values rejected?
- Is there a `assertMsg` for each precondition?
- Can the choice be called with adversarial inputs?

```
-- RED FLAG: No input validation.
    choice Transfer : ContractId Account
      with
        newOwner : Party
        amount : Decimal
      controller owner
      do
        -- What if amount is negative?
        -- What if amount exceeds balance?
        -- What if newOwner == owner?
        create this with
          owner = newOwner
          balance = balance - amount
```

Recommendation: add explicit validation for every parameter.

```
-- FIXED: Full validation.
    choice Transfer : ContractId Account
      with
        newOwner : Party
        amount : Decimal
      controller owner
      do
        assertMsg "Amount must be positive"
          (amount > 0.0)
        assertMsg "Insufficient balance"
          (balance >= amount)
        assertMsg "Cannot transfer to self"
          (newOwner /= owner)
        create this with
          owner = newOwner
```

```
          balance = balance - amount
```

25.4.2 Consuming vs Nonconsuming

For each choice, verify:

- If consuming: is it correct that the contract is archived after exercise?
- If nonconsuming: is it safe for the contract to remain active? Could multiple exercises cause problems?
- Could a nonconsuming choice be exercised repeatedly for malicious purposes?

25.5 Checklist 4: Contract Identity and Discovery

Contract Keys in Daml 3.x

Contract keys, a feature available in Daml 2.x, are not supported in Daml 3.x. If you are auditing a codebase written for Daml 2.x that uses contract keys, the checks below still apply. For Daml 3.x projects, contract identity and discovery are handled through returned contract IDs and the Participant Query Store (PQS).

25.5.1 Contract ID Lifecycle

Check	Status
Every choice that archives and recreates returns the new `ContractId`	_
No code holds a `ContractId` across transaction boundaries without refreshing it	_
PQS queries use appropriate filters to find active contracts	_
Contract references stored in other contracts are updated when the target is recreated	_
External systems use the transaction stream (not cached IDs) to track contract lifecycle	_

Table 25.3: Contract identity checklist

25.5.2 Stable Identifiers

If your contracts need a stable business identifier (e.g., an order ID), model it as a template field rather than relying on the system-assigned `ContractId`:

```
-- Use a business identifier field for stable identity.
template Order
  with
    vendor : Party
    orderId : Text  -- Stable business identifier
    status : Text
  where
    signatory vendor
    -- orderId stays constant across archive-recreate
    -- cycles. Use PQS to query by orderId.
```

25.6 Checklist 5: State Machine Review

Many Daml contracts model state machines—an order moves from pending to confirmed to shipped to delivered. The audit must verify that all transitions are valid.

25.6.1 Enumerate States and Transitions

For each template that models a stateful entity, draw the state machine:

1. List all possible values of the state field (or equivalent).
2. For each choice, identify which state transitions it performs.
3. Verify that no choice allows an invalid transition.
4. Check for unreachable states and dead ends.

```
-- State machine: Pending -> Confirmed -> Shipped
-- Verify: Can you go from Shipped back to Pending?
-- Verify: Can you skip Confirmed and go straight
-- to Shipped?
    choice Confirm : ContractId Order
      controller vendor
      do
        assertMsg "Must be Pending"
          (status == "Pending")
        create this with status = "Confirmed"

    choice Ship : ContractId Order
      controller vendor
      do
        assertMsg "Must be Confirmed"
          (status == "Confirmed")
        create this with status = "Shipped"
```

25.6.2 Terminal State Handling

Check that terminal states (e.g., "Delivered", "Cancelled") have no outgoing transitions. A contract in a terminal state should either be archived or have only read-only (nonconsuming) choices available.

25.7 Checklist 6: Ensure Clause Review

The `ensure` clause is the template's first line of defense. Every template should have one.

Check	Status
Every numeric field has range validation	_
Every text field has non-empty validation (if required)	_
Business rules that must always hold are encoded	_
The ensure clause cannot be bypassed by any choice	_
Compound conditions use `&&` (not separate ensure clauses)	_

Table 25.4: Ensure clause checklist

A template without an `ensure` clause should be treated as a finding. Even if the only invariant is "the amount must be positive," make it explicit.

25.8 Checklist 7: Upgrade Safety

If the contract will be deployed to production and upgraded later, verify:

Check	Status
No fields have been removed since the last deployed version	_
No field types have changed	_
New fields are `Optional` with sensible defaults	_
Signatory set has not changed	_
Key structure has not changed	_
New choices do not conflict with existing ones	_
`dpm upgrade-check` passes	_

Table 25.5: Upgrade safety checklist

25.9 Automated Tools

25.9.1 Test Coverage

Run the test suite and verify coverage:

```
dpm build
dpm test
```

Check that tests cover:

- Every choice (both happy path and failure cases).
- Every branch in `ensure` clauses.
- Authorization boundaries (`submitMustFail` for unauthorized parties).
- Edge cases (zero values, empty strings, self-referential operations).

25.9.2 Upgrade Verification

```
dpm upgrade-check
```

This compares the current package against the deployed version and flags breaking changes.

25.10 Manual Review Techniques

25.10.1 Trace Authority Flow

For each choice, trace where authority comes from:

1. Who submits the transaction?
2. Which contracts are consumed (releasing signatory authority)?
3. Does the accumulated authority cover all required signatories for all creates and exercises in the choice body?

Walk through complex exercise chains on paper. Draw the transaction tree showing which contracts are created and consumed at each level.

25.10.2 Enumerate All Possible Transitions

For each template, list every choice that can be exercised and what state it produces. Verify:

- Can any sequence of choices reach an invalid state?
- Is there a choice that should be restricted to certain states but is not?
- Can a choice be exercised after the contract should have been "finalized"?

25.10.3 Adversarial Thinking

For each party involved in the contract, ask: if this party were malicious, what could they do?

- Can a signatory abuse their position to harm the other party?
- Can a controller exercise a choice with adversarial inputs?
- Can an observer learn more than they should?
- Can a non-stakeholder interfere with the contract in any way?

25.11 Writing an Audit Report

An audit report should include:

25.11.1 Report Structure

1. **Executive summary**: what was audited, overall risk level, critical findings count.
2. **Scope**: which templates and modules were reviewed.
3. **Methodology**: which checklists were applied.
4. **Findings**: each issue with severity, description, and recommendation.
5. **Appendix**: full checklist results.

25.11.2 Severity Levels

Severity	Label	Criteria
Critical	**C**	Authorization bypass, unintended fund movement, data corruption
High	**H**	Privacy violation, missing validation allowing invalid state
Medium	**M**	Performance issue, missing ensure clause, weak input validation
Low	**L**	Code quality, naming conventions, missing documentation
Info	**I**	Observations, suggestions, best practice recommendations

Table 25.6: Audit severity levels

25.12 Example Audit: The IOU Contract

Let us walk through a full audit of a simplified `Iou` contract inspired by Chapter 3. We use flat fields here for clarity (the original uses a nested `Cash` record). Here is the contract under review:

```
template Iou
  with
    issuer : Party
    owner : Party
    amount : Decimal
    currency : Text
  where
    signatory issuer
    observer owner

    choice Transfer : ContractId Iou
      with newOwner : Party
      controller owner
      do create this with owner = newOwner

    choice Settle : ()
      controller issuer
      do pure ()
```

25.12.1 Finding 1: Missing Signatory (Critical)

Description: the `owner` is only an observer. The issuer can unilaterally create IOUs claiming they owe the owner money, and can also unilaterally archive them (since the issuer is the sole signatory).

Impact: the issuer could create and destroy obligations without the owner's consent. In a real system, this means the issuer could issue debt and then cancel it before settlement.

Recommendation: make `owner` a signatory so that both parties must consent to the IOU's existence.

```
-- FIXED: Both parties are signatories.
template Iou
  with
    issuer : Party
    owner : Party
    amount : Decimal
    currency : Text
  where
    signatory issuer, owner
```

25.12.2 Finding 2: Missing ensure Clause (High)

Description: there is no `ensure` clause. An IOU with amount zero, negative amount, or empty currency can be created.

Recommendation: add invariants.

```
    ensure amount > 0.0
      && currency /= ""
```

25.12.3 Finding 3: Transfer Input Validation (Medium)

Description: the `Transfer` choice does not validate `newOwner`. An owner could transfer to themselves (creating a duplicate contract) or to the issuer (confusing roles).

Recommendation: add validation.

```
    choice Transfer : ContractId Iou
      with newOwner : Party
```

```
      controller owner
      do
        assertMsg "Cannot transfer to self"
          (newOwner /= owner)
        assertMsg "Cannot transfer to issuer"
          (newOwner /= issuer)
        create this with owner = newOwner
```

25.12.4 Finding 4: Settle Has No Side Effects (Medium)

Description: the `Settle` choice does `pure ()`, which simply archives the IOU. There is no on-ledger record that settlement occurred, no payment leg, and no receipt. This makes dispute resolution difficult.

Recommendation: create a settlement receipt contract.

```
    choice Settle : ContractId SettlementReceipt
      controller issuer
      do
        create SettlementReceipt with
          issuer; owner; amount; currency
```

25.12.5 Finding 5: No Stable Business Identifier (Low)

Description: the IOU template has no field for a human-readable reference (such as an invoice number). External systems must track opaque contract IDs, which change on every transfer. This complicates integration and reconciliation.

Recommendation: add an `iouId : Text` field for correlation with external systems.

```
template Iou
  with
    issuer : Party
    owner : Party
    amount : Decimal
    currency : Text
    iouId : Text  -- stable business identifier
  where
    signatory issuer, owner
```

25.12.6 Audit Summary

#	Severity	Finding
1	Critical	Owner not a signatory; unilateral creation and archival
2	High	No ensure clause; allows invalid amounts and currencies
3	Medium	Transfer choice missing input validation
4	Medium	Settle produces no on-ledger receipt
5	Low	No stable business identifier; complicates external integration

Table 25.7: IOU audit findings

25.13 Summary

- A Daml audit covers seven areas: authorization, privacy, choice validation, contract identity, state machines, ensure clauses, and upgrade safety.

- **Authorization review** verifies that all obligated parties are signatories and that no party can act unilaterally on others' behalf.
- **Privacy review** checks that observers are minimal and that no unintended visibility exists through exercise chains or divulgence.
- **Choice analysis** ensures every choice validates its inputs and that consuming vs non-consuming is correct.
- **Contract identity** review verifies that contract IDs are correctly propagated, business identifiers are stable, and PQS discovery is reliable.
- **State machine review** enumerates all states and transitions, checking for invalid paths and dead ends.
- **Ensure clause review** verifies that every template encodes its invariants.
- **Upgrade safety** confirms that changes are backward-compatible with deployed contracts.
- Use `dpm test` for coverage and `dpm upgrade-check` for compatibility verification.
- Manual review should include authority flow tracing, transition enumeration, and adversarial thinking.
- Audit findings use severity levels: Critical, High, Medium, Low, and Info.

Chapter 26

Production Security

Chapter 23 covered what Daml and Canton protect by design. This chapter covers everything else: the operational security measures required to run a Canton deployment in production. A perfectly designed Daml contract is useless if the participant's TLS is misconfigured, the keys are stored in plaintext, or the API is open to the internet.

26.1 Canton Participant Hardening

A Canton participant is the gateway between your parties and the network. Hardening it is the first priority.

26.1.1 Minimal Attack Surface

- Run the participant with the minimum required features enabled. Disable unused APIs and services.
- Use a dedicated service account with limited OS-level permissions. Never run Canton as root.
- Keep the Canton binary and JVM up to date. Subscribe to Digital Asset's security advisories.
- Remove development tools, debug endpoints, and sample configurations from production deployments.

26.1.2 Configuration Hardening

```
canton {  // canton.conf -- production hardening
  participants {
    myParticipant {
      ledger-api { address = "10.0.1.5"; port = 6865 }  // internal only
      admin-api  { address = "127.0.0.1"; port = 6866 } // localhost only
    }
  }
}
```

Key configuration principles:

- **Bind to internal addresses**: never expose the Ledger API or Admin API on `0.0.0.0`.
- **Admin API on localhost**: the Admin API should only be accessible from the same machine or through a secure tunnel.

- **Separate API and admin ports**: use different ports and network interfaces for the Ledger API (application access) and Admin API (operator access).

26.2 TLS Configuration

All communication between Canton components must be encrypted with TLS. This includes participant-to-synchronizer, API endpoints, and inter-service communication.

26.2.1 Participant-to-Synchronizer TLS

```
canton {
  participants.myParticipant {
    sequencer-client {
      address = "sync.example.com"
      port = 4401
      transport-security {
        enabled = true
        trust-collection-file = "/etc/canton/certs/ca.crt"  // trusted CA
      }
    }
  }
}
```

26.2.2 Ledger API TLS

```
canton {
  participants.myParticipant {
    ledger-api {
      address = "10.0.1.5"
      port = 6865
      tls {
        cert-chain-file = "/etc/canton/certs/ledger-api.crt"
        private-key-file = "/etc/canton/certs/ledger-api.key"
        client-auth = {     // mTLS: require client certificates
          type = require
          admin-client { cert-chain-file = "/etc/canton/certs/admin.crt" }
        }
        trust-collection-file = "/etc/canton/certs/ca.crt"
      }
    }
  }
}
```

Mutual TLS (mTLS)

For production deployments, enable mutual TLS on the Ledger API. This means both the server and client present certificates. The server verifies the client's certificate against a trusted CA, ensuring that only authorized applications can connect. Without mTLS, anyone who can reach the API endpoint can submit transactions.

26.2.3 TLS Certificate Management

Practice	Details
Use a private CA	Do not use self-signed certificates in production. Use a dedicated CA or an internal PKI.
Short-lived certificates	Set certificate lifetimes to 90 days or less. Automate renewal.
Separate certificates	Use different certificates for different services (Ledger API, Admin API, sequencer client).
Revocation	Maintain a CRL or use OCSP. Have a process for revoking compromised certificates.
Key storage	Store private keys in files readable only by the Canton service account, or use an HSM.

Table 26.1: TLS certificate management practices

26.3 Key Management

Canton uses cryptographic keys for party identity, transaction signing, and encryption. Protecting these keys is critical.

26.3.1 Key Types

Canton manages several types of keys:

Key Type	Purpose
Namespace key	Root of trust for the participant's identity
Signing key	Signs transactions submitted by hosted parties
Encryption key	Encrypts messages between participants
TLS key	Secures transport-layer communication

Table 26.2: Canton key types

26.3.2 Key Protection with KMS

For production deployments, protect signing and namespace keys using a Key Management Service (KMS). Canton supports encrypted private key storage where private keys are wrapped by a KMS-hosted symmetric key, ensuring keys are protected at rest:

- Private keys are encrypted (wrapped) using a KMS-hosted key. The raw private key material is never stored in plaintext on disk.
- Canton supports AWS KMS and GCP KMS for key wrapping.
- The KMS key must be a 128-bit symmetric key usable with AES-GCM.

```
canton {
  participants.myParticipant {
    crypto {
      kms { type = aws; region = "us-east-1"; multi-region-key = false }
      private-key-store {
        encryption { type = kms; wrapper-key-id = ${CANTON_KMS_KEY_ID} }
```

```
      }
    }
  }
}
```

KMS Key ID Management

Never hardcode the KMS wrapper key ID in configuration files. Use environment variables or a secrets manager (HashiCorp Vault, AWS Secrets Manager). The example above uses `$CANTON_KMS_KEY_ID` as an environment variable reference.

26.3.3 Key Rotation

Rotate keys periodically and immediately upon suspected compromise:

1. Generate a new key pair (protected by KMS in production).
2. Register the new key in Canton's topology store.
3. Update the key mapping: the party now signs with the new key.
4. Mark the old key as retired but retain it for verification of historical transactions.
5. After a grace period, revoke the old key.

```
val newKey = myParticipant.keys.secret
  .generate_signing_key("party1-signing-v2")  // generate new key
myParticipant.topology.owner_to_key_mappings  // map party to new key
  .authorize(TopologyChangeOp.Replace, party1,
    Some(newKey.fingerprint), SigningKeyUsage.ProtocolOnly)
```

26.3.4 Key Backup

- Back up the namespace key. Losing it means losing the participant's identity permanently.
- Store backups in a separate, secure location (offline, encrypted, access-controlled).
- Test backup restoration regularly. An untested backup is not a backup.
- For KMS-protected keys, ensure the KMS wrapper key is backed up according to the cloud provider's key management best practices.

26.4 Authentication with Keycloak

Canton's Ledger API uses OAuth 2.0 / OpenID Connect for application authentication. Keycloak is the recommended identity provider.

26.4.1 Realm Configuration

Create a dedicated realm for Canton:

- **Realm name**: `canton-production`
- **Token lifetime**: access tokens should expire quickly (5–15 minutes). Use refresh tokens for longer sessions.

- **Brute-force protection**: enable with lockout after 5 failed attempts.
- **Password policy**: minimum 12 characters, require complexity.

26.4.2 Client Configuration

Each application that connects to the Ledger API needs a Keycloak client:

Setting	Value
Client protocol	`openid-connect`
Access type	`confidential`
Service accounts	Enabled (for backend services)
Direct access grants	Disabled (for security)
Valid redirect URIs	Specific application URLs only

Table 26.3: Keycloak client configuration

26.4.3 Scope and Claim Mapping

Map Keycloak users to Daml parties using custom claims:

```
{
  "sub": "app-backend-001",
  "aud": "canton-ledger-api",
  "actAs": ["Alice::1220abcd..."],
  "readAs": ["PublicParty::1220abcd..."],
  "exp": 1700000000
}
```

The `actAs` claim grants authority to submit transactions as the listed parties. The `readAs` claim grants read-only access. Both must contain fully qualified party identifiers.

Token Scope Minimization

Each application should receive tokens with only the parties it needs. A payment service should have `actAs` for the payment party only, not for all parties on the participant. Follow the principle of least privilege for token claims.

26.4.4 Canton Authentication Configuration

```
canton {
  participants.myParticipant {
    ledger-api {
      auth-services = [{
        type = jwt-rs-256-jwks
        url = "https://keycloak.example.com/realms/canton-production/protocol/openid-connect/certs"
      }]
    }
  }
}
```

26.5 Network Segmentation

Not everything should be reachable from everywhere. Proper segmentation limits the blast radius of a compromise.

26.5.1 Network Zones

Zone	Components	Access
Public	Load balancer, WAF	Internet-facing
DMZ	API gateway, Keycloak	Public zone only
Application	Backend services, PQS	DMZ only
Data	Canton participant, database	Application zone only
Management	Admin API, monitoring	Operator VPN only

Table 26.4: Network zone segmentation

26.5.2 What Should Not Be Exposed

- **Admin API**: never expose to the application network. Access via VPN or bastion host only.
- **Database ports**: Canton's internal database should not be reachable from the application zone.
- **Canton console**: operator access only, through a secure jump host.
- **Sequencer ports**: the participant's sequencer client connection should be on a dedicated, firewalled network segment.

26.6 API Security

The Ledger API and any custom HTTP APIs built on top of Canton need standard API security measures.

26.6.1 Rate Limiting

Protect against abuse and denial-of-service:

- **Per-client rate limits**: limit the number of commands per second per authenticated client.
- **Global rate limits**: set an upper bound on total API throughput.
- **Backpressure**: Canton's command service supports command deduplication and flow control. Configure appropriate queue sizes.

26.6.2 Input Validation

Even though Daml validates contract data, your API layer should also validate:

- **Request size limits**: reject oversized payloads before they reach Canton.

- **Field validation**: validate party identifiers, contract IDs, and choice arguments at the API layer.
- **Command deduplication**: use the Ledger API's deduplication feature to prevent duplicate submissions.

26.6.3 Authentication Enforcement

- Every API request must carry a valid bearer token.
- Reject requests with expired, malformed, or revoked tokens immediately.
- Log all authentication failures with source IP and attempted party.

26.7 Security Monitoring

Monitoring is your early warning system. Without it, you will not know you have been compromised until it is too late.

26.7.1 Transaction Monitoring

Watch for suspicious patterns:

- **High-volume submissions**: a sudden spike in commands from a single party may indicate a compromised key.
- **Failed authorizations**: a burst of authorization failures suggests an attacker probing access boundaries.
- **Unusual timing**: transactions at unusual hours or in unusual patterns may indicate automated attacks.
- **Large value transfers**: flag transactions that exceed business-defined thresholds.

26.7.2 Infrastructure Monitoring

- **TLS certificate expiry**: alert well before certificates expire.
- **Key usage**: monitor HSM audit logs for unexpected signing operations.
- **System resources**: watch CPU, memory, disk, and network. Resource exhaustion can indicate a DoS attack.
- **Configuration changes**: detect and alert on changes to Canton configuration, network rules, and access policies.

26.7.3 Log Management

- Ship logs to a centralized, tamper-evident log management system.
- Retain logs for the period required by your compliance framework (typically 1–7 years).
- Ensure logs do not contain sensitive data (contract payloads, PII). Canton's logging can be configured to exclude payload data.
- Set up alerts for error patterns, not just individual errors.

26.8 Incident Response

Have a plan before you need one.

26.8.1 Key Compromise Response

If a party's signing key is compromised:

1. **Immediately rotate the key**: generate a new key pair and update the topology mapping.
2. **Revoke the compromised key**: remove it from the topology store so it can no longer sign transactions.
3. **Audit recent transactions**: review all transactions signed with the compromised key since the estimated compromise time.
4. **Notify counterparties**: inform parties who transacted with the affected party during the compromise window.
5. **Assess damage**: determine which transactions may have been fraudulent and initiate dispute resolution.

```
// Emergency key rotation via Canton console
val newKey = myParticipant.keys.secret
  .generate_signing_key("party1-emergency")       // 1. generate new key
myParticipant.topology.owner_to_key_mappings
  .authorize(TopologyChangeOp.Remove, party1,
    Some(compromisedKey.fingerprint),
    SigningKeyUsage.ProtocolOnly)                   // 2. revoke old key
myParticipant.topology.owner_to_key_mappings
  .authorize(TopologyChangeOp.Add, party1,
    Some(newKey.fingerprint),
    SigningKeyUsage.ProtocolOnly)                   // 3. activate new key
```

26.8.2 Synchronizer Compromise

If you suspect the synchronizer is compromised:

1. **The synchronizer cannot forge transactions**: Canton's protocol ensures that the synchronizer cannot create valid transactions on behalf of parties.
2. **The synchronizer can disrupt availability**: it can refuse to sequence messages or delay them.
3. **Migration**: Canton supports moving parties to a different synchronizer. Initiate migration to a trusted synchronizer.
4. **Evidence preservation**: preserve sequencer logs for forensic analysis.

26.9 Compliance Considerations

26.9.1 GDPR and Data Privacy

Canton's sub-transaction privacy model is a natural fit for GDPR:

- **Data minimization**: parties only see data they are entitled to (Article 5(1)(c)).
- **Purpose limitation**: observers are added for specific business reasons, not blanket access.
- **Right to erasure**: Canton supports contract pruning, which removes historical contract data from the participant's database. However, cryptographic commitments (hashes) may remain, and other participants who were stakeholders retain their own copies. Full erasure across all participants requires coordination. Consult legal counsel for your jurisdiction's interpretation of "erasure" in a distributed ledger context.
- **Data portability**: parties can export their transaction history from their participant node.

Pruning and Right to Erasure

Canton's pruning feature removes historical contract data from the participant's database. However, other participants who were stakeholders in those contracts retain their own copies. Full erasure across all participants requires coordination. Consult legal counsel for your jurisdiction's specific requirements.

26.9.2 Audit Trails

Canton provides a complete, cryptographically verifiable audit trail:

- Every transaction is signed by the submitting participant.
- The sequencer provides a total order of events with timestamps.
- Participants can independently verify the correctness of their transaction history.
- The PQS (Participant Query Store) can serve as the basis for compliance reporting.

26.9.3 Regulatory Reporting

Use the PQS to build compliance reports:

```
# Query transactions in a date range for regulatory reporting
psql -h localhost -d pqs -U pqs_reader -c "
  SELECT event_id, event_type, template_id, created_at
  FROM events WHERE created_at >= '2026-01-01'
  AND created_at < '2026-04-01' ORDER BY created_at"
```

26.10 Resilience Under Attack

26.10.1 DoS Protection

Canton's architecture provides inherent resilience:

- **Sequencer rate limiting**: the sequencer can enforce per-participant message rate limits.
- **Command deduplication**: duplicate commands are rejected without processing.
- **Flow control**: Canton's gRPC-based APIs support backpressure.
- **Multi-synchronizer**: if one synchronizer is under attack, parties can operate through an alternative.

26.10.2 Traffic Limits

Configure per-participant traffic limits at the synchronizer level:

```
// Synchronizer configuration: traffic limits
canton {
  synchronizers {
    mySynchronizer {
      parameters {
        traffic-control {
          max-rate-per-participant = 100  // per second
          max-burst = 50
        }
      }
    }
  }
}
```

26.11 Production Security Checklist

Use this checklist before going live:

Category	Check	Done
TLS		
	TLS enabled on all participant connections	_
	mTLS on Ledger API	_
	Certificates from a trusted CA (not self-signed)	_
	Certificate rotation automated	_
Keys		
	Signing keys encrypted with KMS	_
	Namespace key backed up securely	_
	Key rotation procedure documented and tested	_
	No plaintext keys in configuration files	_
Authentication		
	Keycloak (or equivalent) configured	_
	Token lifetimes appropriate (5–15 min)	_
	Per-application token scopes minimized	_
	Service accounts use client credentials flow	_
Network		
	Admin API not exposed to application network	_
	Database not accessible from DMZ	_
	Firewall rules restrict inter-zone traffic	_
	Sequencer connection on dedicated segment	_
API		
	Rate limiting configured	_
	Input validation at API gateway	_
	Command deduplication enabled	_
	All endpoints require authentication	_
Monitoring		
	Centralized logging configured	_
	Alerts for authentication failures	_
	Alerts for certificate expiry	_
	Transaction anomaly detection	_
Incident Response		
	Key compromise runbook documented	_
	Synchronizer migration procedure tested	_
	Backup restoration tested	_
	Contact list for counterparties current	_
Compliance		
	Pruning policy defined	_
	Audit trail retention meets requirements	_
	Logs exclude sensitive data	_
	Regulatory reporting queries verified	_

Table 26.5: Production security checklist

26.12 Summary

- **Harden the participant**: bind to internal interfaces, disable unused APIs, run with minimal privileges.
- **TLS everywhere**: enable TLS on all connections. Use mTLS on the Ledger API. Automate certificate rotation.
- **Protect keys with KMS**: signing and namespace keys should never exist in plaintext on disk. Use Canton's encrypted private key storage with AWS KMS or GCP KMS.
- **Key rotation**: rotate keys periodically and immediately upon compromise. Retain old keys for historical verification.
- **Authenticate with Keycloak**: use OAuth 2.0 with short-lived tokens and minimal scopes per application.
- **Segment the network**: separate public, DMZ, application, data, and management zones. The Admin API belongs in the management zone.
- **Secure APIs**: enforce rate limiting, input validation, command deduplication, and authentication on every endpoint.
- **Monitor for security**: watch for authorization failures, transaction anomalies, certificate expiry, and configuration changes.
- **Plan for incidents**: document and test key compromise, synchronizer migration, and backup restoration procedures.
- **GDPR alignment**: Canton's sub-transaction privacy supports data minimization. Use pruning for data erasure, but coordinate across participants.
- **Use the checklist**: review every item before going to production. Security is not a feature you add later.